# [Experiment, Analysis, and Benchmark] HCT-QA: A Benchmark for Question Answering on Human-Centric Tables


Mohammad S. Ahmad
*QCRI, HBKU*
Doha, Qatar
mohammadshahmeerah@hbku.edu.qa

Zan Naeem*
*Meta*
Palo Alto, USA
znaeem@meta.com

Michael Aupetit
*QCRI, HBKU*
Doha, Qatar
maupetit@hbku.edu.qa

Ahmed Elmagarmid
*QCRI, HBKU*
Doha, Qatar
aelmagarmid@hbku.edu.qa

Mohamed Eltabakh
*QCRI, HBKU*
Doha, Qatar
meltabakh@hbku.edu.qa

Xiaosong Ma*
*MBZUAI*
Abu Dhabi, UAE
Xiaosong.Ma@mbzuai.ac.ae

Mourad Ouzzani
*QCRI, HBKU*
Doha, Qatar
mouzzani@hbku.edu.qa

Chaoyi Ruan*
*NUS*
Singapore
ruancy@comp.nus.edu.sg

Hani Al-Sayeh
*QCRI, HBKU*
Doha, Qatar
halsayeh@hbku.edu.qa



***Abstract*—Tabular data embedded in PDF files, web pages, and other types of documents is prevalent in various domains. These tables, which we call human-centric tables (*HCTs* for short), are dense in information but often exhibit complex structural and semantic layouts. To query these HCTs, some existing solutions focus on transforming them into relational formats. However, they fail to handle the diverse and complex layouts of HCTs, making them not amenable to easy querying with SQL-based approaches. Another emerging option is to use Large Language Models (LLMs) and Vision Language Models (VLMs). However, there is a lack of standard evaluation benchmarks to measure and compare the performance of models to query HCTs using natural language. To address this gap, we propose the Human-Centric Tables Question-Answering extensive benchmark (*HCT-QA*) consisting of thousands of HCTs with several thousands of natural language questions with their respective answers. More specifically, HCT-QA includes 1,880 real-world HCTs with 9,835 QA pairs in addition to 4,679 synthetic HCTs with 67.7K QA pairs. Also, we show through extensive experiments the performance of 25 and 9 different LLMS and VLMs, respectively, in an answering HCT-QA's questions. In addition, we show how finetuning an LLM on HCT-QA improves F1 scores by up to 25 percentage points compared to the off-the-shelf model. Compared to existing benchmarks, HCT-QA stands out for its broad complexity and diversity of covered HCTs and generated questions, its comprehensive metadata enabling deeper insight and analysis, and its novel synthetic data and QA generator.**



***Index Terms*—Human-Centric Tables, LLMs, Data Querying**


## I. Introduction

Answering natural language (NL) questions over structured and semi-structured data is a fundamental problem [1]–[4]. The task of **Table QA** focuses on retrieving or computing answers to NL questions grounded in tabular data [5]–[10]. Early studies used primarily relational tables from Wikipedia and widely used databases [6], [11], [12]. Over time, research has evolved in two directions: (1) Direct question answering using neural models [5], [13], [14], and (2) translating NL questions into SQL queries (NL-to-SQL) [7]–[10], [15]. Despite increasing attention to this area, existing efforts remain concentrated on relational or relational-like table structures.

Tabular data embedded in PDFs files, web pages, and other types of documents is widespread across many domains. Figure 1 illustrates examples of such data obtained from the Qatar National Planning Council [16], scientific publication archives [17], the US Census [18], and the Pakistan Bureau of Statistics [19]. These tables employ intricate structural and visual layouts to convey large volumes of information in a concise representation to support strategic decision-making.

Unlike traditional relational tables, which have a simple layout, document-embedded tables are primarily designed for human readability rather than automated processing. We refer to this form of tabular data as **Human-Centric Tables (HCTs)**. HCTs represent a distinct data modality, characterized by intricate layouts, rich semantic content, and significant practical value. However, systematically processing HCTs for tasks such as information extraction, interpretation, and question answering remains a major challenge.

With the rapid advances in multimodal document understanding [20]–[23], the scope of Table QA is progressively extending beyond relational tables [8], [10], [24]. This evolution warrants the need for not only systems capable of robust question answering over HCTs, but also methodologies and benchmarks that enable rigorous evaluation and identification of existing limitations of these systems while answering questions on HCTs. Although a few benchmarks have been recently introduced, they suffer from key shortcomings—most notably, the absence of metadata-driven analyses (i.e., analyses incorporating HCTs and question characteristics), and limited scalability due to the reliance on extensive manual effort.

**Properties and Challenges of HCTs.** HCTs exhibit highly diverse structural features designed for human interpretation.

* Work done while at QCRI.

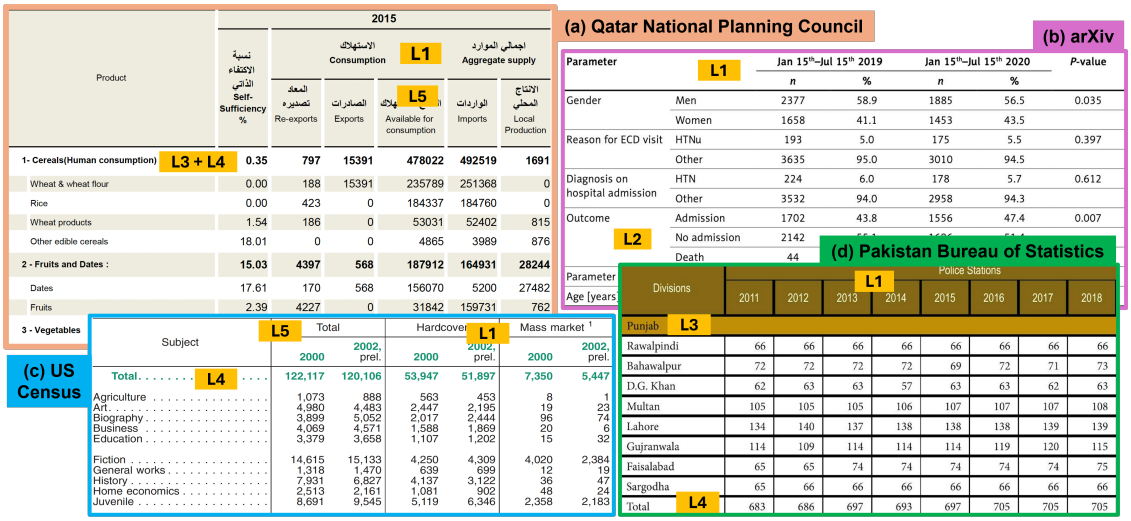



Fig. 1. Examples of Human-Centric Tables (HCTs) from different real-world data sources exhibiting various levels of structural and semantic complexity including column nesting (L1), row nesting (L2), row group labels (L3), row aggregation (L4), and column aggregation (L5)

Figure 1 showcases common structural properties observed in HCTs including *column nesting* (L1), *row nesting* (L2), and *row group labels* (L3), as well as *embedded aggregations* that may appear *row-wise* (L4) or *column-wise* (L5). Visual cues-such as variations in text or row color (e.g., L3 in Table (d) and L4 in Table (c)), bold or emphasized text (e.g., L4 in Table (a) and Table (c)), and the positioning of elements within cells (e.g., L2 in Table (b)) convey additional semantic information.

These structural and visual complexities make direct question answering with neural QA models challenging. Also, as HCTs are not inherently relational, SQL queries cannot be executed on them directly. Existing efforts, such as AutoTables [25], have attempted to transform HCT-like tables into relational formats. However, such approaches are limited to relatively simple layouts, often struggle on complex real-world HCTs. Consequently, the pipeline that first converts HCTs to relational tables and then applies NL to SQL transformations remains fragile and error-prone [8], [10], [24], [26].

**New Opportunities with LLMs and VLMs.** Recent advances in Large Language Models (LLMs) and Vision Language Models (VLMs) pave the way for a new *LLM-oriented* approach for querying HCTs. This approach can (1) efficiently handle HCTs alongside other human-centric data formats (e.g., charts and images), (2) avoid the complex and error-prone NL-to-SQL pipeline, and (3) directly understand NL questions. However, a systematic evaluation to identify the strengths, weaknesses, and potential improvements of LLM-oriented approaches remains largely an uncharted area. To address this gap, we propose HCT-QA, a novel benchmark designed to assess LLMs and VLMs as the engines for querying HCTs. HCT-QA can be also used to evaluated other types of data processing engines designed to answer NL questions on HCTs.

**Challenges and Contributions.** Devising the proposed benchmark involves several challenges: (1) Collecting and generating HCTs with an extensive diversity of complex layouts (Figure 1). (2) Constructing an extensive variety of NL questions ranging from simple selection to complex aggregation and ranking questions. (3) Obtaining accurate ground truth answers, and developing meaningful evaluation metrics to compare them to LLMs' answers. (4) Exploring the impact of LLM-specific strategies such as finetuning on answering questions on HCTs. The key contributions of HCT-QA are:

- A large set of HCTs and question-answer pairs (QA) consisting of 1,880 HCTs from four real-world data sources spanning diverse domains. HCTs are provided in various formats including images, CSV, HTML, and Markdown. We also provide 9,835 QA pairs crafted by experts and verified by annotators (Section III-A).
- Extensive manually-obtained metadata of the HCTs and QA pairs to capture the most common properties of HCTs and questions. This metadata allows for a deeper understanding of LLM/VLM performance and a better identification of performance gaps (Section III-B).
- A novel synthetic HCT generator to complement the real-world datasets. We used the generator to produce 4,679 HCTs and 67,747 template-based QA pairs across seven semantic domains. The generator offers flexible configuration making it a valuable and practical resource for the research community (Section IV).
- An extensive evaluation of 25 LLMs and 9 VLMs of different sizes ranging from $3B$ to $100B+$ parameters. Our evaluation reveals that large closed-weight models perform the best overall, the promise of processing HCT images directly using VLMs, and the effectiveness of finetuning some models using HCT-QA (Section V).

Our benchmark is publicly available on HuggingFace [31]. All experimentation code and the synthetic HCT generator are publicly available on our GitHub repository [32].

## II. Related Work

Several research and commercial efforts addressed document understanding and information extraction from complex documents [20], [21], [23]. However, none of these fully

TABLE I
COMPARISON OF MAJOR TABLE QA BENCHMARKS WITH HCT-QA.

| Benchmark Name | Data Type | Num HCT Tables | Table Modalities | Num Sources | Num QA Pairs | Table Metadata | QA Metadata | Avg. Questions Per Table | Question Syntactic Similarity |
|---|---|---|---|---|---|---|---|---|---|
| HiTab [27] | Tables | 3,597 | JSON | 3 | 10,672 | ✗ | ✗ | 3 | 0.45 |
| MultiHiertt [28] | Tables + Text | 9,842 | HTML | 2 | 10,440 | ✗ | ✗ | N/A | 0.23 |
| TableVQA-Bench [29] | Tables | 205 | HTML, Image | 1 | 250 | ✗ | ✗ | 1.2 | 0.186 |
| TAT-QA [30] | Tables + Text | 2,757 | 2D List | 1 | 12,650 | ✗ | ✗ | N/A | 0.2879 |
| **HCT-QA (Ours)** | **Tables** | **6,559** | **HTML, CSV, Image, MD** | **4 + Synthetic** | **77,582** | ✓ | ✓ | **11** | **0.325** |

address the core question *What are the opportunities and challenges of using LLM-based approaches to query HCTs?*

**Table extraction techniques** in several commercial tools [33]–[36] and recent research [37]–[44] focus on extracting tables similar to HCTs from PDFs into machine-readable formats such as CSV and Markdown. However, they do not provide end-users any systematic support for querying, analyzing, or discovering the extracted tables. Also, our analysis shows that VLMs can bypass this extraction step by operating directly on the image version of HCTs.

**Table QA approaches** focus on relational tables as they use NL-to-SQL to answer the questions [45]–[50]. However, SQL queries cannot be executed on tables with complex layouts like HCTs. As mentioned above, AutoTables [25] fail to achieve high accuracy on complex real-world HCTs. Other approaches tried to perform repairs and transformations on non-relational tables but still do not present a means to convert a HCT into a relational table [51], [52]. Such a multi-step, database-oriented pipeline-spanning HCTs extraction, schema discovery, and NL-to-SQL transformation introduces several error-prone phases and could be an overkill for practical applications. Other non-NL-to-SQL related approaches such as direct inference using deep learning models have again been restricted to handling relational tables [5], [13], [53], [54].

**Table QA benchmarks** mostly focus on flat Wikipedia tables which, when compared to HCTs, are significantly simpler [12], [55]–[61]. FinQA [62] consist of tables from the finance domain but specifically exclude tables with complex nested structures. There are also several benchmarks that cover only one source or domain and have a very small number of QA pairs and tables such as AIT-QA [63] with only 515 QA pairs on 116 tables and TableVQA-Bench [29] with only 250 QA pairs on 205 HCTs. Some benchmarks focus on multi-modal QA where questions are on text, tables, images or other modalities [28], [64]–[68]. Compared to these benchmarks, HCT-QA focuses on complex-layout HCTs from diverse real-world sources, provides tens of thousands of QA pairs on thousands of HCTs without requiring metadata, and provides a synthetic HCT and QA pair generator for scalability.

In Table I, we present the key distinguishing dimensions of the related benchmarks. It can be observed that each of them suffers from key limitations like a limited number of HCTs or QA pairs, a small number of questions per table, etc. Also, as the table shows, the most distinctive advantage of HCT-QA lies in its comprehensive table- and QA-level metadata, which enables fine-grained and metadata-driven analysis.

In addition, to assess the diversity of questions, we introduced two statistics: the *average number of questions* per table and the *question syntactic similarity*. The syntactic similarity is computed at the table level (for tables with more than one question) by calculating the pairwise word-level Jaccard similarity between all question pairs, followed by averaging the results. As shown, HCT-QA exhibits a higher average number of questions per table and greater lexical diversity.

Zooming-in on HiTab [27] (the closest to HCT-QA), we see that it includes HCTs from three sources and provides only 10,672 QA pairs. In contrast, HCT-QA encompasses nearly twice as many HCTs drawn from a broader range of sources and offers over seven times more QA pairs. Also, HCT-QA includes the visual representations (images) of the HCTs, thereby enabling evaluation of VLMs. Moreover, a closer examination of samples from existing benchmarks revealed several data quality issues. For instance, after manually inspecting 206 randomly selected QA samples from the HiTab benchmark, we found that 22% of the ground-truth answers were incorrect, 8% of the questions were irrelevant to their associated tables, 37% contained minor grammatical errors, and 12% included typographical mistakes. In developing HCT-QA, we placed a strong emphasis on ensuring data quality through a comprehensive and rigorous verification process.

## III. REAL-WORLD DATASETS

In this section, we present the real-world dataset. Each of its element consists of a quadruplet in the form of $(T_{HCT}, Q_{NL}, A, M)$, where $T_{HCT}$ is an HCT object, $Q_{NL}$ is a NL question, $A$ is the ground truth answer, and $M$ is rich metadata about $T_{HCT}$ and $Q_{NL}$. In the following, we first present the details of the HCTs collected from thousands of PDFs across diverse domains (Section III-A), and then present the details of the QA collection process (Section III-B). Figure 2 illustrates the dataset preparation process.

### A. Real-world HCTs

**Data Sources.** We collected thousands of PDFs from diverse sources, including Qatar National Planning Council (QNPC) [16] Research Papers from *ArXiv.org*, *biorxiv.org*, and *medarch.org*, US Census covering PDF reports from the US Census Bureau [18] and Pakistan Census from the official website of the Pakistan government [19]. These sources cover various domains including education, land usage, machine learning, data management, finance, astrophysics, materials science, cryptography, astrophysics, medicine, and security. In total, we extracted **1,880** HCTs.

**Extraction and Transformation.** We employed the Landing AI tool [69] to generate an HTML representation of the

TABLE II
DEFINITIONS OF HCT (TABLE) PROPERTIES.

| Property Name | Definition |
|---|---|
| Column Nesting | Headers/column names are spread across more than one level (row). |
| Balanced * | The depth of each branch within the nesting is the same throughout the table. |
| Unbalanced | The depth of each branch within the nesting is not the same. |
| Symmetric* | Child column names repeat uniformly across table columns where nesting is present. |
| Asymmetric* | Child column names are different across the table in columns where nesting is present. |
| Row Nesting | A cell spans multiple rows but is displayed as one merged cell. |
| Balanced* | The depth of each row branch is the same. |
| Unbalanced | The depth of each row branch is not the same. |
| Symmetric* | Child cell values repeat across the table uniformly in rows where nesting is present. |
| Asymmetric* | Child cell values are different across the table in rows where nesting is present. |
| Col. Aggregation | Columns that aggregate all or a subset of other columns. |
| Global* | A column that aggregates all other numerical columns in the table (excluding local-group aggregation columns). |
| Local* | A column that aggregates a subset of numerical columns in the table. |
| Explicit* | An aggregation column where its name clearly states it's an aggregation (e.g., total). |
| Implicit* | An aggregation column where its name is ambiguous about whether it's an aggregation. |
| Row Aggregation | Rows that aggregate all or a subset of other rows. |
| Global* | A row aggregating all other numerical rows in the table (excluding local-group aggregation rows). |
| Local* | A row that aggregates a subset of numerical rows in the table. |
| Explicit* | A row where some textual cell value clearly states it's an aggregation (e.g., total) |
| Implicit* | A row where all textual cell values are ambiguous about whether it's an aggregation. |
| Others | |
| Row Group Label | A row that acts as a heading/title for either all or a subset of rows below or above it. |
| Split Header Cell | One cell that is split diagonally (visually) and contains two values. |
| Stand. Rel. Table* | Table displays none of the other HCT properties (nesting, aggregations, labels). |

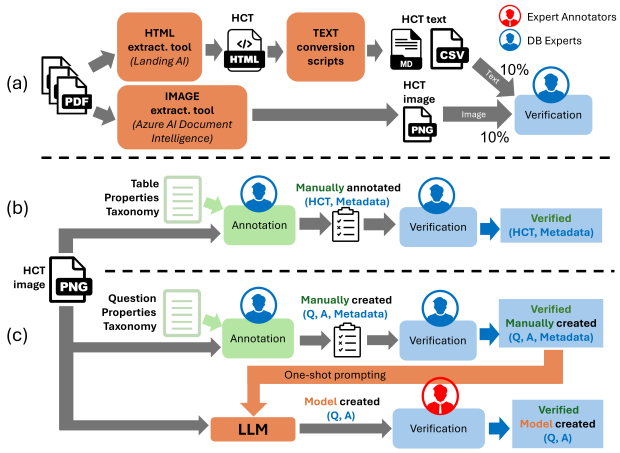



Fig. 2. Preparing the real-world dataset. (a) Extracting HCTs. (b) Annotating HCTs with table properties. (c) Generating QA pairs with question properties.

HCTs, from which we further derived the corresponding CSV and Markdown formats. As part of the extraction pipeline, we conducted manual verification on over 10% of the extracted HCTs to ensure the tool's ability to capture the structural complexities such as column nesting, row nesting, row groupings, and embedded aggregations. The verified sample demonstrated 100% accuracy. Also, we utilized Azure AI Document Intelligence [34] to extract HCT images. A similar 10% manual verification confirmed the absence of errors. Figure 3(a) presents statistics on each source.

**Characterization of HCT Properties.** Across the broad range of HCTs we investigated, we observed that certain structural layouts are highly prevalent such as column nesting, row nesting, and embedded aggregations whereas others, like nested tables, occur less frequently and are considered outliers. In this study, we focus on common structural patterns and exclude HCTs exhibiting rare or irregular layouts. The dominant layout patterns (referred to as *properties*) are summarized in Table II. The top-level properties are highlighted in green, while additional sub-properties are specified for cases where finer distinctions are expected to influence the LLM/VLM's comprehension of HCTs, and hence impact the QA accuracy. For instance, if an HCT exhibits column nesting (L1 in Figure 1), we further specify whether the nesting is *balanced*—meaning all child nodes occur at the same hierarchical depth—or *unbalanced*, where nesting levels vary. Referring to Figure 1, Tables (b), (c), and (d) demonstrate balanced column nesting, whereas Table (a) illustrates an unbalanced column nesting. We also record whether the nesting is *symmetric*—where column names and layouts are consistent across parent nodes—or *asymmetric*, where they differ. A similar level of detail is captured for the row nesting property (L2 in Figure 1). Embedded aggregation columns and rows also represent a common structural characteristic across HCTs. For these cases, we track whether the aggregation is *global* (spanning all columns or rows) or *local* (restricted to a subset), and whether it is *explicit*, e.g., "Total," "Sum," and "Avg.", or *implicit*, e.g., "Fruits," denoting a collection of fruit categories.

Manually creating fine-grained metadata for each HCT object represents a distinctive feature HCT-QA, enabling more in-depth analyses of LLM and VLM performance. Figure 3(a) presents aggregated statistics illustrating the distribution of tables across various property combinations. As shown, the combination of {*column nesting*, *row aggregation*, *column aggregation*} is the most frequent, appearing in 396 HCTs and *column nesting* is the most prevalent structural property overall, occurring in more than 900 HCTs. To support the systematic annotation of this metadata, we developed a custom annotation interface [32] that allows human experts to visually inspect each HCT and select the corresponding layout properties. Each annotation is subsequently reviewed and verified by a second annotator to ensure accuracy and consistency.

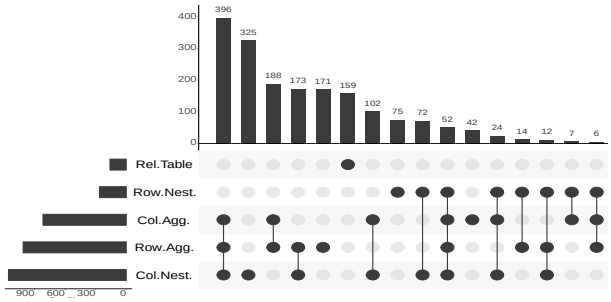
### B. Questions and Ground-Truth Answers Generation

A question over an HCT object refers to a NL question aimed at retrieving specific information—either directly (e.g., selecting particular value(s)) or through computation (e.g., expressions or aggregations). Our focus is on single-HCT questions. For the real-world dataset, we adopt two approaches for question generation: (1) Manually-generated questions – these are produced by the authors of this paper who are database experts with several years of combined experience in academia and industry, and (2) model-generated questions – these are produced by an LLM. All QA pairs, regardless of their source, are subsequently subject to independent human verification to maintain high dataset quality.

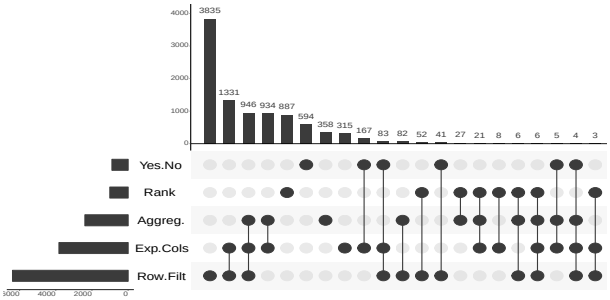
**Characterization of Question Properties.** We designed a set of properties, inspired by common SQL operations, to systematically characterize each question. These properties capture both the type and complexity of operations—such as filtering, aggregation, rank, and projection—required to derive the answer. For each operation, additional sub-properties are maintained to capture more details on the operation's complexity. For example, the question properties include the number

| | Number of | QNPC | Pak. C. | US C. | ArXiv | Synth. | Total |
|---|---|---|---|---|---|---|---|
| Average | Rows | 16 | 20 | 40 | 12 | 11 | |
| | Columns | 9 | 7 | 10 | 8 | 14 | |
| | Cells | 145 | 146 | 437 | 102 | 151 | |
| | Table Words | 201 | 187 | 575 | 138 | 153 | |
| Total | **HCTs** | **958** | **128** | **219** | **575** | **4,679** | **6,559** |
| | **QA Pairs** | **7,128** | **979** | **387** | **1,341** | **67,747** | **77,582** |

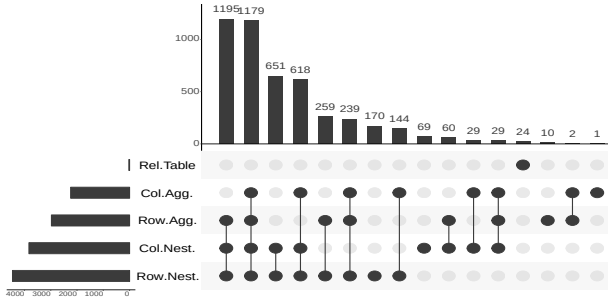
(a) Statistics of HCTs and QA pairs




(b) Main properties of real-world HCTs




(c) Main properties of real-world QA pairs




(d) Main properties of synthetic HCTs

Fig. 3. Main properties of the HCTs and QA pairs in HCT-QA.

of filtering conditions, whether the projected columns include expressions or not, the number and type of aggregators (e.g., min, max, and avg.) as well as if they are local (a specific subset of values in the HCT) or global (values across the HCT), and the ranking on plain column or expression. These properties provide valuable insights into the types of questions that models tend to find most challenging. To facilitate capturing these properties, we designed a custom interface as depicted in Figure 4. These properties are only provided for the manually-generated questions.

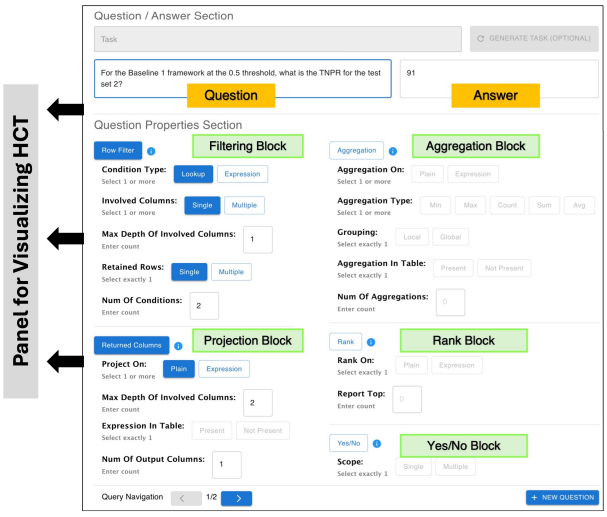



Fig. 4. Question interface for the properties of manually-generated questions

**Manually-Generated QA Pairs.** To generate a question and its answer, we use the annotation interface shown in Figure 4. This interface enables the user to visualize the table image, formulate a question with its associated answer, and annotate the relevant question properties. Also, the interface presents real-time statistics on the types of questions generated, encouraging the user to create a balanced distribution of questions involving diverse operators, expressions, and aggregations. Each question undergoes verification by two independent persons to ensure quality and consistency. We guarantee that every HCT includes at least one manually-generated question. In total, we produced **3,248** such questions with their properties based on real-world HCTs, with each question requiring approximately **1.5** to **3** minutes to complete.

**Model-Generated QA Pairs.** To expand our question set beyond, we use OpenAI's GPT [70] to automatically generate additional questions. The model is provided with the HCT image, a sample manually-generated QA pair from the previous step (using one-shot prompting), and detailed instructions specifying the required constraints, such as the target operations and the desired level of question complexity. The model then produces a set of eight QA pairs per HCT, enabling efficient large-scale question generation. Using this approach, we generated **6,587** additional QA pairs on real-world HCTs in approximately **30** minutes.

To ensure the accuracy and clarity of model-generated questions, we established a rigorous human verification and correction pipeline. Each model-generated QA pair is reviewed by a team of annotators who (i) assess the quality of the question as *good*, *ambiguous* or *nonsensical* and (ii) correct the corresponding answer if it is incorrect or poorly formatted. Questions that are rated as nonsensical are excluded from the dataset, while those marked as ambiguous are re-written. Our annotation team comprised of 14 undergrad students majoring in Computer Science who had experience with databases. Annotators were given a training session by the authors on the task description and did supervised mock exercises of the task. Each QA pair is evaluated by two annotators independently.

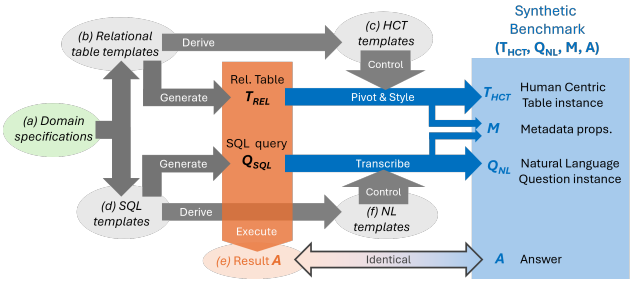


Fig. 5. HCT-QA Synthetic Data Generator workflow

In cases of disagreement, a third annotator makes the final decision. To monitor the consistency and attentiveness of annotators, we embedded a set of pre-validated, manually-generated questions within their evaluation batches. Annotators who misclassified more than one of these verified questions (e.g., labeling them as ambiguous or nonsensical or incorrectly flagging their answers) were excluded from the annotation pool. Only a single annotator failed our hidden test questions and was replaced.

Our model-based pipeline for QA generation and verification was highly effective and scalable. Overall, only 4% of model-generated questions were marked as nonsensical, 24% as ambiguous, and 72% as good. Moreover, the model's answers required minor corrections for nearly 50% of the questions. Combining the manually-generated and model-generated questions, we ended up with a total of **9,835** QA pairs over the **1,880** HCTs. In Figure 3(a), we present the distribution of the questions across the different datasets. Moreover, in Figure 3(b), we present aggregated distribution of the question properties over all questions. For example, roughly **2,000** questions involve aggregations, while **1,000** questions involve ranking, and there are **594** yes/no questions. Overall, Figure 3(b) shows a good distribution of NL questions on real-world HCTs ranging from simple yes/no questions to complex combination of aggregation, ranking, column expression, etc.

**Manual Effort.** Referring to Figure 2, to prepare 1,880 real-world HCTs, a combined 150 manual work hours (MWH) were spent as follows: 15 MWH for HCTs filtering to meet certain standards (e.g., visual quality and no out-of-scope properties), 15 MWH for sample verification (10% from text and image modalities), 60 MWH for annotating HCT properties, and 80 MWH for 2-way verification of the annotated HCTs and their properties. To prepare 9,835 QA pairs, 1020 MWH were spent as follows: 150 MWH for manually creating and annotating QA pairs and their properties, 270 MWH for 2-way verification of the manually created and annotated QA pairs, and 600 MWH to verify model generated QA pairs. The creation and verification of the manual QA pairs was done by the authors, whereas the model generated QA pair verification was done by a team of 14 annotators. Combined, this entire effort took around 1,170 MWH.

## IV. HCT-QA Synthetic Generator

To enable large-scale generation of HCTs with their corresponding QA pairs, we built a synthetic generator comprising two components: a **HCT Data Generator** (Section IV-A) and a **QA Generator** (Section IV-B). This generator serves as a valuable resource for the research community, facilitating the exploration of research challenges on HCTs. The generator is highly configurable, allowing it to capture nearly all common HCT and question properties defined earlier. It can also be configured to create QA pairs with multiple operators, which would be difficult to create manually at scale.

The main goal of the HCT-QA Synthetic Generator is to reduce the time and labor cost required to create HCTs and QA pairs on them. The entire process of manually extracting real-world HCTs, creating manually generated or human-verified model-generated QA pairs, and labeling the HCT and question properties takes around 4 minutes per QA pair. In contrast, creating the domain vocabulary for an entire family of synthetic HCTs, specifying the related table and question templates takes about 1 hour in total with the generation of a synthetic HCT instance and a related QA pair from that family taking only less than a second. Thus, the synthetic generator enables scaling up the generation process to several thousands HCTs and QA pairs in a few hours, while still controlling their diversity and complexity through the template design. As we will show throughout this section, the HCT-QA Synthetic Generator has the following four properties:

1) Scalability: It can be easily extended to cover new domains and generate thousands of HCTs and QA pairs with minimal efforts.
2) Guaranteed semantic correctness: Based on a template-based approach, semantic fidelity can be ensured across the constructed HCTs and their generated NL questions.
3) Broad coverage of question complexity: NL questions are generated using various SQL templates spanning different levels of sophistication and operator complexities.
4) Disambiguation: The generated NL questions have a clear interpretation in terms of what they are referring to in the table regardless of their complexity.

The synthetic HCT and QA generation process (Figure 5) begins with domain specifications (a) to create custom *table templates* (b) and *question templates* (d) from which, inspired by SQL, sample instances of relational tables $T_{REL}$ and SQL queries $Q_{SQL}$ are generated, respectively. Executing $Q_{SQL}$ on $T_{REL}$ gives ground-truth results $A$ (e). We derive HCT templates (c) from relational table templates to control how to pivot and style the same $T_{REL}$ instance into different variants of HCTs instances $T_{HCT}$. We derive NL templates (f) from SQL templates that shape the transcription of $Q_{SQL}$ queries into NL questions $Q_{NL}$. Results $A$ (e) to $(T_{REL}, Q_{SQL})$ instances are used as answers to $(T_{HCT}, Q_{NL})$ instances as they are identical by design. Metadata $M$ characterizing HCTs and question properties (Section III) are computed too, forming the $(T_{HCT}, Q_{NL}, A, M)$ benchmark quadruplets.

### A. Synthetic HCT Data Generator

The *HCT Data Generator* generates HCTs in multiple formats such as CSV, HTML, PDF, and PNG, and supports various stylistic features such as row header indentation, various background colors and fonts, and line separations.

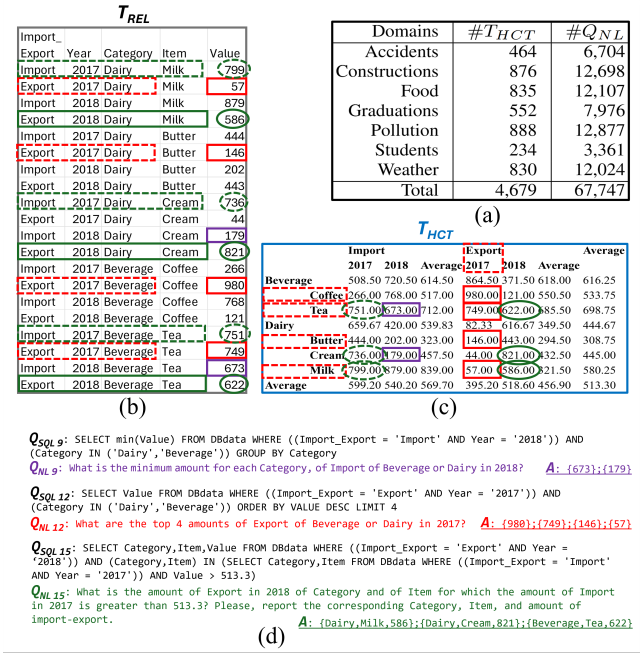



Fig. 6. (a) Statistics of the synthetic HCTs and QAs for 7 domains. An example of a relational table $T_{REL}$ (b) and a generated $T_{HCT}$ from it (c). (d) Examples of synthetic QAs, where $Q_{SQL}$ is transformed into $Q_{NL}$ using custom templates, and executed on $T_{REL}$ to get the ground-truth answer $A$.

Through a combination of configuration files and templates, the generator can reproduce 15 (marked with "*" in Table II) of the 19 HCT properties. The remaining 4 properties are not supported in this version due to their complex irregularities.

**Domain Specifications (Figure 5(a))**. We begin by defining a set of realistic application domains, such as *import–export statistics across countries and months over several years*. Each domain is associated with a *domain vocabulary* consisting of (1) a JSON configuration file listing attribute names and their set of possible values and (2) catalog tables listing a comprehensive set of attributes from the vocabulary, their data types, and possible value ranges. Various HCTs are then created by different combinations of attributes and value ranges. For example, in a "Food import–export" domain (Figure 6(b)), user-defined attributes such as `Year: {2015,...,2023}`, `Import–Export Op.: {Import, Export}`, `Food Category: {Meat, Dairy}`, and `Food Item: {Chicken, Beef, Cream, Milk, Yogurt}` are used to generate HCTs describing food import-export activities over years.

**Types of Attributes (Figure 5(a)).** We distinguish between two types of attributes, *independent* attributes such as `Year` and `Import–Export` for which all pairs of values are possible, e.g., `(2022, Import)`, and *hierarchical* attributes such as `(Country, State, City)` and `(Food category, Food item)`, where attribute values follow an inherent order and only certain combinations are meaningful. For example, combinations such as `(California, Boston)` are invalid. *Independent* attributes can appear as either row or column headers at any nesting level. In contrast, *hierarchical* attributes must respect logical order — parent attributes, such as `Country`, must precede child ones, such as `City`, from left to right or top to bottom within headers. Moreover, all attributes belonging to the same hierarchy must be placed together, either entirely in the row headers or entirely in the column headers, e.g., an HCT cannot have `Country` as a row header and `City` as a column header. More details are provided in [32].

**Relational Table Templates (Figure 5(b)).** We manually define *Table templates* in JSON format that give details about which attributes and values to pick for a given scenario to form a family of relational tables. For example, for the *import-export* scenario, the data generator uses the table template to create the relational table ($T_{REL}$) depicted in Figure 6(b). The template defines a target field, e.g., the `Value` numerical attribute in Figure 6(b), to contain the fine-grained value (automatically generated) for each combination over the rest of the attributes. The generator enforces uniqueness constraints on the target field to prevent ambiguity when evaluating model-predicted answers against ground-truth values. Relational tables serve two essential purposes: (1) They enable systematic generation of HCTs through pivoting operations [71], [72]; and (2) they are used to construct the ground truth answers by executing SQL queries on them without human intervention.

**HCT Templates (Figure 5(c)).** These templates define the final construction rules for producing HCTs instances from the domain specifications and structural combinations set in the relational table template from which they are automatically generated. They are highly configurable, for example, one can specify the depth of row and column headers, the assignment of attributes to each header, and the inclusion and type of aggregation functions and whether the aggregation occurs at a global or local level. They also capture layout specifications such as nesting balance and symmetry, as well as stylistic elements, e.g., line separations, and indentation.

As shown in Figure 6(c), an example HCT instance ($T_{HCT}$) is generated from its corresponding relational table ($T_{REL}$). In this case, attributes `(Import_Export, Year)` form nested column headers, while `(Category, Items)` define nested row headers. The column nesting is balanced and symmetric, whereas the row nesting is balanced but asymmetric. This instance includes both local and global aggregation rows and columns (average). Food `Category` row headers (`Beverage` and `Dairy`) are indented and their row contains implicit local average aggregation of food `Items` (*e.g.*, `Coffee` and `Tea` for `Beverage`), while global aggregation of food `Category` and column aggregations are all explicit ("Average" appears in the header). These properties are automatically recorded in the metadata $M$. It is worth mentioning that with the relational table and the HCT templates, the Synthetic HCT Data Generator only requires the domain vocabulary to generate HCTs for new domains.

**Statistics on Synthetic HCTs.** In our benchmark and as summarized in Figure 6(a), we covered seven distinct domains and generated hundreds of HCTs per domain with a total of **4,679** synthetic HCTs. These HCTs exhibit different complexities and layouts as presented in Figure 3(c).

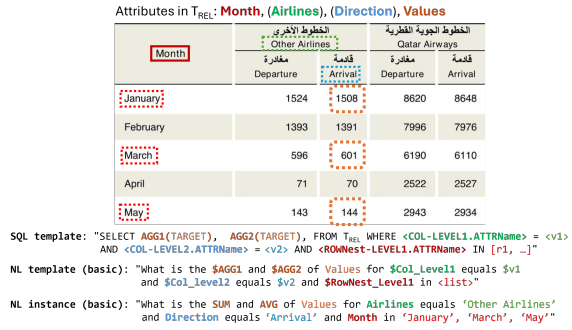



Fig. 7. Example of a SQL query template, a basic NL template, and its instantiation on the HCT (top) for synthetic query generation.

### B. Synthetic Question & Answer Generator

Our synthetic QA Generator coupled with the HCT Generator, allows us to scale HCT-QA to produce tens of thousands of QA on thousands of synthetic HCTs with 100% accuracy.

**SQL Query Templates (Figure 5(d).** The QA Generator builds upon the same domain specifications and table templates described in Section IV-A. Table III lists 15 SQL query templates, each representing a distinct level of complexity. These templates are used to instantiate SQL queries with varying structures and operations from simple or multiple rows and columns selection ($SQL_{1,2,3,6}$), to multiple aggregations over one row or one column ($SQL_{4,7}$), when the same aggregation is already displayed in the HCT ($SQL_5$), or grouped by any attribute levels or only by the top level one, and with or without reporting the attributes ($SQL_{8,9,10,11}$). We also generate sorting and top-k queries ($SQL_{12,13}$) as well as selections with operations ($SQL_{14}$) and a selection conditioned on another selection ($SQL_{15}$).

As an example, Figure 7 presents a SQL query template ($SQL_7$ in Table III) to generate global aggregation queries (i.e., without sub-groups) on the relational representation $T_{REL}$ (not shown) of an HCT. The template applies one equality condition to each attribute involved in column nesting, allowing it to naturally extend to multi-level nesting scenarios. When applied to the presented HCT, these conditions trace through the structure, one level at a time, to reach a specific column (green & blue colors of the WHERE clauses correspond to the same color boxes in the HCT column headers). Additionally, the template introduces IN conditions on attributes associated with row nesting. The HCT in Figure 7 contains no row nesting, and hence a maximum of one condition is possible (i.e., on the `Month` column). In contrast, the HCT in Figure 6(c) contains row nesting, and the template allows putting conditions on the corresponding columns (i.e., on the `Category` and `Item` columns in $T_{REL}$). These conditions contribute to increasing the query's complexity. Finally, the template applies one or more aggregation functions over the selected output values.

**Ground Truth Generation (Figure 5(e).** The relational table $T_{REL}$ and the generated SQL queries are imported into a relational DBMS for execution and the generation of the ground-truth answer $A$, thus forming the triplet $(T_{REL}, Q_{SQL}, A)$. In Figure 6(d), we illustrate three SQL query instances generated from different SQL query templates ($SQL_9$, $SQL_{12}$, and $SQL_{15}$ in Table III).

**Natural Language Templates (Figures 5(d-f)).** For each SQL query template, we manually define a NL template to transcribe $Q_{SQL}$ into $Q_{NL}$ specific to the HCT domain. To illustrate the need for these custom NL templates, let's consider the SQL query template in Figure 7 that would be transcribed into a (basic) NL template as:

```
"What is the $AGG1 and $AGG2 of Values
for $Col_Level1 equals $v1 and $Col_level2
equals $v2 and $RowNest_Level1 IN <list>"
```

Then, a (basic) NL question instance, given some values from the HCT in Figure 7, would be:

```
"What is the SUM and AVG of Values for
Airlines equals 'Other Airlines' and
Direction equals 'Arrival' and Month IN
['January', 'March', 'May']"
```

Obviously, this basic direct transcription does not form a seamless NL question. Hence, we customize the NL templates for each of the seven domains covered in our generator (Figure 6(a)), generating more natural questions. We illustrate a few examples in Figure 6(d). Similar to the construction of synthetic HCTs, the Synthetic QA generator is domain-agnostic. It relies solely on specifying a domain by its vocabulary describing attributes (e.g. `Month`) and values (`May`, `June`...) in a JSON format, and the manual design of 15 $Q_{NL}$ templates based on this vocabulary and simple rules to ensure English syntax. Each $Q_{NL}$ template transcribes in natural language one of the 15 $Q_{SQL}$ templates shown in Table III, ensuring non-repetitive queries. The full list of 15 $Q_{NL}$ templates for each of the 7 domains and the rules to build them, is available in Appendix D in the extended report [32]. As each $Q_{NL}$ template corresponds to a $Q_{SQL}$ template with specific query properties, we automatically record them in the metadata $M$.

**Statistics on Synthetic Questions**. We generated **67,747** QA pairs on the $4,679$ synthetic HCTs across seven domains.

**Scalability**. Thanks to manually defined JSON templates, our Synthetic Data Generator can be augmented with minimal effort. The domain specifications can be augmented at will, adding attribute names and their list of values, to generate HCTs in any domain of interest. Also, the relational table templates allow controlling the diversity and complexity of the generated HCTs. SQL query templates can be easily configurable due to their computational nature. The corresponding NL templates require minimum effort to adapt to a new domain, using a small set of syntax rules. This template-based automation allows unlimited HCT-QA accurate data generation in minimal time without the need for costly verification.

**Manual Effort.** A key advantage of the synthetic data generator is that, once the tool is implemented—a process that required several hundred hours of development—adding new SQL or natural language templates for a specific domain typically takes no more than two to three hours.

## V. Experimental Evaluation

While HCT-QA is designed to evaluate data processing engines during answering NL questions on HCTs, our evaluation focus on LLMs and VLMs. To this end, we explain

TABLE III

SQL TEMPLATES OF SYNTHETIC HCT-QA. $r|c$: DEPTH OF ROW|COLUMN (R|C) HEADERS OF THE HCT. $Attr^*_C$: ALL ATTRIBUTES OF C. $Attr^1_R$: LEFT-MOST ATTRIBUTE OF R. $Num$: AVG. OF SELECTED COLUMN. $Dir \overset{\text{RAND}}{\sim} \{\text{ASC, DESC}\}$, $k \overset{\text{RAND}}{\sim} \{2,3,4,5\}$, AND $Op \overset{\text{RAND}}{\sim} \{<,>\}$.

| | SQL templates |
|---|---|
| $SQL_1$ | **SELECT** Value **FROM** Data **WHERE** ($Attr_R$ = $Val_R$)$^r$ **AND** ($Attr_C$ = $Val_C$)$^c$ |
| $SQL_2$ | **SELECT** Value **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^r$ **AND** ($Attr_C$ = $Val_C$)$^c$ |
| $SQL_3$ | **SELECT** Value **FROM** Data **WHERE** ($Attr_R$ = $Val_R$)$^r$ **AND** ($Attr_C$ **IN** $ListVal_C$)$^c$ |
| $SQL_4$ | **SELECT** $AGG_1$(Value), $AGG_2$(Value) **FROM** Data **WHERE** ($Attr_R$ = $Val_R$) **AND** ($Attr_C$ **IN** $ListVal_C$) |
| $SQL_5$ | **SELECT** $AGG_*$(Value) **FROM** Data **WHERE** ($Attr_R$ = $Val_R$)$^r$ **AND** ($Attr_C$ **IN** $ListVal_C$)$^c$ |
| $SQL_6$ | **SELECT** Value **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^r$ **AND** ($Attr_C$ **IN** $ListVal_C$)$^c$ |
| $SQL_7$ | **SELECT** $AGG_1$(Value), $AGG_2$(Value) **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^r$ **AND** ($Attr_C$ = $Val_C$)$^c$ |
| $SQL_8$ | **SELECT** $Attr^*_C$, $AGG_1$(Value), $AGG_2$(Value) **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^r$ **AND** ($Attr_C$ **IN** $ListVal_C$)$^c$ **GROUP BY** $Attr^*_C$ |
| $SQL_9$ | **SELECT** MIN(Value) **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^r$ **AND** ($Attr_C$ = $Val_C$)$^c$ **GROUP BY** $Attr^1_R$ |
| $SQL_{10}$ | **SELECT** $Attr^1_R$, $AGG_1$(Value) **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^r$ **AND** ($Attr_C$ = $Val_C$)$^c$ **GROUP BY** $Attr^1_R$ |
| $SQL_{11}$ | **SELECT** $Attr^1_R$, $Attr^*_C$, $AGG_1$(Value) **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^r$ **AND** ($Attr_C$ **IN** $ListVal_C$)$^c$ **GROUP BY** $Attr^1_R$, $Attr^*_C$ |
| $SQL_{12}$ | **SELECT** Value **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^1$ **AND** ($Attr_C$ = $Val_C$)$^c$ **ORDER BY VALUE** Dir **LIMIT** k |
| $SQL_{13}$ | **SELECT** Value **FROM** Data **WHERE** ($Attr_R$ **IN** $ListVal_R$)$^1$ **AND** ($Attr_C$ = $Val_C$)$^c$ **ORDER BY VALUE** Dir |
| $SQL_{14}$ | **SELECT** $Attr^*_R$ **FROM** Data **WHERE** ($Attr_C$ = $Val_C$)$^c$ **AND** Value Op Num |
| $SQL_{15}$ | **SELECT** $Attr^*_R$, Value **FROM** Data **WHERE** ($Attr_{C \neq C_{14}}$ = $Val_{C \neq C_{14}}$)$^c$ **AND** ($Attr^*_R$ **IN** Result($SQL_{14}$)) |

our experimental setup in Section V-A and then answer seven research questions (RQs) experimentally in Section V-B.

### A. Experimental setup

**Models.** We tested a large variety of open- and closed-weight LLMs and VLMs (Table IV). All open-weight models were downloaded from HuggingFace.

**Datasets.** There are 1,880 real-world HCTs from four data sources along with 9,835 QA pairs and 4,679 synthetic HCTs with 67,747 QA pairs (total of **6,559** HCTs with **77,582** QA pairs). For the results in Table IV, the text modality HCTs were represented in HTML format and the vision-text modality HCTs were images encoded in base64.

**Prompting Approach and Hyperparameters.** We experimented with zero-shot and one-shot prompting with no clear indication that the latter improved model performance. Thus, all subsequent experiments use zero-shot prompting. We ask the model to "only answer the question using information from the HCT " and allow it to refuse to answer the question if it deems it unanswerable. For inference, the temperature was set to 0, and the maximum output tokens to 256. For finetuning, the learning rate was 5.0e-07 and the number of epochs was 2. The finetuning config file can be found in [32]. We post-process the model responses using a rule-based approach to extract the actual answer if one was provided. All inference and finetuning experiments were run on 8 NVIDIA A100 GPUs. Inference which required around 77k prompts took a total of 129 hours, averaging 1 hour, 1.5 hours, and 22 hours per small, medium, and cloud model, respectively. Additionally, our finetuning experiments took 30 hours.

**Evaluation Metrics.** We use two metrics for model accuracy: (i) *F1 Score*–commonly adopted in several table QA benchmarks, e.g., [6], [27], [28], [30], [59], and (ii) *Complete Containment (CC Score)*–a binary score that evaluates to 1 iff the recall is 100%, otherwise it is 0. The two metrics are complementary: F1 rewards partial correctness of the answer while CC rewards an answer only if it contains the ground truth in its entirety. For example, assume the ground truth is *["Apple", "300"]* and three distinct models output: *M1 ["Apple", "300"]*, *M2 ["Apple"]*, and *M3 ["Apple", "300", "Orange", "10"]*, then their scores will be: M1 (F1= 1.0, CC= 1), M2 (F1= 0.66, CC= 0), and M3 (F1= 0.66, CC= 1).

### B. Analysis and findings

**(RQ1) How do language models perform overall on HCT-QA?** As shown in Table IV, the relatively recent versions of large closed-weight LLMs like ChatGPT 4o and ChatGPT 4.1 on the text modality perform reasonably well, achieving F1 scores of 66% and 62% respectively. This shows that even for state-of-the-art LLMs, there is ample room for improvement on this task. Despite ChatGPT 4o being the best performing model overall, we see that its predecessor ChatGPT 3.5 scores an average F1 of 40% which is less than all of the medium sized open-weight LLMs. This highlights the progress more recent LLMs have made in their reasoning abilities despite their much smaller size. Furthermore, Qwen2.5-72B-Instruct achieves an average F1 of 62.9%, which is only 3.4 percentage points less than ChatGPT 4o's score, despite its size of 72B parameters being considerably smaller than the OpenAI model. Similarly, Qwen2.5-72B-Instruct's CC Score of 52% is only 4 percentage points less that ChatGPT 4o CC Score of 56%. On average, large LLMs achieve an F1 of 56% compared to 57% for medium-sized LLMs. Even, when excluding the lagging ChatGPT 3.5, large LLMs have an average F1 score of 64% which is only 7 percentage points higher than the medium-sized LLMs' average. This showcases the potential for medium-sized LLMs on this task.

With an average F1 of 30%, small LLMs lag behind their larger competitors. Even among small LLMs, gemma-3-12b-it's average F1 score of 50% outperforms ChatGPT 3.5 by nearly 10 percentage points. However, we see a considerable drop in gemma-3-12b-it's CC Score; only 36.4% which is far below the average for medium and large models. This indicates that it does get a lot of the questions partially correct but struggles to provide the fully correct answer. Moreover, small VLMs with an average F1 of 21.4% are far worse off than small LLMs. But, among the VLMs, Pixtral-12B outperforms many LLMs such as Llama-3.1-8B-Instruct, Mistral-7B-Instruct-v0.3, and Qwen2-7B-Instruct. This indicates that VLMs do have the potential to compete with similarly sized LLMs on HCT-QA, possibly by utilizing the visual features

TABLE IV
OVERALL PERFORMANCE OF MODELS ON HCT-QA

| Model Name | Model Param Size | QNPC F1 | QNPC CC Score | PAK F1 | PAK CC Score | US CENSUS F1 | US CENSUS CC Score | arXiv F1 | arXiv CC Score | Synthetic Tables F1 | Synthetic Tables CC Score | Average F1 | Average CC Score |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **TEXT-ONLY LLMs** | | | | | | | | | | | | | |
| **Large Models – 100B+ (Closed Weight)** | | | | | | | | | | | | | |
| ChatGPT 3.5 | 175B | 0.413 | 0.315 | 0.445 | 0.356 | 0.385 | 0.333 | 0.459 | 0.386 | 0.345 | 0.279 | **0.409** | **0.334** |
| ChatGPT 4o | Unknown | 0.663 | 0.571 | 0.720 | 0.639 | 0.650 | 0.583 | 0.719 | 0.654 | 0.563 | 0.365 | **0.663** | **0.562** |
| ChatGPT 4.1 | Unknown | 0.621 | 0.545 | 0.640 | 0.576 | 0.650 | 0.600 | 0.667 | 0.605 | 0.543 | 0.415 | **0.624** | **0.548** |
| *Average* | - | *0.566* | *0.477* | *0.602* | *0.524* | *0.562* | *0.505* | *0.615* | *0.548* | *0.484* | *0.353* | *0.566* | *0.481* |
| **Medium Models – 27B–72B (Open Weight)** | | | | | | | | | | | | | |
| Llama-3.1-70B-Instruct | 70B | 0.586 | 0.484 | 0.639 | 0.551 | 0.585 | 0.519 | 0.632 | 0.555 | 0.512 | 0.323 | **0.591** | **0.486** |
| Llama-3.3-70B-Instruct | 70B | 0.573 | 0.497 | 0.636 | 0.571 | 0.548 | 0.530 | 0.645 | 0.595 | 0.499 | 0.361 | **0.580** | **0.511** |
| Qwen2.5-72B-Instruct | 72B | 0.621 | 0.532 | 0.673 | 0.593 | 0.589 | 0.540 | 0.678 | 0.612 | 0.582 | 0.339 | **0.629** | **0.523** |
| Qwen3-32B | 32B | 0.523 | 0.435 | 0.578 | 0.491 | 0.538 | 0.479 | 0.576 | 0.503 | 0.458 | 0.227 | **0.535** | **0.427** |
| gemma-2-27b-it | 27B | 0.549 | 0.444 | 0.617 | 0.514 | 0.509 | 0.466 | 0.631 | 0.552 | 0.433 | 0.207 | **0.548** | **0.437** |
| gemma-3-27b-it | 27B | 0.535 | 0.445 | 0.598 | 0.514 | 0.527 | 0.469 | 0.622 | 0.560 | 0.476 | 0.275 | **0.552** | **0.453** |
| *Average (Medium)* | - | *0.565* | *0.473* | *0.624* | *0.539* | *0.550* | *0.500* | *0.631* | *0.563* | *0.493* | *0.289* | *0.572* | *0.473* |
| **Small Models – 4B–20B (Open Weight)** | | | | | | | | | | | | | |
| gemma-2-9b-it | 9B | 0.475 | 0.360 | 0.537 | 0.413 | 0.409 | 0.357 | 0.497 | 0.422 | 0.383 | 0.171 | **0.460** | **0.345** |
| gemma-3-12b-it | 12B | 0.498 | 0.371 | 0.545 | 0.412 | 0.465 | 0.401 | 0.543 | 0.443 | 0.449 | 0.195 | **0.500** | **0.364** |
| Qwen1.5-7B-Chat | 7B | 0.220 | 0.173 | 0.268 | 0.220 | 0.194 | 0.182 | 0.314 | 0.268 | 0.124 | 0.059 | **0.224** | **0.180** |
| Qwen2-7B-Instruct | 7B | 0.333 | 0.273 | 0.356 | 0.305 | 0.283 | 0.289 | 0.375 | 0.341 | 0.295 | 0.107 | **0.328** | **0.263** |
| Qwen2.5-7B-Instruct | 7B | 0.417 | 0.343 | 0.454 | 0.384 | 0.361 | 0.383 | 0.483 | 0.445 | 0.334 | 0.124 | **0.410** | **0.336** |
| Qwen3-14B | 14B | 0.490 | 0.430 | 0.535 | 0.482 | 0.458 | 0.454 | 0.577 | 0.547 | 0.394 | 0.156 | **0.491** | **0.414** |
| Mistral-7B-Instruct-v0.1 | 7B | 0.071 | 0.062 | 0.077 | 0.065 | 0.032 | 0.027 | 0.143 | 0.129 | 0.062 | 0.040 | **0.077** | **0.065** |
| Mistral-7B-Instruct-v0.2 | 7B | 0.119 | 0.108 | 0.120 | 0.123 | 0.137 | 0.124 | 0.169 | 0.173 | 0.059 | 0.035 | **0.121** | **0.112** |
| Mistral-7B-Instruct-v0.3 | 7B | 0.295 | 0.219 | 0.321 | 0.255 | 0.253 | 0.222 | 0.342 | 0.298 | 0.151 | 0.068 | **0.273** | **0.213** |
| Mathstral-7B-v0.1 | 7B | 0.326 | 0.266 | 0.387 | 0.320 | 0.184 | 0.169 | 0.362 | 0.329 | 0.215 | 0.097 | **0.295** | **0.236** |
| Meta-Llama-3-8B-Instruct | 8B | 0.344 | 0.241 | 0.374 | 0.268 | 0.334 | 0.302 | 0.376 | 0.301 | 0.262 | 0.100 | **0.338** | **0.242** |
| Llama-3.1-8B-Instruct | 8B | 0.363 | 0.257 | 0.393 | 0.278 | 0.327 | 0.290 | 0.416 | 0.342 | 0.310 | 0.119 | **0.362** | **0.257** |
| Phi-3-mini-4k-instruct | 4B | 0.233 | 0.167 | 0.265 | 0.185 | 0.223 | 0.198 | 0.339 | 0.267 | 0.211 | 0.107 | **0.254** | **0.185** |
| Phi-3.5-vision-instruct | 4B | 0.274 | 0.193 | 0.235 | 0.170 | 0.190 | 0.143 | 0.245 | 0.189 | 0.108 | 0.059 | **0.211** | **0.151** |
| Phi-4-mini-instruct | 4B | 0.225 | 0.156 | 0.131 | 0.086 | 0.044 | 0.037 | 0.154 | 0.116 | 0.062 | 0.021 | **0.123** | **0.083** |
| Fanar-C-9B-Instruct | 9B | 0.431 | 0.322 | 0.463 | 0.351 | 0.473 | 0.416 | 0.522 | 0.451 | 0.437 | 0.217 | **0.465** | **0.351** |
| *Average (Small)* | - | *0.320* | *0.246* | *0.341* | *0.270* | *0.273* | *0.249* | *0.366* | *0.316* | *0.241* | *0.105* | *0.308* | *0.237* |
| **VISION-TEXT VLMs** | | | | | | | | | | | | | |
| **Large Models – 100B+ (Closed Weight)** | | | | | | | | | | | | | |
| ChatGPT 4o Vision | Unknown | 0.543 | 0.513 | 0.638 | 0.613 | 0.348 | 0.328 | 0.618 | 0.597 | - | - | **0.537** | **0.513** |
| **Small Models – 3B–12B (Open Weight)** | | | | | | | | | | | | | |
| Pixtral-12B-2409 | 12B | 0.378 | 0.289 | 0.471 | 0.372 | 0.300 | 0.273 | 0.457 | 0.411 | 0.364 | 0.161 | **0.394** | **0.301** |
| InternVL2-4B | 4B | 0.236 | 0.181 | 0.281 | 0.199 | 0.148 | 0.162 | 0.305 | 0.297 | 0.265 | 0.102 | **0.247** | **0.188** |
| InternVL2_5-8B-MPO | 8B | 0.193 | 0.129 | 0.270 | 0.196 | 0.177 | 0.170 | 0.349 | 0.288 | 0.208 | 0.076 | **0.240** | **0.172** |
| Phi-3-vision-128k-instruct | 4B | 0.214 | 0.187 | 0.280 | 0.252 | 0.210 | 0.199 | 0.339 | 0.317 | 0.241 | 0.135 | **0.257** | **0.218** |
| Phi-3.5-vision-instruct | 4B | 0.208 | 0.138 | 0.314 | 0.227 | 0.148 | 0.120 | 0.332 | 0.269 | 0.265 | 0.133 | **0.253** | **0.177** |
| llava-1.5-7b-hf | 7B | 0.067 | 0.061 | 0.096 | 0.092 | 0.067 | 0.057 | 0.042 | 0.039 | 0.030 | 0.006 | **0.061** | **0.051** |
| llava-v1.6-vicuna-7b-hf | 7B | 0.123 | 0.091 | 0.157 | 0.118 | 0.095 | 0.077 | 0.133 | 0.107 | 0.115 | 0.032 | **0.125** | **0.085** |
| Molmo-7B-D-0924 | 7B | 0.130 | 0.067 | 0.164 | 0.100 | 0.092 | 0.075 | 0.147 | 0.103 | 0.156 | 0.027 | **0.138** | **0.074** |
| *Average (Small)* | - | *0.194* | *0.143* | *0.254* | *0.194* | *0.155* | *0.142* | *0.263* | *0.229* | *0.205* | *0.084* | *0.214* | *0.158* |

*Note: ChatGPT-4o Vision was not run on synthetic HCT images due to API constraints*

of HCTs that may be lost in a textual representation. Overall, there remains clear room for improvement for both LLMs and VLMs. Furthermore, the F1 scores of models on synthetic HCTs is, on average, slightly less than on real-world HCTs. This shows that our synthetic generator produces HCTs and QA pairs that are challenging for all models. However, there is a considerable drop in the CC Scores on synthetic HCTs. This could be due to many of the synthetic QA pairs requiring answers with many more values than the QA pairs on real-world HCTs–as the number of values in the answer increases, the models' ability to generate a completely correct answer significantly degrades. For example, the *Qwen2.5-72B-Instruct* model under the synthetic HCT tables has the highest F1 score while its CC score is relatively low.

In Figure 8 we also compare how models perform on QA pairs created using our three methods. Synthetic questions appear to be the most challenging which is expected as the majority require *Row Filtering*, *Expression on Columns*, and *Aggregation* together. On average, models of all sizes perform best on the manually-generated QA pairs. This indicates that our model-generated and synthetic questions provide a slightly harder challenge for the evaluated models.

**(RQ2) Are models improving over time?** There appears to be no consistent trend with regards to newer generations of models outperforming their predecessors. The Qwen family of small LLMs shows continuous improvement as Qwen1.5-7B, Qwen2-7B, Qwen2.5-7B, and Qwen3-14B all outperform their predecessor with average F1 scores of 22.4%, 32.8%, 41%, and 49.1% respectively. This shows that the Qwen family is improving the performance of their small LLMs by an average of 9% per generation. The same can be seen with their CC Scores as Qwen3-14B is 23 percentage points better than Qwen1.5-7B-Chat. This shows that the models are getting much better at giving the complete correct answer. We see similar trends with the Mistral 7B LLMs which also improve by nearly 9% per generation and Gemma LLMs that improve by 4%. Conversely, the Phi family of small LLMs shows consistent deterioration in each successive generation with Phi-3-mini-4k-instruct achieving an average F1 score of 25.4% which is nearly double that of Phi-4-mini-instruct, despite the latter being released 8 months later. The ChatGPT family shows a mixed trend with a significant improvement of 26 percentage points between ChatGPT 3.5 and ChatGPT 4.0, but then ChatGPT 4.1 under-performs its predecessor by 4 percentage points. There are also cases like gemma-2-27b-it and gemma-3-27b-it which shows the newer generation

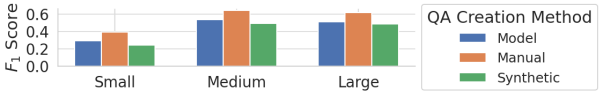


Fig. 8. Model performance by size across different QA creation types

TABLE V
MODEL F1 SCORES ACROSS DIFFERENT HCTS INPUT FORMATS

| Model | CSV | Markdown | HTML | Text Obfuscated HTML |
|---|---|---|---|---|
| ChatGPT 4.1 | 0.484 | 0.529 | 0.543 | 0.499 |
| ChatGPT 4o | 0.541 | 0.567 | 0.563 | 0.531 |
| Qwen2.5-72B-Instruct | 0.422 | 0.453 | 0.582 | 0.508 |
| Llama-3.3-70B-Instruct | 0.431 | 0.443 | 0.499 | 0.398 |
| Qwen2.5-7B-Instruct | 0.301 | 0.311 | 0.334 | 0.358 |
| Llama-3.1-8B-Instruct | 0.191 | 0.262 | 0.311 | 0.277 |
| *Average* | *0.395* | *0.426* | *0.472* | *0.42* |

TABLE VI
FINETUNED MODEL PERFORMANCES ACROSS DIFFERENT TEST SETS.

| Model Name | Finetuning Set | Test Sets (F1 Score) | | |
|---|---|---|---|---|
| | | Real HCTs | Synthetic HCTs | HiTab [27] |
| Llama-3.1-8B-Instruct | None | 0.351 | 0.333 | 0.538 |
| Llama-3.1-8B-Instruct | Real HCTs | 0.593 | 0.534 | 0.606 |
| Llama-3.1-8B-Instruct | Synth. HCTs | 0.471 | 0.879 | 0.561 |
| Llama-3.1-8B-Instruct | Real & Synth. HCTs | 0.595 | 0.886 | 0.616 |

TABLE VII
EFFECT OF TABLE PROPERTIES ON F1. ONLY PROPERTIES WITH SIGNIFICANT EFFECT ($p < 0.05$) ARE SHOWN

| Table Properties | Coef. (p-value) | Table Properties | Coef. (p-value) |
|---|---|---|---|
| Symmetric Row Nesting | -0.103 (0.016) | Global Column Aggreg. | -0.035 (0.028) |
| Asymmetric Row Nesting | -0.142 (0.001) | Implicit Col. Aggreg. | 0.081 (0.001) |
| Balanced Col. Nesting | -0.270 (0.022) | Implicit Row Aggreg. | -0.065 (0.007) |
| Unbalanced Col. Nesting | -0.315 (0.009) | Row Group Label | -0.069 (0.000) |

improving by a mere 0.4 percentage points in terms of average F1. We also see a similar trend with small VLMs as Phi-3-vision-128k-instruct outperforms its successor, Phi-3.5-vision-instruct, by 0.4 percentage points. Thus, we cannot say that newer versions of models outperform their older versions across the board.

**(RQ3) Does the input format of the HCT matter in text-modality?** For the text modality LLMs, we tested the effect of representing the HCT in CSV and Markdown formats instead of HTML. The results are shown in Table V. The HTML format outperforms CSV and Markdown formats by an average F1 score difference of 7.7 and 4.6 percentage points, respectively. This result is intuitive as HTML is a more expressive format that can better retain the complex structure of the HCTs. Some models are more sensitive than others to the table format. Qwen2.5-72B-Instruct performs 13 percentage points better with the HTML format than with Markdown and shows a similar difference between the HTML and CSV formats as well. Whereas, ChatGPT 4o shows a relative indifference to the table format as there is only a 0.2 percentage point difference in F1 scores between the Markdown and HTML formats, with the former performing minutely better. Our results show that while large LLMs like ChatGPT can perform well with both Markdown and HTML, medium and small sized LLMs perform much better with an HTML representation of the HCT.

We also explore the effect of semantically coherent and meaningful text within the HCTs. We created *Text Obfuscated-HTML* versions of the HCT HTMLs where we (i) replace all column names with ambiguous terms (ii) replace all text cell values in the HCTs with similar ambiguous terms, and (iii) replace the corresponding QA pairs with the respective ambiguous terms from the obfuscated table. Although the complexity of the questions and table structure remains unchanged, we see that the text obfuscation negatively impacts model performance. LLMs on average, experience a decrease in F1 score of 5 percentage points on Text Obfuscated-HTML format when compared to the original HTMLs. Even a large model like ChatGPT 4.1 cannot compensate for the lack of semantics as its F1 score drops by 4.4 percentage points.

**(RQ4) Does finetuning models improve performance?** We finetuned the Llama-3.1-8B-Instruct model on three FT sets: (i) *Real HCTs* which has 7,867 (80% of total) HCT-QA triplets on real world HCTs only, (ii) *Synth. HCTs* which has 54,186 (80% of total) HCT-QA triplets on synthetic HCTs only, (iii) *Real + Synth. HCTs* which combines both the *Real HCTs* and *Synth. HCTs* sets (62,053 in total). We then evaluate the models on the respective *Real HCTs* and *Synth. HCTs* test sets. We ensure that no HCT in the test set is seen during finetuning. Furthermore, to test the generalization of finetuning in HCT-QA we test on the HiTab dataset [27]. The results are shown in Table VI. Finetuning on the *Real + Synth. HCTs* set drastically increases the performance of the model across the board. The F1 score improvement, compared to the non-finetuned model is 24.1 percentage points on real HCTs from HCT-QA, 55 percentage points on synthetic HCTs from HCT-QA, and nearly 8 percentage points on the out of domain HiTab dataset. Furthermore, finetuning on the *Synth. HCTs* set yields an F1 score improvement of 12 percentage points on the real HCTs test set. This reinforces the usefulness of our synthetic data generator since training only on synthetic HCTs generalizes well to real-world HCTs.

**(RQ5) How do HCT properties affect model performance?** Table VII shows the regression coefficient of various HCT properties against F1 scores. We can see that the presence of most of these properties, except *Implicit Column Aggregation Terms*, decreases the F1 score on average. We also see a clear trend in models being able to better understand symmetrical and balanced nesting patterns compared to asymmetrical and unbalanced ones: *Symmetric Row Nesting* decreases F1 by 10 percentage points whereas *Asymmetric Row Nesting* decreases it by 14 percentage points. The same trend is seen when comparing *Balanced Column Nesting* with *Unbalanced Column Nesting* with the former having a decrease of 27 percentage points on F1 against 31.5 percentage points for the latter. Column and row nesting increases the HCT difficulty more than any type of aggregate rows or columns, having the largest negative impact on F1 scores.

**(RQ6) How do question properties affect model performance?** Our question metadata allows for fine-grained analysis to understand what types of questions different models excel at and struggle with. Figure 9 shows such an analysis on a subset of models and metadata. Heatmap (A) shows that all models struggle with questions requiring aggregations

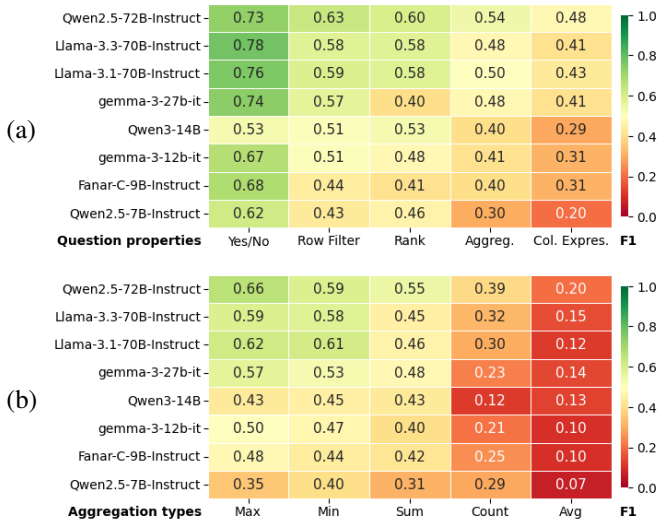



Fig. 9. F1 scores for models across a subset of question properties (a) and aggregation types (b). Models are ordered by decreasing size from top to bottom, and by decreasing average scores from left to right.

or expressions on columns. We can also see that Qwen3-14B, despite being better than models like Fanar-C-9B and gemma-3-27b at questions involving ranking and row filtering, pales in comparison to both on yes/no questions. Furthermore, Qwen2.5-72B shows good performance on aggregation questions compared to other models. In general, models perform better with yes/no and row filter questions as this type of question is answered directly from the data cells, similarly to scan, filter and project stateless operators in databases (i.e., a single row/cell is enough to answer the question), while ranking is more challenging for the models because, similar to the `ORDER BY` stateful operator in databases, it requires a full view of the data. Aggregation is even more challenging because it operates on multiple cells, which can be in different locations of the HCT. In Figure 9), we observe that, for example, end users would be better off using gemma-3-27b than the larger Llama-3.3-70B and Llama-3.1-70B models for questions requiring aggregation, row filtering, and yes/no answers, as their performance is nearly the same, while gemma-3-27b is much more efficient due to its smaller size.

We go one step deeper and analyze what types of aggregations these models struggle with in particular. Heatmap (B) in Figure 9 shows the F1 scores of the models on the five different types of aggregation. We can clearly see that all models struggle significantly with averaging, presumably because this involves a multi-step calculation. We can also see that both Llama-3.1-70B and Llama-3.3-70B are particularly good at min and max questions but struggle more with sum questions. Whereas, Qwen2.5-72B is the only model that performs reasonably well on sum questions which would explain how it outperforms the other models on aggregation questions in general. As expected, all models struggle with questions involving aggregation and particularly those that require averaging values. The metadata provided in HCT-QA allows for such granular analysis to be done for each individual

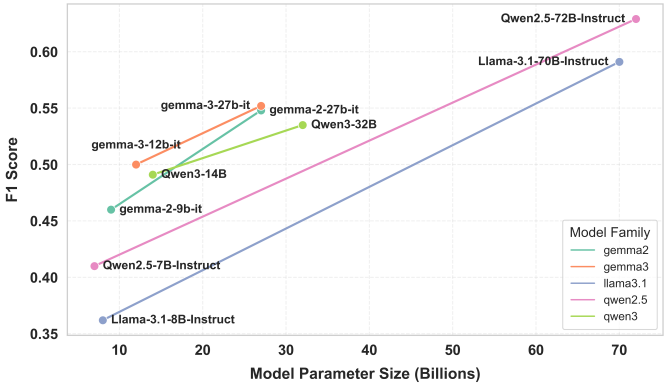



Fig. 10. F1 score across model size (parameters) within model variants

model so that specific areas for improvement can be identified.

**(RQ7) How are model size and performance correlated?** Figure 10 shows how F1 score changes between different size variants of the same model. As expected, larger models consistently outperform their smaller counterparts. The gemma-2 models have the sharpest increase with $0.40\%$ F1 per 1B parameters (F1/B). Even when scaling to larger sizes like with Qwen2.5 and Llama-3.1 we see an increase of $0.34\%$ F1/B and $0.37\%$ F1/B , respectively. Hence, it is worth exploring fine-tuning smaller models for this task rather than simply relying on larger, resource-hungry off-the-shelf models.

## VI. Conclusion and Future Work

In this paper, we presented HCT-QA, a comprehensive benchmark designed to evaluate language models on question answering over human-centric tables (HCTs), typically embedded within documents and web pages.

Our benchmark reveals several key insights. First, both LLMs and VLMs exhibit strong potential for question answering on HCTs. Second, finetuning LLMs substantially enhances performance, even when finetuned on only synthetic data, models demonstrated strong generalization to real-world HCTs. Finally, even the best-performing models (e.g., ChatGPT 4o) continue to face challenges with complex queries (e.g., involving aggregations) on complex HCT structures. Futhermore, our benchmark can be beneficial to other researchers in at least two ways: (i) Using the synthetic data generator to create training and test HCTs and QA pairs for new thematic domains in a cheap and quick manner. This is especially useful for training, finetuning, and evaluating new models. (ii) Adding question templates to the generator to cover more types of questions.

**Future Work.** Our benchmark can be extended along several complementary directions. First, we plan to broaden the scope of supported queries by enabling cross-HCT question answering, e.g., queries involving joins or unions across multiple HCTs. We also aim to incorporate queries that require more advanced OLAP operations, including `CUBE` and `PIVOT`. Initial experiments with two medium-sized LLMs indicate that these models are currently unable to handle such operators, highlighting substantial room for improvement.

Second, we intend to conduct deeper studies on fine-tuning strategies for both LLMs and VLMs to improve their structural understanding of HCTs. In addition, we will evaluate emerging systems that go beyond current LLM/VLM architectures, as these models may offer improved reasoning capabilities for complex tabular and visual data.

Third, we plan to enhance the linguistic diversity of the synthetic NL questions. One direction is use question paraphrasing capabilities using LLMs, which would increase lexical variation in the generated QA pairs. This extension will require robust validation strategies to ensure semantic equivalence between the original and paraphrased questions. Furthermore, the generator can be enriched with additional NL templates that capture implicit comparative terms such as *most*, *least*, *highest*, and *lowest*, instead of relying solely on explicit SQL-style operators such as *MIN* and *MAX*.

## VII. AI-Generated Content Acknowledgment

Here are the different places where we used AI models in our work:

- LLMS were used to generate natural language questions and answers for the benchmark dataset as explained in Sections III-A.
- The LandingLens from LandingAI tool was used for automated extraction and conversion of tabular data from PDF documents into HTML format (Section III-A).
- Azure AI Document Intelligence was used to extract table images from PDF documents (Section III-A).

## References


[1] W. Wang, Y. Tian, H. Wang, and W.-S. Ku, "A natural language interface for database: Achieving transfer-learnability using adversarial method for question understanding," in *2020 IEEE 36th International conference on data engineering (ICDE)*. IEEE, 2020, pp. 97–108.

[2] Y. Song, J. Lu, Y. Song, C. C. Cao, R. C.-W. Wong, and H. Zhang, "Fevisqa: Free-form question answering over data visualizations," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 2025, pp. 2726–2739.

[3] S. Acharya, K. Sornalakshmi, B. Paul, and A. Singh, "Question answering system using NLP and BERT," in *2022 3rd International Conference on Smart Electronics and Communication (ICOSEC)*. IEEE, 2022, pp. 925–929.

[4] Y. Peng, X. Hu, J. Peng, X. Geng, X. Yang *et al.*, "Live: Learnable in-context vector for visual question answering," *Advances in Neural Information Processing Systems*, vol. 37, pp. 9773–9800, 2024.

[5] J. Herzig, P. K. Nowak, T. Müller, F. Piccinno, and J. Eisenschlos, "TaPas: Weakly Supervised Table Parsing via Pre-training," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020. [Online]. Available: http://dx.doi.org/10.18653/v1/2020.acl-main.398

[6] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and Text-to-SQL task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds. Brussels, Belgium: Association for Computational Linguistics, Oct.-Nov. 2018, pp. 3911–3921. [Online]. Available: https://aclanthology.org/D18-1425/

[7] U. Brunner and K. Stockinger, "Valuenet: A natural Language-to-SQL system that learns from database information," 2021. [Online]. Available: https://arxiv.org/abs/2006.00888

[8] Y. Fan, Z. He, T. Ren, D. Guo, L. Chen, R. Zhu, G. Chen, Y. Jing, K. Zhang, and X. Wang, "Gar: A generate-and-rank approach for natural language to SQL translation," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 110–122.

[9] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 7567–7578. [Online]. Available: https://aclanthology.org/2020.acl-main.677/

[10] H. Fu, C. Liu, B. Wu, F. Li, J. Tan, and J. Sun, "CatSQL: Towards Real World Natural Language to SQL Applications," *Proc. VLDB Endow.*, vol. 16, no. 6, p. 1534–1547, Feb. 2023. [Online]. Available: https://doi.org/10.14778/3583140.3583165

[11] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo *et al.*, "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[12] M. Iyyer, W.-t. Yih, and M.-W. Chang, "Search-based Neural Structured Learning for Sequential Question Answering," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, R. Barzilay and M.-Y. Kan, Eds. Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1821–1831. [Online]. Available: https://aclanthology.org/P17-1167/

[13] Q. Liu, B. Chen, J. Guo, M. Ziyadi, Z. Lin, W. Chen, and J.-G. Lou, "TAPEX: Table Pre-training via Learning a Neural SQL Executor," 2022. [Online]. Available: https://arxiv.org/abs/2107.07653

[14] Z. Jiang, Y. Mao, P. He, G. Neubig, and W. Chen, "Omnitab: Pretraining with natural and synthetic data for few-shot table-based question answering," 2022. [Online]. Available: https://arxiv.org/abs/2207.03637

[15] B. Li, Y. Luo, C. Chai, G. Li, and N. Tang, "The Dawn of Natural Language to SQL: Are We Fully Ready? [Experiment, Analysis & Benchmark ]," *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 3318–3331, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p3318-luo.pdf

[16] "Qatar National Planning Council," https://www.npc.qa/en/statistics, accessed: 2025-05-15.

[17] "arXiv," https://arxiv.org/, accessed: 2025-05-15.

[18] "US Census Bureau," https://www.census.gov/, accessed: 2025-05-15.

[19] "Pakistan Bureau of Statistics," https://www.pbs.gov.pk/, accessed: 2025-05-15.

[20] W. Shen and H. Wen, "Ambiguous Entity Oriented Targeted Document Detection," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 874–886.

[21] Y. Lin, M. Hulsebos, R. Ma, S. Shankar, S. Zeighami, A. G. Parameswaran, and E. Wu, "Querying templatized document collections with large language models," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 2025, pp. 2422–2435.

[22] M. Lu, S. Bangalore, G. Cormode, M. Hadjieleftheriou, and D. Srivastava, "A dataset search engine for the research document corpus," in *2012 IEEE 28th International Conference on Data Engineering*. IEEE, 2012, pp. 1237–1240.

[23] Z. Li, X. Li, Z. Duan, B. Dong, N. Liu, and J. Wang, "Toward a unified framework for unsupervised complex tabular reasoning," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 1691–1704.

[24] A. Mitsopoulou and G. Koutrika, "Analysis of text-to-SQL benchmarks: limitations, challenges and opportunities," in *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025*. OpenProceedings. org, 2025, pp. 199–212.

[25] P. Li, Y. He, C. Yan, Y. Wang, and S. Chaudhuri, "Auto-Tables: Synthesizing Multi-Step Transformations to Relationalize Tables without Using Examples," *Proc. VLDB Endow.*, vol. 16, no. 11, p. 3391–3403, jul 2023. [Online]. Available: https://doi.org/10.14778/3611479.3611534

[26] H. Kim, B.-H. So, W.-S. Han, and H. Lee, "Natural language to SQL: Where are we today?" *Proceedings of the VLDB Endowment*, vol. 13, no. 10, pp. 1737–1750, 2020.

[27] Z. Cheng, H. Dong, Z. Wang, R. Jia, J. Guo, Y. Gao, S. Han, J.-G. Lou, and D. Zhang, "HiTab: A hierarchical table dataset for question answering and natural language generation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 1094–1110. [Online]. Available: https://aclanthology.org/2022.acl-long.78/

[28] Y. Zhao, Y. Li, C. Li, and R. Zhang, "MultiHiertt: Numerical Reasoning over Multi Hierarchical Tabular and Textual Data," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 6588–6600. [Online]. Available: https://aclanthology.org/2022.acl-long.454/

[29] Y. Kim, M. Yim, and K. Y. Song, "TableVQA-Bench: A Visual Question Answering Benchmark on Multiple Table Domains," 2024, https://arxiv.org/abs/2404.19205. [Online]. Available: https://arxiv.org/abs/2404.19205

[30] F. Zhu, W. Lei, Y. Huang, C. Wang, S. Zhang, J. Lv, F. Feng, and T.-S. Chua, "TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds. Online: Association for Computational Linguistics, Aug. 2021, pp. 3277–3287. [Online]. Available: https://aclanthology.org/2021.acl-long.254

[31] "HCTQA benchmark data." https://huggingface.co/datasets/qcri-ai/HCTQA, accessed: 2025-05-15.

[32] "HCT-QA: A Benchmark for Question Answering on Human-Centric Tables," https://github.com/qcri/HCTQA-Benchmark, 2025.

[33] "Aspose," https://docs.aspose.com/pdf, accessed: 2025-05-15.

[34] M. Corporation, "Azure AI Document Intelligence," https://azure.microsoft.com/en-us/products/ai-services/ai-document-intelligence, 2025, accessed: 2025-05-15.

[35] "Innodata," https://solutions.innodata.com/document-intelligence/, accessed: 2025-05-15.

[36] "Docugami," https://www.docugami.com/, accessed: 2025-05-15.

[37] D. Burdick, M. Danilevsky, A. V. Evfimievski, Y. Katsis, and N. Wang, "Table extraction and understanding for scientific and enterprise applications," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3433–3436, 2020.

[38] A. Nassar, N. Livathinos, M. Lysak, and P. Staar, "Tableformer: Table structure understanding with transformers," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 4604–4613.

[39] C. Ma, W. Lin, L. Sun, and Q. Huo, "Robust table detection and structure recognition from heterogeneous document images," *Pattern Recognition*, vol. 133, p. 109006, Jan. 2023. [Online]. Available: http://dx.doi.org/10.1016/j.patcog.2022.109006

[40] J. Wan, S. Song, W. Yu, Y. Liu, W. Cheng, F. Huang, X. Bai, C. Yao, and Z. Yang, "Omniparser: A unified framework for text spotting, key information extraction and table recognition," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 15 641–15 653.

[41] A. Nassar, A. Marafioti, M. Omenetti, M. Lysak, N. Livathinos, C. Auer, L. Morin, R. T. de Lima, Y. Kim, A. S. Gurbuz, M. Dolfi, M. Farré, and P. W. J. Staar, "SmolDocling: An ultra-compact vision-language model for end-to-end multi-modal document conversion," 2025, https://arxiv.org/abs/2503.11576. [Online]. Available: https://arxiv.org/abs/2503.11576

[42] U. Khan, S. Zahid, M. A. Ali, A. Ul-Hasan, and F. Shafait, "TabAug: Data Driven Augmentation for Enhanced Table Structure Recognition," in *Document Analysis and Recognition – ICDAR 2021: 16th International Conference, Lausanne, Switzerland, September 5–10, 2021, Proceedings, Part II*. Berlin, Heidelberg: Springer-Verlag, 2021, p. 585–601. [Online]. Available: https://doi.org/10.1007/978-3-030-86331-9_38

[43] E. Oro and M. Ruffolo, "Pdf-trex: An approach for recognizing and extracting tables from pdf documents," in *2009 10th International Conference on Document Analysis and Recognition*, 2009, pp. 906–910.

[44] L. Qiao, Z. Li, Z. Cheng, P. Zhang, S. Pu, Y. Niu, W. Ren, W. Tan, and F. Wu, "Lgpma: Complicated table structure recognition with local and global pyramid mask alignment," in *Document Analysis and Recognition – ICDAR 2021: 16th International Conference, Lausanne, Switzerland, September 5–10, 2021, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2021, p. 99–114. [Online]. Available: https://doi.org/10.1007/978-3-030-86549-8_7

[45] N. Weir, P. Utama, A. Galakatos, A. Crotty, A. Ilkhechi, S. Ramaswamy, R. Bhushan, N. Geisler, B. Hättasch, S. Eger, U. Cetintemel, and C. Binnig, "Dbpal: A fully pluggable nl2sql training pipeline," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 2347–2361. [Online]. Available: https://doi.org/10.1145/3318464.3380589

[46] F. Basik, B. Hättasch, A. Ilkhechi, A. Usta, S. Ramaswamy, P. Utama, N. Weir, C. Binnig, and U. Cetintemel, "DBPal: A Learned NL-Interface for Databases," in *Proceedings of the 2018 International Conference on Management of Data*, ser. SIGMOD '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 1765–1768, https://doi.org/10.1145/3183713.3193562.

[47] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for Text-to-SQL parsers," in *Annual Meeting of the Association for Computational Linguistics*, 2019, https://api.semanticscholar.org/CorpusID:207863446.

[48] U. Brunner and K. Stockinger, "ValueNet: A Natural Language-to-SQL System that Learns from Database Information," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 2177–2182.

[49] J. Sen, C. Lei, A. Quamar, F. Özcan, V. Efthymiou, A. Dalmia, G. Stager, A. Mittal, D. Saha, and K. Sankaranarayanan, "ATHENA++: natural language querying for complex nested SQL queries," *Proc. VLDB Endow.*, vol. 13, no. 12, p. 2747–2759, jul 2020. [Online]. Available: https://doi.org/10.14778/3407790.3407858

[50] X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Zhang, J. Fan, G. Li, N. Tang, and Y. Luo, "A Survey of NL2SQL with Large Language Models: Where are we, and where are we going?" 2024, https://arxiv.org/abs/2408.05109. [Online]. Available: https://arxiv.org/abs/2408.05109

[51] Y. He, X. Chu, K. Ganjam, Y. Zheng, V. Narasayya, and S. Chaudhuri, "Transform-data-by-example (TDE): an extensible search engine for data transformations," *Proc. VLDB Endow.*, vol. 11, no. 10, p. 1165–1177, Jun. 2018.

[52] Z. Jin, Y. He, and S. Chauduri, "Auto-transform: learning-to-transform by patterns," *Proc. VLDB Endow.*, vol. 13, no. 12, p. 2368–2381, Jul. 2020.

[53] X. Zhang, S. Luo, B. Zhang, Z. Ma, J. Zhang, Y. Li, G. Li, Z. Yao, K. Xu, J. Zhou, D. Zhang-Li, J. Yu, S. Zhao, J. Li, and J. Tang, "TableLLM: Enabling Tabular Data Manipulation by LLMs in Real Office Usage Scenarios," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 10 315–10 344. [Online]. Available: https://aclanthology.org/2025.findings-acl.538/

[54] S. A. Chemmengath, V. Kumar, S. Bharadwaj, J. Sen, M. Canim, S. Chakrabarti, A. Gliozzo, and K. Sankaranarayanan, "Topic Transferable Table Question Answering," 2021. [Online]. Available: https://arxiv.org/abs/2109.07377

[55] X. Wu, J. Yang, L. Chai, G. Zhang, J. Liu, X. Du, D. Liang, D. Shu, X. Cheng, T. Sun, T. Li, Z. Li, and G. Niu, "TableBench: A Comprehensive and Complex Benchmark for Table Question Answering," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, pp. 25 497–25 506, Apr. 2025. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/34739

[56] L. Nan, C. Hsieh, Z. Mao, X. V. Lin, N. Verma, R. Zhang, W. Kryściński, H. Schoelkopf, R. Kong, X. Tang, M. Mutuma, B. Rosand, I. Trindade, R. Bandaru, J. Cunningham, C. Xiong, D. Radev, and D. Radev, "FeTaQA: Free-form Table Question Answering," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 35–49, 2022. [Online]. Available: https://aclanthology.org/2022.tacl-1.3/

[57] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang, "TabFact: A Large-scale Dataset for Table-based Fact Verification," 2020, https://arxiv.org/abs/1909.02164.

[58] W. Chen, M.-W. Chang, E. Schlinger, W. Wang, and W. W. Cohen, "Open Question Answering over Tables and Text," 2021, https://arxiv.org/abs/2010.10439.

[59] W. Chen, H. Zha, Z. Chen, W. Xiong, H. Wang, and W. Y. Wang, "HybridQA: A Dataset of Multi-Hop Question Answering over Tabular and Textual Data," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 1026–1036. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.91/

[60] Y. Zhang, J. Henkel, A. Floratou, J. Cahoon, S. Deep, and J. M. Patel, "ReAcTable: enhancing ReAct for table question answering," *Proceedings of the VLDB Endowment*, vol. 17, no. 8, pp. 1981–1994, 2024.

[61] J.-P. Zhu, P. Cai, K. Xu, L. Li, Y. Sun, S. Zhou, H. Su, L. Tang, and Q. Liu, "Autotqa: Towards autonomous tabular question answering through multi-agent large language models," *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 3920–3933, 2024.

[62] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova, D. Langdon, R. Moussa, M. Beane, T.-H. Huang, B. R. Routledge *et al.*, "FinQA: A Dataset of Numerical Reasoning over Financial Data," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 3697–3711.

[63] Y. Katsis, S. Chemmengath, V. Kumar, S. Bharadwaj, M. Canim, M. Glass, A. Gliozzo, F. Pan, J. Sen, K. Sankaranarayanan, and S. Chakrabarti, "AIT-QA: Question Answering Dataset over Complex Tables in the Airline Industry," 2021, https://arxiv.org/abs/2106.12944. [Online]. Available: https://arxiv.org/abs/2106.12944

[64] S. Pramanick, R. Chellappa, and S. Venugopalan, "SPIQA: A Dataset for Multimodal Question Answering on Scientific Papers," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang,

Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/d74033a247989e8f6f3bf9e0c9629fb5-Abstract-Datasets_and_Benchmarks_Track.html

[65] F. Zhu, W. Lei, F. Feng, C. Wang, H. Zhang, and T.-S. Chua, "Towards Complex Document Understanding By Discrete Reasoning," in *Proceedings of the 30th ACM International Conference on Multimedia*, ser. MM '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 4857–4866. [Online]. Available: https://doi.org/10.1145/3503161.3548422

[66] X. Zhang, D. Wang, B. Wang, L. Dou, X. Lu, K. Xu, D. Wu, Q. Zhu, and W. Che, "SCITAT: A Question Answering Benchmark for Scientific Tables and Text Covering Diverse Reasoning Types," 2024, https://arxiv.org/abs/2412.11757. [Online]. Available: https://arxiv.org/abs/2412.11757

[67] M. Urban and C. Binnig, "CAESURA: language models as multi-modal query planners," *arXiv preprint arXiv:2308.03424*, 2023.

[68] Y. Ma, Y. Zang, L. Chen, M. Chen, Y. Jiao, X. Li, X. Lu, Z. Liu, Y. Ma, X. Dong, P. Zhang, L. Pan, Y. Jiang, J. Wang, Y. Cao, and A. Sun, "MMLONGBENCH-DOC: Benchmarking Long-context Document Understanding with Visualizations," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/ae0e43289bffea0c1fa34633fc608e92-Abstract-Datasets_and_Benchmarks_Track.html

[69] Landing AI, "LandingLens: AI-powered data annotation and model training platform," https://landing.ai, 2025, accessed: 2025-10-18.

[70] OpenAI, "GPT-4 Turbo Model," 2023, https://platform.openai.com/docs/models/gpt-4-turbo.

[71] B. Jelen and M. Alexander, *Pivot Table Data Crunching*. USA: Que Corp., 2005.

[72] "R package pivottabler," https://github.com/cbailiss/pivottabler/, accessed: 2025-05-15.

| Month | الوفيات والإصابات الناتجة عن الحرائق الخاضعة لشروط الأمن والسلامة العامه في الدولة Deaths and injuries resulting from fires that are subject to public safety and security conditions in the country * | | | | الوفيات والإصابات الناتجة عن جميع الحرائق في الدولة Deaths and injuries resulting from all fires in the country | | | |
|---|---|---|---|---|---|---|---|---|
| | المجموع Total | الاصابات Injuries | | وفـاة Death | المجموع Total | الاصابات Injuries | | وفـاة Death |
| | | خفيفة Slight | بليغة Sever | | | خفيفة Slight | بليغة Sever | |
| January | 1 | 1 | 0 | 0 | 4 | 4 | 0 | 0 |
| February | 1 | 1 | 0 | 0 | 3 | 3 | 0 | 0 |
| March | 2 | 2 | 0 | 0 | 2 | 2 | 0 | 0 |
| April | 0 | 0 | 0 | 0 | 10 | 10 | 0 | 0 |
| May | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| June | 2 | 2 | 0 | 0 | 5 | 5 | 0 | 0 |
| July | 0 | 0 | 0 | | 11 | 11 | 0 | 0 |

| Area/ Age Group | Urban | | Rural | |
|---|---|---|---|---|
| | Women | Men | Women | Men |
| Pakistan | | | | |
| Total | 61.81 | 57.05 | 68.05 | 61.84 |
| 15-19 years | 6.37 | 1.32 | 14.59 | 3.26 |
| 20-24 years | 39.75 | 15.24 | 59.19 | 29.29 |
| 25-29 years | 76.32 | 49.86 | 85.47 | 67.33 |
| 30-34 years | 91.70 | 79.45 | 94.15 | 88.35 |
| 35-39 years | 93.74 | 92.13 | 94.83 | 94.41 |
| 40-44 years | 92.93 | 96.07 | 94.10 | 96.60 |
| 45-49 years | 87.74 | 96.36 | 90.72 | 96.47 |
| 50-54 years | 81.53 | 92.69 | 84.37 | 95.32 |
| 55-59 years | 69.55 | 89.96 | 77.04 | 92.67 |
| 60+ | 43.99 | 78.24 | 48.07 | 79.28 |

| Methods | GCN | | GIN | | Sage | |
|---|---|---|---|---|---|---|
| | before | after | before | after | before | after |
| Clean | 83.33 | **86.84** | **87.72** | 86.84 | **79.82** | 34.21 |
| Random | **84.21** | 73.68 | 65.79 | **84.21** | **81.58** | 34.21 |
| EMaxS | **34.21** | 34.21 | 79.82 | **81.58** | **73.68** | 68.42 |
| EMinS | **80.70** | 68.42 | 74.56 | **81.58** | **78.07** | 71.05 |
| EMinF | **83.33** | 78.95 | **85.09** | 78.95 | **83.33** | 55.26 |
| SubInj | **35.96** | 34.21 | 35.96 | **65.79** | **73.68** | 57.89 |

Fig. 11. Example of Symmetric Column Nesting

## APPENDIX

### A. HCT Property definitions and examples

Definitions of HCT specific table properties are given in Table II and expanded upon with examples in this section.

**Balanced Column Nesting:** Column nesting can be likened to a tree-like data structure. Similar to the definition of a balanced tree, we can see that each branch of column names (from the top most parent till each leaf) has the same depth. In case of all the table shown in the Figure 14, the depth is 2.

**Unbalanced Column Nesting:** In this the depth of all branches in the nested column tree do not have the same depth. In Figure 13, the first table has "Population characteristics, 2003" as the top most parent, and has branches with depth of 2; the second table has the parent "Cultivable Land" with branches of depth 2 and 3 as well.

**Symmetric Column Nesting:** Column nesting is labeled as symmetric when the children of all the cells in highest level of nesting that has more than 1 cell are all identical. In the case of the first table in Figure 11, the top two cells have identical child branches (Total, Injuries-Slight, Injuries-Sever, Death). Whereas the second table has the parent cells "Urban" and "Rural" both with identical children values ("Women" and "Men"). We do not consider column values such as "Month" (in the first table) and "Area/Age Group" in the second table as they do not exhibit any nesting. The first table also shows an example of where column nesting is unbalanced but is still symmetrical.

| Year | Nationality & Gender | Total المجموع | | | | Non-Qataris غير قطريين | | | | Qataris قطريون | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | المجموع العام G.Total | نسبة الاناث %Female | ذكور F | نسبة الذكور % Male | ذكور M | % | المجموع Total | إناث F | ذكور M | % | المجموع Total | إناث F | ذكور M |
| 2007 | | 15,681 | 48.6 | 7,625 | 51.4 | 8,056 | 54.2 | 8,503 | 4,104 | 4,399 | 45.8 | 7,178 | 3,521 | 3,657 |
| 2008 | | 17,210 | 49.4 | 8,507 | 50.6 | 8,703 | 57.3 | 9,855 | 4,857 | 4,998 | 42.7 | 7,355 | 3,650 | 3,705 |
| 2009 | | 18,351 | 48.8 | 8,950 | 51.2 | 9,401 | 59.9 | 10,987 | 5,342 | 5,645 | 40.1 | 7,364 | 3,608 | 3,756 |
| 2010 | | 19,504 | 49.1 | 9,578 | 50.9 | 9,926 | 60.4 | 11,771 | 5,681 | 6,090 | 39.6 | 7,733 | 3,897 | 3,836 |
| 2011 | | 20,623 | 49.1 | 10,136 | 50.9 | 10,487 | 63.2 | 13,031 | 6,366 | 6,665 | 36.8 | 7,592 | 3,770 | 3,822 |
| 2012 | | 21,423 | 49.2 | 10,533 | 50.8 | 10,890 | 67.5 | 14,470 | 7,117 | 7,353 | 32.5 | 6,953 | 3,416 | 3,537 |

| Metropolitan statistical area **Metropolitan statistical area with metropolitan divisions** *Metropolitan division* | Population characteristics, 2003 | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Age (percent) | | | | | | | | | | One race (percent) | | | | | | Hispanic or Latino origin (percent) | Males per 100 females |
| | Under 5 years | 5 to 14 years | 15 to 24 years | 25 to 34 years | 35 to 44 years | 45 to 54 years | 55 to 64 years | 65 to 74 years | 75 years and over | White alone | Black or African American alone | American Indian, Alaska Native alone | Asian alone | Native Hawaiian and Other Pacific Islander alone | | | |
| Manchester-Nashua, NH | 6.3 | 14.6 | 12.6 | 12.5 | 18.0 | 15.4 | 9.9 | 5.4 | 5.2 | 94.4 | 1.7 | 2.5 | 0.3 | 0.0 | 3.6 | 98.0 |
| Mansfield, OH | 6.2 | 13.4 | 13.6 | 11.9 | 14.6 | 14.9 | 10.7 | 7.7 | 7.1 | 88.5 | 9.5 | 0.6 | 0.2 | 0.0 | 0.9 | 101.4 |
| McAllen-Edinburg-Mission, TX | 11.3 | 19.2 | 16.7 | 14.8 | 12.4 | 9.9 | 6.3 | 4.9 | 4.5 | 97.7 | 0.7 | 0.7 | 0.6 | 0.1 | 89.2 | 94.6 |
| Medford, OR | 5.6 | 13.3 | 13.9 | 11.9 | 13.0 | 15.3 | 11.2 | 7.3 | 8.4 | 95.0 | 0.5 | 1.0 | 1.1 | 0.2 | 7.4 | 94.4 |
| Memphis, TN-MS-AR | 7.6 | 15.4 | 14.4 | 14.3 | 15.3 | 14.2 | 9.0 | 5.3 | 4.6 | 52.4 | 44.9 | 1.6 | 0.2 | 0.0 | 2.6 | 93.1 |
| Merced, CA | 8.4 | 18.6 | 17.5 | 14.0 | 13.9 | 11.3 | 7.3 | 4.9 | 4.1 | 85.3 | 4.1 | 6.5 | 1.8 | 0.2 | 49.5 | 101.1 |
| **Miami-Fort Lauderdale-Miami Beach, FL** | **6.5** | **13.4** | **12.2** | **12.7** | **15.8** | **13.5** | **9.9** | **7.6** | **8.3** | **75.7** | **20.9** | **1.9** | **0.3** | **0.1** | **36.8** | **94.1** |
| *Fort Lauderdale-Pompano Beach-Deerfield Beach, FL* | *6.6* | *13.8* | *11.7* | *12.9* | *16.7* | *14.1* | *9.6* | *6.5* | *8.1* | *71.2* | *24.3* | *2.7* | *0.3* | *0.1* | *20.2* | *94.2* |
| *Miami-Miami Beach-Kendall, FL* | *6.8* | *13.7* | *13.1* | *13.5* | *15.9* | *13.3* | *10.1* | *7.3* | *6.2* | *76.2* | *20.9* | *1.4* | *0.3* | *0.1* | *60.5* | *93.9* |
| *West Palm Beach-Boca Raton-Boynton Beach, FL* | *5.8* | *12.3* | *11.2* | *11.0* | *14.4* | *13.1* | *10.1* | *9.6* | *12.5* | *80.8* | *15.9* | *1.8* | *0.4* | *0.1* | *14.9* | *94.5* |
| Michigan City-La Porte, IN | 6.4 | 13.9 | 13.4 | 12.6 | 15.1 | 14.9 | 10.2 | 6.6 | 6.9 | 87.9 | 10.1 | 0.5 | 0.3 | 0.0 | 3.5 | 106.0 |
| Midland, TX | 7.6 | 16.1 | 15.7 | 11.6 | 13.9 | 14.9 | 8.2 | 6.3 | 5.6 | 90.6 | 6.9 | 1.0 | 0.7 | 0.0 | 31.8 | 93.6 |
| Milwaukee-Waukesha-West Allis, WI | 6.9 | 14.5 | 13.8 | 12.8 | 15.6 | 14.7 | 9.3 | 6.1 | 6.3 | 79.2 | 16.4 | 2.5 | 0.6 | 0.1 | 6.9 | 94.6 |
| Minneapolis-St. Paul-Bloomington, MN-WI | 7.0 | 14.4 | 13.8 | 14.6 | 17.0 | 14.7 | 8.9 | 4.8 | 4.8 | 86.9 | 5.9 | 4.7 | 0.7 | 0.1 | 3.8 | 98.0 |
| Missoula, MT | 5.6 | 11.8 | 17.4 | 15.8 | 14.0 | 15.5 | 9.9 | 5.1 | 5.0 | 94.1 | 0.4 | 1.2 | 2.5 | 0.1 | 1.7 | 100.2 |
| Mobile, AL | 7.3 | 15.0 | 14.6 | 12.9 | 14.2 | 14.0 | 9.8 | 6.4 | 5.7 | 62.3 | 34.4 | 1.7 | 0.7 | 0.0 | 1.2 | 91.8 |
| Modesto, CA | 7.6 | 16.9 | 16.2 | 14.3 | 14.6 | 12.5 | 8.0 | 5.1 | 4.9 | 87.7 | 3.0 | 4.7 | 1.4 | 0.5 | 36.0 | 97.8 |
| Monroe, LA | 7.4 | 14.7 | 16.2 | 13.7 | 13.5 | 13.2 | 9.0 | 6.7 | 5.7 | 65.1 | 33.4 | 0.7 | 0.2 | 0.0 | 1.5 | 90.8 |

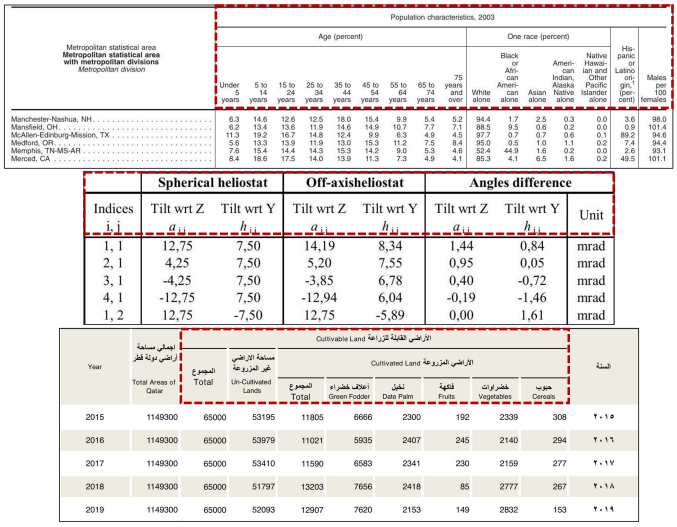
Fig. 12. Example of Asymmetric Column Nesting

**Asymmetric Column Nesting:** Asymmetric column nesting is when the conditions stated above from symetric column nesting are not met. This entails parent cells having different children cells (branches). In the first table in Figure 12 we see the parent cell "Total" has the children ("G.Total", "%Female", "F", "%Male", "M") whereas the other 2 parent cells ("Non-Qataris" and "Qataris" have a different set of children ("%", "Total", "F", "M"). Similarly, in second table, we see that the parent cells "Age (percent)" and "One race (percent)" have different children cells, thus making this column nesting assymetric.

**Balanced Row Nesting:** Very similar to column nesting, balanced row nesting entails that all row nesting branches have the same depth (in this case depth is measured left-to-right rather than top-down as with columns). The 3 tables in Figure 15 show this property as the children values (the second column from the left in each table) are identical across their parent cells (left most column in each table).

**Unbalanced Row Nesting:** This is when the conditions for balanced row nesting are unmet and branches of nesting in rows have different depths. In the table shown in Figure 16 we see that the branches "Orca - Sys. message - Synthetic" (depth of 3) and "Orca - Original model" (depth of 2) have different depths, thus they are unbalanced.

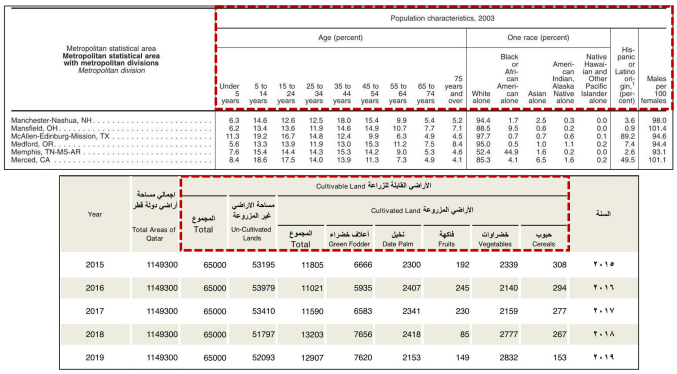
**Symmetric Row Nesting:** Similar to symmetric column nesting, this property entails each parent cell in the first column (left most) that exhibits nesting properties having identical children cells (branches). We can see in the first table in Figure 17 that all the parent cells ("HDFS", "Abase", "Faas", "Alibaba") have the same children cells ("RMSE", "MAE", "p-risk"). Similarly, all the parent cells in the second table have the same children cells ("No.of Hotels", "No.of Rooms", "No.of Beds").




| Indices $i, j$ | Spherical heliostat Tilt wrt Z $a_{i,j}$ | Spherical heliostat Tilt wrt Y $h_{i,j}$ | Off-axisheliostat Tilt wrt Z $a_{i,j}$ | Off-axisheliostat Tilt wrt Y $h_{i,j}$ | Angles difference Tilt wrt Z $a_{i,j}$ | Angles difference Tilt wrt Y $h_{i,j}$ | Unit |
|---|---|---|---|---|---|---|---|
| 1, 1 | 12,75 | 7,50 | 14,19 | 8,34 | 1,44 | 0,84 | mrad |
| 2, 1 | 4,25 | 7,50 | 5,20 | 7,55 | 0,95 | 0,05 | mrad |
| 3, 1 | -4,25 | 7,50 | -3,85 | 6,78 | 0,40 | -0,72 | mrad |
| 4, 1 | -12,75 | 7,50 | -12,94 | 6,04 | -0,19 | -1,46 | mrad |
| 1, 2 | 12,75 | -7,50 | 12,75 | -5,89 | 0,00 | 1,61 | mrad |

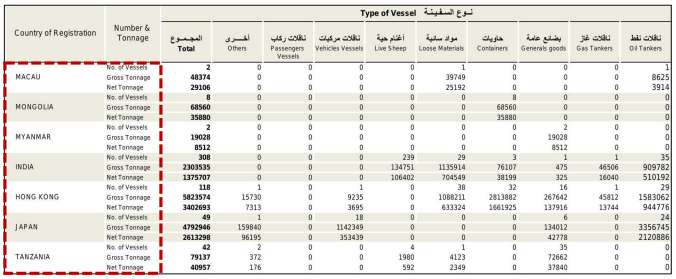
| Year | Total Areas of Qatar | Cultivable Land الأراضي القابلة للزراعة: Total | Un-Cultivated Lands | Cultivated Land الأراضي المزروعة: Total | Green Fodder | Date Palm | Fruits | Vegetables | Cereals | السنة |
|---|---|---|---|---|---|---|---|---|---|---|
| 2015 | 1149300 | 65000 | 53195 | 11805 | 6666 | 2300 | 192 | 2339 | 308 | ٢٠١٥ |
| 2016 | 1149300 | 65000 | 53979 | 11021 | 5935 | 2407 | 245 | 2140 | 294 | ٢٠١٦ |
| 2017 | 1149300 | 65000 | 53410 | 11590 | 6583 | 2341 | 230 | 2159 | 277 | ٢٠١٧ |
| 2018 | 1149300 | 65000 | 51797 | 13203 | 7656 | 2418 | 85 | 2777 | 267 | ٢٠١٨ |
| 2019 | 1149300 | 65000 | 52093 | 12907 | 7620 | 2153 | 149 | 2832 | 153 | ٢٠١٩ |

Fig. 13. Example of Unbalanced Column Nesting




| Year | Total Areas of Qatar | Cultivable Land الأراضي القابلة للزراعة: Total | Un-Cultivated Lands | Cultivated Land الأراضي المزروعة: Total | Green Fodder | Date Palm | Fruits | Vegetables | Cereals | السنة |
|---|---|---|---|---|---|---|---|---|---|---|
| 2015 | 1149300 | 65000 | 53195 | 11805 | 6666 | 2300 | 192 | 2339 | 308 | ٢٠١٥ |
| 2016 | 1149300 | 65000 | 53979 | 11021 | 5935 | 2407 | 245 | 2140 | 294 | ٢٠١٦ |
| 2017 | 1149300 | 65000 | 53410 | 11590 | 6583 | 2341 | 230 | 2159 | 277 | ٢٠١٧ |
| 2018 | 1149300 | 65000 | 51797 | 13203 | 7656 | 2418 | 85 | 2777 | 267 | ٢٠١٨ |
| 2019 | 1149300 | 65000 | 52093 | 12907 | 7620 | 2153 | 149 | 2832 | 153 | ٢٠١٩ |

Fig. 14. Example of Balanced Column Nesting




| Country of Registration | Number & Tonnage | Total المجموع | Others أخرى | Passengers Vessels ناقلات ركاب | Vehicles Vessels ناقلات مركبات | Live Sheep أغنام حية | Loose Materials مواد سائبة | Containers حاويات | General goods بضائع عامة | Gas Tankers ناقلات غاز | Oil Tankers ناقلات نفط |
|---|---|---|---|---|---|---|---|---|---|---|---|
| MACAU | No. of Vessels | 2 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| | Gross Tonnage | 48374 | 0 | 0 | 0 | 0 | 39749 | 0 | 0 | 0 | 8625 |
| | Net Tonnage | 29106 | 0 | 0 | 0 | 0 | 25192 | 0 | 0 | 0 | 3914 |
| MONGOLIA | No. of Vessels | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| | Gross Tonnage | 68560 | 0 | 0 | 0 | 0 | 0 | 68560 | 0 | 0 | 0 |
| | Net Tonnage | 35880 | 0 | 0 | 0 | 0 | 0 | 35880 | 0 | 0 | 0 |
| MYANMAR | No. of Vessels | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 |
| | Gross Tonnage | 19028 | 0 | 0 | 0 | 0 | 0 | 0 | 19028 | 0 | 0 |
| | Net Tonnage | 8512 | 0 | 0 | 0 | 0 | 0 | 0 | 8512 | 0 | 0 |
| INDIA | No. of Vessels | 308 | 0 | 0 | 0 | 239 | 29 | 3 | 1 | 1 | 35 |
| | Gross Tonnage | 2303535 | 0 | 0 | 0 | 134751 | 1135914 | 76107 | 475 | 46506 | 909782 |
| | Net Tonnage | 1375707 | 0 | 0 | 0 | 106402 | 704549 | 38199 | 325 | 16040 | 510192 |
| HONG KONG | No. of Vessels | 118 | 1 | 0 | 1 | 0 | 38 | 32 | 16 | 1 | 29 |
| | Gross Tonnage | 5823574 | 15730 | 0 | 9235 | 0 | 1088211 | 2813882 | 267642 | 45812 | 1583062 |
| | Net Tonnage | 3402693 | 7313 | 0 | 3695 | 0 | 633324 | 1661925 | 137916 | 13744 | 944776 |
| JAPAN | No. of Vessels | 49 | 1 | 0 | 18 | 0 | 0 | 0 | 6 | 0 | 24 |
| | Gross Tonnage | 4792946 | 159840 | 0 | 1142349 | 0 | 0 | 0 | 134012 | 0 | 3356745 |
| | Net Tonnage | 2613298 | 96195 | 0 | 353439 | 0 | 0 | 0 | 42778 | 0 | 2120886 |
| TANZANIA | No. of Vessels | 42 | 2 | 0 | 0 | 4 | 1 | 0 | 35 | 0 | 0 |
| | Gross Tonnage | 79137 | 372 | 0 | 0 | 1980 | 4123 | 0 | 72662 | 0 | 0 |
| | Net Tonnage | 40957 | 176 | 0 | 0 | 592 | 2349 | 0 | 37840 | 0 | 0 |

| Indicators | Sector | 2019 Rate* المعدل* | 2019 Number العدد | 2018 Rate* المعدل* | 2018 Number العدد | 2017 Rate* المعدل* | 2017 Number العدد | 2016 Rate* المعدل* | 2016 Number العدد |
|---|---|---|---|---|---|---|---|---|---|
| Total No. of Physicians and Dentists in Qatar | Government | 2.2 | 6,047 | 2.0 | 5,535 | 1.9 | 5,041 | 2.0 | 5,187 |
| | Private | 1.2 | 3,424 | 1.2 | 3,238 | 1.1 | 3,048 | 1.6 | 4,120 |
| | Total | 3.4 | 9,471 | 3.2 | 8,773 | 3.0 | 8,089 | 3.6 | 9,307 |
| Population per Physician and Dentist | Government | | 463 | | 497 | | 540 | | 505 |
| | Private | | 818 | | 852 | | 894 | | 635 |
| | Total | | 296 | | 314 | | 337 | | 281 |
| No. of Physicians | Government | 2.0 | 5,558 | 1.8 | 5,098 | 1.7 | 4,634 | 1.8 | 4,685 |
| | Private | 0.7 | 2,086 | 0.7 | 1,937 | 0.7 | 1,832 | 0.9 | 2,467 |
| | Total | 2.7 | 7,644 | 2.5 | 7,035 | 2.4 | 6,466 | 2.7 | 7,152 |
| Population per Physicians | Government | | 504 | | 541 | | 588 | | 559 |
| | Private | | 1,342 | | 1,425 | | 1,487 | | 1,061 |
| | Total | | 366 | | 392 | | 421 | | 366 |

| Datasets | Metrics | ARIMA* | GPR* | RFR* | ETR* | Prophet* | ETS* | FFORMS* | FFORMA* |
|---|---|---|---|---|---|---|---|---|---|
| HDFS | *RMSE* | **0.0365** | 0.0794 | 0.0668 | 0.0505 | 0.1225 | 0.0402 | 0.0417 | 0.0400 |
| | *MAE* | 0.0157 | 0.0310 | 0.0263 | 0.0203 | 0.0448 | 0.0157 | **0.0154** | **0.0151** |
| | *p*-risk | **0.0058** | 0.0090 | 0.0071 | 0.0064 | 0.0090 | **0.0054** | 0.0068 | 0.0071 |
| Abase | *RMSE* | 0.0779 | **0.0512** | 0.0624 | 0.0540 | 0.0605 | 0.0944 | 0.0547 | 0.0515 |
| | *MAE* | 0.0522 | 0.0329 | 0.0415 | 0.0348 | 0.0414 | 0.0677 | 0.0342 | **0.0322** |
| | *p*-risk | 0.0627 | 0.0513 | 0.0666 | 0.0557 | 0.0573 | 0.0711 | 0.0560 | 0.0517 |
| FaaS | *RMSE* | 0.0366 | 0.0422 | 0.0429 | 0.0399 | 0.0505 | 0.0364 | 0.0359 | 0.0351 |
| | *MAE* | 0.0195 | 0.0235 | 0.0236 | 0.0220 | 0.0289 | 0.0199 | 0.0195 | 0.0190 |
| | *p*-risk | **0.0425** | 0.0476 | 0.0449 | **0.0443** | 0.0563 | 0.0444 | 0.0541 | 0.0527 |
| Alibaba | *RMSE* | 0.1049 | 0.1102 | 0.1077 | 0.1088 | 0.1235 | 0.1060 | 0.1097 | **0.1046** |
| | *MAE* | **0.0766** | 0.0825 | 0.0807 | 0.0810 | 0.0936 | 0.0775 | 0.0806 | 0.0772 |
| | *p*-risk | 0.1235 | 0.1320 | 0.1264 | 0.1276 | 0.1418 | 0.1269 | 0.1124 | 0.1099 |

Fig. 15. Example of Balanced Row Nesting

| Model | Parameters | Data | Ungrounded (⇓) | Grounded (⇑) | %↓ Ungrounded (⇑) | %↓ Grounded (⇓) |
|---|---|---|---|---|---|---|
| Orca | *Original model* | | *9.9* | *17.2* | - | - |
| | Sys. message | Synthetic | 7.8 | 16.0 | 21.4% | 6.8% |
| | | Synth. + Ref. | 8.0 | 16.9 | 19.2% | 1.4% |
| | Full model | Synthetic | 2.0 | 5.8 | 79.4% | 66.2% |
| | | Synth. + Ref. | 6.3 | 15.1 | 36.5% | 12.2% |
| Vicuna | *Original model* | | *9.4* | *17.6* | - | - |
| | Sys. message | Synthetic | 4.7 | 11.2 | 49.3% | 36.5% |
| | | Synth. + Ref. | 6.3 | 15.0 | 32.3% | 14.8% |
| | Full model | Synthetic | 1.4 | 0.2 | 84.9% | 99.0% |
| | | Synth. + Ref. | 5.3 | 14.5 | 43.2% | 17.6% |

Fig. 16. Example of Unbalanced Row Nesting

**Asymmetric Row Nesting:** Antithetical to symetric row nesting, this property entails the parent cells having different children cells. The table on the left in Figure 18 shows different children values for each of the parent cells (”Weight-based” with 5 children cells, ”Activation-based” with 2 children cells, ”Passport-based” with 2 children cells that have different values from all the others). The same can be seen in the table on the right as the children values are different for both parent cells.

**Global Column Aggregation:** This property entails that there is a column in the table that meets one of the following criteria: (1) is the aggregate of all columns in the table including or excluding local column-group aggregate columns or (2) is the aggregate of all local column-group aggregate columns present in the table. In this case an aggregation can mean any operation or function (e.g sum, max, min, average, different, or any other equation applied consistently across the column that uses values from other columns). For the first table in Figure 19, the column ”Total - Total” is a global aggregate as it meets both the conditions: it is the sum of all columns excluding ”Total - NonQatari” and ”Total - Qatari” which are local-group aggregate columns, and it is the sum of the aforementioned local-group aggregate columns (which are the only such columns in the table). Similarly, the ”Total” column in the second table and the ”Avg” column in the third table perform the sum and arithmetic mean (average) operations on all the other columns in their tables respectively.

**Local Column-Group Aggregation:** A column that aggregates only a subset of numerical columns (including other local column-group aggregation columns) within the table. The first table in Figure 20 has two such columns. ”Cultivable Land - Cultivated Land - Total” is the sum of all the other children columns under the parent cell ”Cultivable Land - Cultivated Land”. ”Cultivable Land - Total” is the sum of the aforementioned local-group aggregate and the ”Cultivable Land - UnCultivated Lands” column. Similarly, in second table the ”Tonne kilometres performed - Total” column sums all the children columns of the parent cell ”Tonne kilometres performed” and thus is a local column-group aggregate column.

| Datasets | Metrics | ARIMA* | GPR* | RFR* | ETR* | Prophet* | ETS* | FFORMS* |
|---|---|---|---|---|---|---|---|---|
| HDFS | *RMSE* | **0.0365** | 0.0794 | 0.0668 | 0.0505 | 0.1225 | 0.0402 | 0.0417 |
| | *MAE* | 0.0157 | 0.0310 | 0.0263 | 0.0203 | 0.0448 | 0.0157 | **0.0154** |
| | *ρ*-risk | **0.0058** | 0.0090 | 0.0071 | 0.0064 | 0.0090 | **0.0054** | 0.0068 |
| Abase | *RMSE* | 0.0779 | **0.0512** | 0.0624 | 0.0540 | 0.0605 | 0.0944 | 0.0547 |
| | *MAE* | 0.0522 | 0.0329 | 0.0415 | 0.0348 | 0.0414 | 0.0677 | 0.0342 |
| | *ρ*-risk | 0.0627 | 0.0513 | 0.0666 | 0.0557 | 0.0573 | 0.0711 | 0.0560 |
| FaaS | *RMSE* | 0.0366 | 0.0422 | 0.0429 | 0.0399 | 0.0505 | 0.0364 | 0.0359 |
| | *MAE* | 0.0195 | 0.0235 | 0.0236 | 0.0220 | 0.0289 | 0.0199 | 0.0195 |
| | *ρ*-risk | **0.0425** | 0.0476 | 0.0449 | **0.0443** | 0.0563 | 0.0444 | 0.0541 |
| Alibaba | *RMSE* | 0.1049 | 0.1102 | 0.1077 | 0.1088 | 0.1235 | 0.1060 | 0.1097 |
| | *MAE* | **0.0766** | 0.0825 | 0.0807 | 0.0810 | 0.0936 | 0.0775 | 0.0806 |
| | *ρ*-risk | 0.1235 | 0.1320 | 0.1264 | 0.1276 | 0.1418 | 0.1269 | 0.1124 |

| Hotel Class \ Year | Item | 2018 | 2017 | *2016 | *2015 |
|---|---|---|---|---|---|
| Five Star (*****) | No.of Hotels | 46 | 41 | 38 | 33 |
| | No.of Rooms | 12,109 | 11,167 | 10,435 | 9,196 |
| | No.of Beds | 17,478 | 16,041 | 15,103 | 13,599 |
| Four Star (****) | No.of Hotels | 33 | 26 | 25 | 24 |
| | No.of Rooms | 9,470 | 8,434 | 8,019 | 7,326 |
| | No.of Beds | 13,464 | 10,410 | 10,163 | 8,750 |
| Three Star (***) | No.of Hotels | 23 | 28 | 27 | 28 |
| | No.of Rooms | 2,219 | 2,610 | 2,268 | 2,336 |
| | No.of Beds | 3,637 | 3,993 | 3,609 | 3,604 |

Fig. 17. Example of Symmetric Row Nesting

| Category | Watermark Scheme | Model-Dataset | Model Utility | *w/o. Attack* |
|---|---|---|---|---|
| Weight-based | Uchida et al. [13] | WRN-Cifar10 | 91.55%/91.55% | 0 |
| | RIGA [14] | Inceptionv3-CelebA | 95.90%/95.90% | 0 |
| | IPR-GAN [18] | DCGAN-CUB200 | 54.33/54.33 (FID) | 0 |
| | Greedy Residuals [15] | ResNet18-Caltech256 | 55.05%/55.05% | 0 |
| | Lottery Verification [19] | ResNet18-Cifar100 | 66.40%/66.40% | 0 |
| Activation-based | DeepSigns [21] | WRN-Cifar10 | 89.94%/89.94% | 0 |
| | IPR-IC [20] | ResNet50+LSTM-COCO | 72.06/72.06 (BLEU-1) | 0 |
| Passport-based | DeepIPR [16] | ResNet18-Cifar100 | 67.94%/67.94% | 0 (67.89%) |
| | Passport-Aware [17] | ResNet18-Cifar100 | 74.78%/74.78% | 0 (72.74%) |

| Category | Type | العدد Number |
|---|---|---|
| Archaeological Heritage | Archaeological Settlement | 95 |
| | Cairn + cairn cluster | 2,500 |
| | Fish trap | 350 |
| | Midden | 55 |
| | Rock Carving | 50 |
| | Artefact Scatter | 1,000 |
| | Temporary Settlement/Camp / Mosque | 1,300 |
| | Date Press | 6 |
| | Earthenwork | 70 |
| | Hearth | 155 |
| | Shelter | 100 |
| | Shipwreck | 70 |
| | Other | 249 |
| | **Total** | **6,000** |
| Architectural Heritage | Fort / Fortification | 62 |
| | Settlement | 130 |
| | Residence / Homestead | 208 |
| | Islamic Grave / Cemetery | 561 |
| | Majlis | 2 |
| | Mosque | 105 |
| | Police / Coast Guard Station | 6 |
| | Stables | 3 |
| | Farmstead | 30 |
| | Well | 304 |
| | Wall / Enclosure | 544 |
| | Other | 45 |
| | **Total** | **2,000** |

Fig. 18. Example of Asymmetric Row Nesting

**Explicit Column Aggregation Terms:** A column where the cell value clearly indicates aggregation (e.g., "Total", "Percentage", "Sum"). For nested columns, if even one of the cells in the branch has an explicit cell value then the aggregation is considered as explicit. Figure 21 shows three tables that show columns having values like "Change", "Average", and "Total".

**Implicit Column Aggregation Terms:** A column that aggregates data, but the column name does not make this explicit. Figure 22 illustrates this ambiguity as the column "Yield" in the first table is the ratio of the other two columns. Although arguable that the name "yield" is indicative of a aggregation, we do not consider such terms that require some degree of domain familiarity to be understood. Thus, it is considered

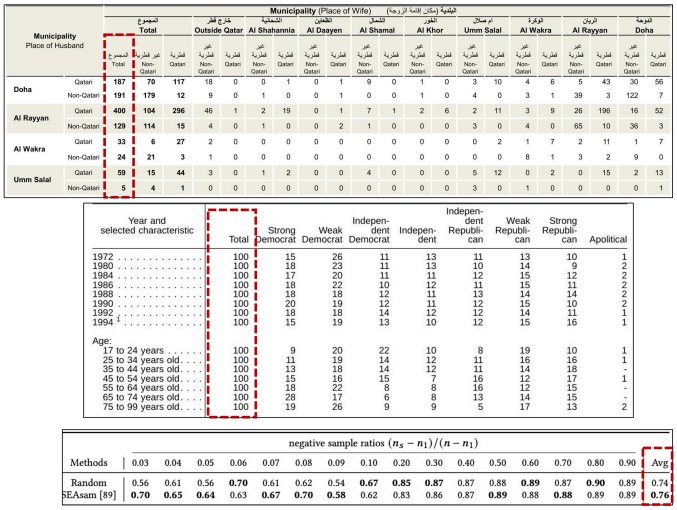
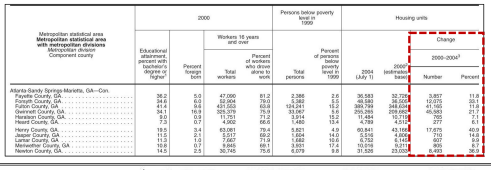



Fig. 19. Example of Global Column Aggregation

| Year | Total Areas of Qatar إجمالي مساحة اراضي دولة قطر | Total المجموع | Un-Cultivated Lands مساحة الاراضي غير المزروعة | Cultivated Land الأراضي المزروعة: Total المجموع | Green Fodder أعلاف خضراء | Date Palm نخيل | Fruits فاكهة | Vegetables خضراوات | Cereals حبوب |
|---|---|---|---|---|---|---|---|---|---|
| | | | | Cultivable Land الأراضي القابلة للزراعة | | | | | |
| 2015 | 1149300 | 65000 | 53195 | 11805 | 6666 | 2300 | 192 | 2339 | 308 |
| 2016 | 1149300 | 65000 | 53979 | 11021 | 5935 | 2407 | 245 | 2140 | 294 |
| 2017 | 1149300 | 65000 | 53410 | 11590 | 6583 | 2341 | 230 | 2159 | 277 |
| 2018 | 1149300 | 65000 | 51797 | 13203 | 7656 | 2418 | 85 | 2777 | 267 |
| 2019 | 1149300 | 65000 | 52093 | 12907 | 7620 | 2153 | 149 | 2832 | 153 |

| Year | Kilometres flown | Passenger Kilometres performed | Tonne kilometres performed | | | |
|---|---|---|---|---|---|---|
| | | | Passengers | Freight | Mail | Total |
| **Domestic scheduled** | | | | | | |
| 2004-05 | 15988 | 1710023 | 154136 | 39093 | 506 | 193735 |
| 2005-06 | 16518 | 1989274 | 179329 | 40053 | 586 | 219968 |
| 2006-07 | 15606 | 1909876 | 175162 | 36649 | 515 | 212326 |
| 2007-08 | 16687 | 1808827 | 162923 | 37833 | 379 | 201135 |
| 2008-09 | 16839 | 1769896 | 159294 | 28935 | 278 | 188507 |
| 2009-10 | 16801 | 1786435 | 160831 | 27862 | 178 | 188871 |
| 2010-11 | 16974 | 1984827 | 178913 | 27510 | 242 | 206665 |
| 2011-12 | 16226 | 1883788 | 169695 | 23856 | 183 | 193734 |
| 2012-13 | 11651 | 1422018 | 128214 | 18569 | 69 | 146852 |
| 2013-14 | 10447 | 1212607 | 110399 | 11042 | 49 | 121490 |

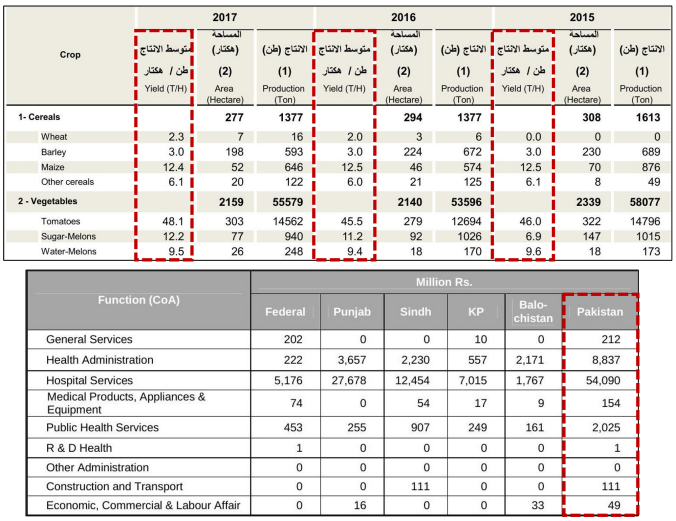
Fig. 20. Example of Local Column-Group Aggregation




| Caption Post-processing | YouCook2 R10↑ | YouCook2 MR↓ | MSR-VTT R10↑ | MSR-VTT MR↓ | MSVD R10↑ | MSVD MR↓ | LSMDC R10↑ | LSMDC MR↓ | **Average** R10↑ | **Average** MR↓ |
|---|---|---|---|---|---|---|---|---|---|---|
| Lower bound: original ASR as supervision | 39.3 | 20 | 61.7 | 5 | 77.1 | 2 | 31.5 | 56 | 52.4 | 20.8 |
| No post-processing | 40.2 | 18 | 65.9 | 4 | 79.8 | 2 | 34.4 | 40 | 55.1 | 16.0 |
| Filtering (using BLIP) | 42.5 | 16 | 71.2 | 3 | 81.7 | 2 | 37.4 | 30 | 58.2 | 12.8 |
| Filtering&alignment (using BLIP) | 42.4 | 17 | 71.7 | 3 | 82.2 | 2 | 38.5 | 29.5 | 58.7 | 12.9 |
| Filtering&alignment (with ours) | 44.1 | 15 | 73.3 | 3 | 82.1 | 2 | 38.6 | 29 | 59.5 | 12.3 |

| Age Group (in Years) / Nationality & Gender | Total المجموع: المجموع Total | Total: اناث Females | Total: ذكور Males | Non-Qataris غير قطريين: المجموع Total | Non-Qataris: اناث Females | Non-Qataris: ذكور Males | Qataris قطريون: المجموع Total | Qataris: اناث Females | Qataris: ذكور Males |
|---|---|---|---|---|---|---|---|---|---|
| Under 1 Year | 161 | 79 | 82 | 108 | 54 | 54 | 53 | 25 | 28 |
| 1 | 15 | 9 | 6 | 13 | 8 | 5 | 2 | 1 | 1 |
| 2 | 4 | 1 | 3 | 3 | 1 | 2 | 1 | 0 | 1 |
| 3 | 7 | 1 | 6 | 7 | 1 | 6 | 0 | 0 | 0 |
| 4 | 3 | 2 | 1 | 2 | 1 | 1 | 1 | 1 | 0 |
| Total | 190 | 92 | 98 | 133 | 65 | 68 | 57 | 27 | 30 |

Fig. 21. Example of Explicit Column Aggregation Terms



| Crop | 2017 متوسط الانتاج طن / هكتار Yield (T/H) | 2017 المساحة (هكتار) (2) Area (Hectare) | 2017 الانتاج (طن) (1) Production (Ton) | 2016 متوسط الانتاج طن / هكتار Yield (T/H) | 2016 المساحة (هكتار) (2) Area (Hectare) | 2016 الانتاج (طن) (1) Production (Ton) | 2015 متوسط الانتاج طن / هكتار Yield (T/H) | 2015 المساحة (هكتار) (2) Area (Hectare) | 2015 الانتاج (طن) (1) Production (Ton) |
|---|---|---|---|---|---|---|---|---|---|
| **1- Cereals** | | **277** | **1377** | | **294** | **1377** | | **308** | **1613** |
| Wheat | 2.3 | 7 | 16 | 2.0 | 3 | 6 | 0.0 | 0 | 0 |
| Barley | 3.0 | 198 | 593 | 3.0 | 224 | 672 | 3.0 | 230 | 689 |
| Maize | 12.4 | 52 | 646 | 12.5 | 46 | 574 | 12.5 | 70 | 876 |
| Other cereals | 6.1 | 20 | 122 | 6.0 | 21 | 125 | 6.1 | 8 | 49 |
| **2 - Vegetables** | | **2159** | **55579** | | **2140** | **53596** | | **2339** | **58077** |
| Tomatoes | 48.1 | 303 | 14562 | 45.5 | 279 | 12694 | 46.0 | 322 | 14796 |
| Sugar-Melons | 12.2 | 77 | 940 | 11.2 | 92 | 1026 | 6.9 | 147 | 1015 |
| Water-Melons | 9.5 | 26 | 248 | 9.4 | 18 | 170 | 9.6 | 18 | 173 |

| Function (CoA) | Million Rs. Federal | Punjab | Sindh | KP | Balo-chistan | Pakistan |
|---|---|---|---|---|---|---|
| General Services | 202 | 0 | 0 | 10 | 0 | 212 |
| Health Administration | 222 | 3,657 | 2,230 | 557 | 2,171 | 8,837 |
| Hospital Services | 5,176 | 27,678 | 12,454 | 7,015 | 1,767 | 54,090 |
| Medical Products, Appliances & Equipment | 74 | 0 | 54 | 17 | 9 | 154 |
| Public Health Services | 453 | 255 | 907 | 249 | 161 | 2,025 |
| R & D Health | 1 | 0 | 0 | 0 | 0 | 1 |
| Other Administration | 0 | 0 | 0 | 0 | 0 | 0 |
| Construction and Transport | 0 | 0 | 111 | 0 | 0 | 111 |
| Economic, Commercial & Labour Affair | 0 | 16 | 0 | 0 | 33 | 49 |

Fig. 22. Example of Implicit Column Aggregation Terms

as an implicit aggregation term. The second table presents a more definitive example of this property as the column ”Pakistan” is the summation of all the other columns (which represent different parts of Pakistan). Since it is difficult to understand that this is an aggregation without having prior knowledge about Pakistani geography, we consider this an implicit aggregation term.

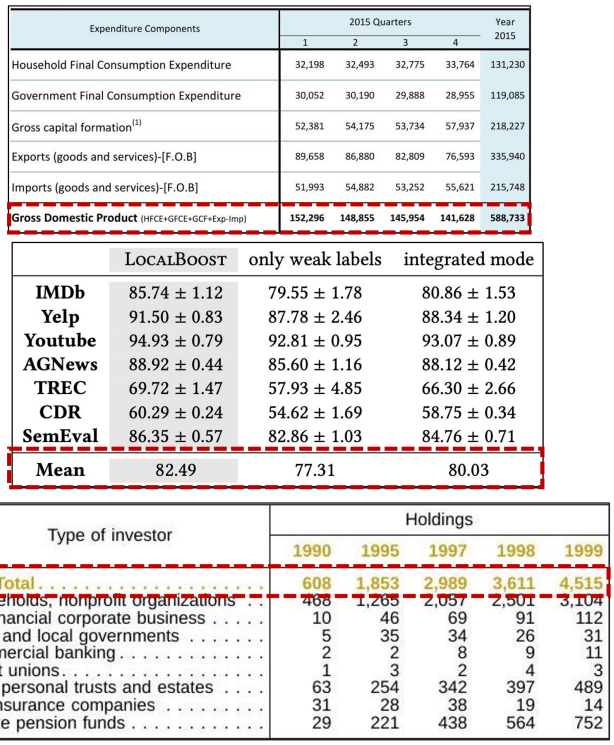
**Global Row Aggregation:** A row that aggregates (using any function) all other rows in the table. Figure 23 presents examples of such aggregate rows that use the sum and arithmetic mean aggregations.

**Local Row-Group Aggregation:** A row that aggregates a specific subset of rows (including other row-group aggregate rows) but not the entire table. Examples are shown in Figure 24 where the highlighted rows aggregate (sum in this case) subsets of rows. This property in particular has various visual manifestations: indentation in the first table, numbered rows in the second, explicit ”total” cell in third.

**Explicit Row Aggregation Terms:** A row where at least one cell explicitly indicates aggregation (e.g., “Total”, “Average”, “Sum”). This is clearly shown in Figure 25.

**Implicit Row Aggregation Terms:** A row that likely aggregates data, but none of its textual cells explicitly say so. Figure 26 illustrates this. As stated earlier, this property is often shown using numbered cells, indentation, domain specific terms (such as ”Gross Domestic Product”, font stylization, and other visual characteristics.

**Row Group Label:** A row that acts as a heading or title for a set of rows above or below it. See Figure 27 for examples. The first table has the row group labels ”Pakistan” and ”Punjab” respectively. The second table has ”C. Oil and Non-Oil”.



| Expenditure Components | 2015 Quarters 1 | 2 | 3 | 4 | Year 2015 |
|---|---|---|---|---|---|
| Household Final Consumption Expenditure | 32,198 | 32,493 | 32,775 | 33,764 | 131,230 |
| Government Final Consumption Expenditure | 30,052 | 30,190 | 29,888 | 28,955 | 119,085 |
| Gross capital formation[1] | 52,381 | 54,175 | 53,734 | 57,937 | 218,227 |
| Exports (goods and services)-[F.O.B] | 89,658 | 86,880 | 82,809 | 76,593 | 335,940 |
| Imports (goods and services)-[F.O.B] | 51,993 | 54,882 | 53,252 | 55,621 | 215,748 |
| **Gross Domestic Product** (HFCE+GFCE+GCF+Exp-Imp) | **152,296** | **148,855** | **145,954** | **141,628** | **588,733** |

| | LocalBoost | only weak labels | integrated mode |
|---|---|---|---|
| **IMDb** | 85.74 ± 1.12 | 79.55 ± 1.78 | 80.86 ± 1.53 |
| **Yelp** | 91.50 ± 0.83 | 87.78 ± 2.46 | 88.34 ± 1.20 |
| **Youtube** | 94.93 ± 0.79 | 92.81 ± 0.95 | 93.07 ± 0.89 |
| **AGNews** | 88.92 ± 0.44 | 85.60 ± 1.16 | 88.12 ± 0.42 |
| **TREC** | 69.72 ± 1.47 | 57.93 ± 4.85 | 66.30 ± 2.66 |
| **CDR** | 60.29 ± 0.24 | 54.62 ± 1.69 | 58.75 ± 0.34 |
| **SemEval** | 86.35 ± 0.57 | 82.86 ± 1.03 | 84.76 ± 0.71 |
| **Mean** | 82.49 | 77.31 | 80.03 |

| Type of investor | Holdings 1990 | 1995 | 1997 | 1998 | 1999 |
|---|---|---|---|---|---|
| Total | 608 | 1,853 | 2,989 | 3,611 | 4,515 |
| Households, nonprofit organizations | 468 | 1,265 | 2,057 | 2,501 | 3,104 |
| Nonfinancial corporate business | 10 | 46 | 69 | 91 | 112 |
| State and local governments | 5 | 35 | 34 | 26 | 31 |
| Commercial banking | 2 | 2 | 8 | 9 | 11 |
| Credit unions | 1 | 3 | 2 | 4 | 3 |
| Bank personal trusts and estates | 63 | 254 | 342 | 397 | 489 |
| Life insurance companies | 31 | 28 | 38 | 19 | 14 |
| Private pension funds | 29 | 221 | 438 | 564 | 752 |

Fig. 23. Example of Global Row Aggregation

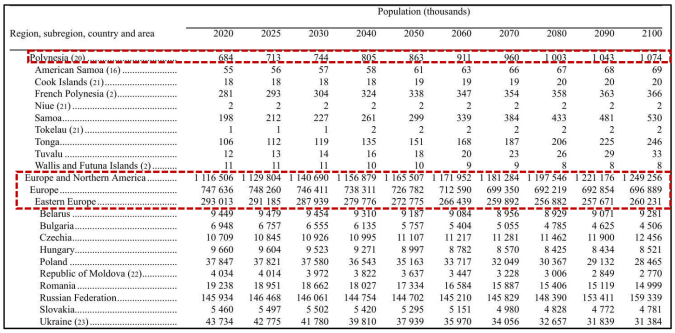
**Split Header Cell:** A single header cell split diagonally and containing two distinct values. This visual property is shown in Figure 28. In the first table the term ”Sector” describes the column names and ”Occupation” the values in the column below it. Similarly, in the second table, ”Year” describes the other column names, and ”Type of License” is the column name for that particular column.

**Standard Relational Table:** A table that does not exhibit any of the defined HCT properties — no nesting, no aggregation, no labels. Figure 29 shows examples of this simple table form.

### *B. Table Annotation Tool*

We developed a user-friendly tool for annotating tables and setting their properties as illustrated in Figure 30. The left side visualizes one table at a time, while the right side lists the possible allowed properties.

As an example, the illustrated table in Figure 30 contains “Column Nesting”. And since the child column names (e.g., {“Grand Total”, “Females”, “Males”}) across all groups are at the same level (same depth from the top parent cell), it is considered a “Balanced Column Nesting”. Similarly, since all branches of the parents have identical structure and cell values, the column nesting is considered “Symmetric”. Similar rules apply to the “Row Nesting” property. For example, the table in Figure 30 does not containing any row nesting, thus that property is not selected. The table also contains column aggregation at both levels, local (e.g., “Total” under



| Region, subregion, country and area | Population (thousands) 2020 | 2025 | 2030 | 2040 | 2050 | 2060 | 2070 | 2080 | 2090 | 2100 |
|---|---|---|---|---|---|---|---|---|---|---|
| Polynesia (20) | 684 | 713 | 744 | 805 | 863 | 911 | 960 | 1 003 | 1 043 | 1 074 |
| American Samoa (16) | 55 | 56 | 57 | 58 | 61 | 63 | 66 | 67 | 68 | 69 |
| Cook Islands (21) | 18 | 18 | 18 | 18 | 19 | 19 | 19 | 20 | 20 | 20 |
| French Polynesia (2) | 281 | 293 | 304 | 324 | 338 | 347 | 354 | 358 | 363 | 366 |
| Niue (21) | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| Samoa | 198 | 212 | 227 | 261 | 299 | 339 | 384 | 433 | 481 | 530 |
| Tokelau (21) | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| Tonga | 106 | 112 | 119 | 135 | 151 | 168 | 187 | 206 | 225 | 246 |
| Tuvalu | 12 | 13 | 14 | 16 | 18 | 20 | 23 | 26 | 29 | 33 |
| Wallis and Futuna Islands (2) | 11 | 11 | 11 | 10 | 10 | 9 | 9 | 8 | 8 | 8 |
| Europe and Northern America | 1 116 506 | 1 129 804 | 1 140 690 | 1 156 879 | 1 165 507 | 1 171 952 | 1 181 264 | 1 197 546 | 1 221 176 | 1 249 256 |
| Europe | 747 636 | 748 260 | 746 411 | 738 311 | 726 782 | 712 590 | 699 350 | 692 219 | 692 854 | 696 889 |
| Eastern Europe | 293 013 | 291 185 | 287 939 | 279 776 | 272 775 | 266 439 | 259 892 | 256 882 | 257 671 | 260 231 |
| Belarus | 9 449 | 9 479 | 9 454 | 9 310 | 9 187 | 9 084 | 8 956 | 8 929 | 9 071 | 9 281 |
| Bulgaria | 6 948 | 6 757 | 6 555 | 6 135 | 5 757 | 5 404 | 5 055 | 4 785 | 4 625 | 4 506 |
| Czechia | 10 709 | 10 845 | 10 926 | 10 995 | 11 107 | 11 217 | 11 281 | 11 462 | 11 900 | 12 456 |
| Hungary | 9 660 | 9 604 | 9 523 | 9 271 | 8 997 | 8 782 | 8 570 | 8 425 | 8 434 | 8 521 |
| Poland | 37 847 | 37 821 | 37 580 | 36 543 | 35 163 | 33 717 | 32 049 | 30 367 | 29 132 | 28 465 |
| Republic of Moldova (22) | 4 034 | 4 014 | 3 972 | 3 822 | 3 637 | 3 447 | 3 228 | 3 006 | 2 849 | 2 770 |
| Romania | 19 238 | 18 951 | 18 662 | 18 027 | 17 334 | 16 584 | 15 887 | 15 406 | 15 119 | 14 999 |
| Russian Federation | 145 934 | 146 468 | 146 061 | 144 754 | 144 702 | 145 210 | 145 829 | 148 390 | 153 411 | 159 339 |
| Slovakia | 5 460 | 5 497 | 5 502 | 5 420 | 5 295 | 5 151 | 4 980 | 4 828 | 4 772 | 4 781 |
| Ukraine (23) | 43 734 | 42 775 | 41 780 | 39 810 | 37 939 | 35 970 | 34 056 | 32 657 | 31 839 | 31 384 |

| Crop | 2017 متوسط الانتاج طن / هكتار Yield (T/H) | 2017 المساحة (هكتار) (2) Area (Hectare) | 2017 الانتاج (طن) (1) Production (Ton) | 2016 متوسط الانتاج طن / هكتار Yield (T/H) | 2016 المساحة (هكتار) (2) Area (Hectare) | 2016 الانتاج (طن) (1) Production (Ton) |
|---|---|---|---|---|---|---|
| **1- Cereals** | | **277** | **1377** | | **294** | **1377** |
| Wheat | 2.3 | 7 | 16 | 2.0 | 3 | 6 |
| Barley | 3.0 | 198 | 593 | 3.0 | 224 | 672 |
| Maize | 12.4 | 52 | 646 | 12.5 | 46 | 574 |
| Other cereals | 6.1 | 20 | 122 | 6.0 | 21 | 125 |
| **2 - Vegetables** | | **2159** | **55579** | | **2140** | **53596** |
| Tomatoes | 48.1 | 303 | 14562 | 45.5 | 279 | 12694 |
| Sugar-Melons | 12.2 | 77 | 940 | 11.2 | 92 | 1026 |
| Water-Melons | 9.5 | 26 | 248 | 9.4 | 18 | 170 |
| Cucumbers | 97.5 | 123 | 11992 | 97.6 | 134 | 13081 |
| Marrows | 16.0 | 228 | 3645 | 16.0 | 228 | 3655 |
| Pumpkin | 20.0 | 154 | 3083 | 20.0 | 139 | 2774 |

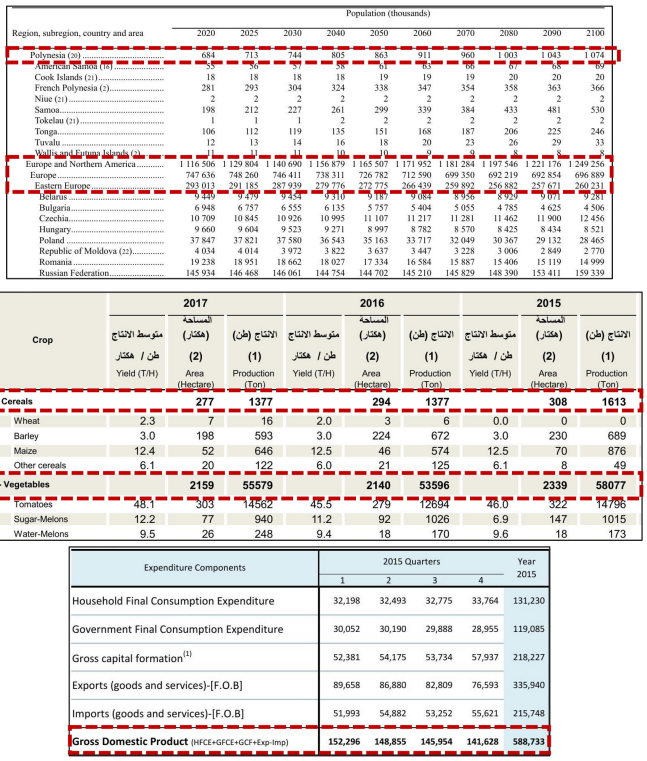
| Indicators | Sector | 2019 *المعدل Rate* | 2019 العدد Number | 2018 *المعدل Rate* | 2018 العدد Number | 2017 *المعدل Rate* | 2017 العدد Number |
|---|---|---|---|---|---|---|---|
| Total No. of Physicians and Dentists in Qatar | Government | 2.2 | 6,047 | 2.0 | 5,535 | 1.9 | 5,041 |
| | Private | 1.2 | 3,424 | 1.2 | 3,238 | 1.1 | 3,048 |
| | Total | 3.4 | 9,471 | 3.2 | 8,773 | 3.0 | 8,089 |
| Population per Physician and Dentist | Government | | 463 | | 497 | | 540 |
| | Private | | 818 | | 852 | | 894 |
| | Total | | 296 | | 314 | | 337 |
| No. of Physicians | Government | 2.0 | 5,558 | 1.8 | 5,098 | 1.7 | 4,634 |
| | Private | 0.7 | 2,086 | 0.7 | 1,937 | 0.7 | 1,832 |
| | Total | 2.7 | 7,644 | 2.5 | 7,035 | 2.4 | 6,466 |

Fig. 24. Example of Local Row-Group Aggregation

| Indicators | Sector | 2019 *المعدل Rate* | 2019 العدد Number | 2018 *المعدل Rate* | 2018 العدد Number | 2017 *المعدل Rate* | 2017 العدد Number | 2016 *المعدل Rate* | 2016 العدد Number |
|---|---|---|---|---|---|---|---|---|---|
| Total No. of Physicians and Dentists in Qatar | Government | 2.2 | 6,047 | 2.0 | 5,535 | 1.9 | 5,041 | 2.0 | 5,187 |
| | Private | 1.2 | 3,424 | 1.2 | 3,238 | 1.1 | 3,048 | 1.6 | 4,120 |
| | Total | 3.4 | 9,471 | 3.2 | 8,773 | 3.0 | 8,089 | 3.6 | 9,307 |
| Population per Physician and Dentist | Government | | 463 | | 497 | | 540 | | 505 |
| | Private | | 818 | | 852 | | 894 | | 635 |
| | Total | | 296 | | 314 | | 337 | | 281 |
| No. of Physicians | Government | 2.0 | 5,558 | 1.8 | 5,098 | 1.7 | 4,634 | 1.8 | 4,685 |
| | Private | 0.7 | 2,086 | 0.7 | 1,937 | 0.7 | 1,832 | 0.9 | 2,467 |
| | Total | 2.7 | 7,644 | 2.5 | 7,035 | 2.4 | 6,466 | 2.7 | 7,152 |

| | DQ1 | DQ2 | DQ3 | DQ4 | DQ5 | DQ6 | DQ7 | DQ8 | DQ9 | DQ10 | DQ11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| AQ2019Q1 | 1 | 2 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| AQ2019Q2 | 2 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| AQ2019Q3 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| AQ2019Q4 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| AQ2020Q1 | 0 | 2 | 18 | 8 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| AQ2020Q2 | 1 | 5 | 1 | 0 | 1 | 2 | 0 | 0 | 0 | 0 | 0 |
| AQ2020Q3 | 2 | 1 | 2 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| AQ2020Q4 | 0 | 3 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| AQ2021Q1 | 2 | 3 | 5 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| AQ2021Q2 | 3 | 3 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| AQ2021Q3 | 1 | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| AQ2021Q4 | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Sum | 17 | 23 | 29 | 12 | 5 | 2 | 0 | 2 | 0 | 0 | 0 |

Fig. 25. Example of Explicit Row Aggregation Terms



| Region, subregion, country and area | Population (thousands) 2020 | 2025 | 2030 | 2040 | 2050 | 2060 | 2070 | 2080 | 2090 | 2100 |
|---|---|---|---|---|---|---|---|---|---|---|
| Polynesia (20) | 684 | 713 | 744 | 805 | 863 | 911 | 960 | 1 003 | 1 043 | 1 074 |
| American Samoa (16) | 55 | 56 | 57 | 58 | 61 | 63 | 66 | 67 | 68 | 69 |
| Cook Islands (21) | 18 | 18 | 18 | 18 | 19 | 19 | 19 | 20 | 20 | 20 |
| French Polynesia (2) | 281 | 293 | 304 | 324 | 338 | 347 | 354 | 358 | 363 | 366 |
| Niue (21) | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| Samoa | 198 | 212 | 227 | 261 | 299 | 339 | 384 | 433 | 481 | 530 |
| Tokelau (21) | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| Tonga | 106 | 112 | 119 | 135 | 151 | 168 | 187 | 206 | 225 | 246 |
| Tuvalu | 12 | 13 | 14 | 16 | 18 | 20 | 23 | 26 | 29 | 33 |
| Wallis and Futuna Islands (2) | 11 | 11 | 11 | 10 | 10 | 9 | 9 | 8 | 8 | 8 |
| Europe and Northern America | 1 116 506 | 1 129 804 | 1 140 690 | 1 156 879 | 1 165 507 | 1 171 952 | 1 181 264 | 1 197 546 | 1 221 176 | 1 249 256 |
| Europe | 747 636 | 748 260 | 746 411 | 738 311 | 726 782 | 712 590 | 699 350 | 692 219 | 692 854 | 696 889 |
| Eastern Europe | 293 013 | 291 185 | 287 939 | 279 776 | 272 775 | 266 439 | 259 892 | 256 882 | 257 671 | 260 231 |
| Belarus | 9 449 | 9 479 | 9 454 | 9 310 | 9 187 | 9 084 | 8 956 | 8 929 | 9 071 | 9 281 |
| Bulgaria | 6 948 | 6 757 | 6 555 | 6 135 | 5 757 | 5 404 | 5 055 | 4 785 | 4 625 | 4 506 |
| Czechia | 10 709 | 10 845 | 10 926 | 10 995 | 11 107 | 11 217 | 11 281 | 11 462 | 11 900 | 12 456 |
| Hungary | 9 660 | 9 604 | 9 523 | 9 271 | 8 997 | 8 782 | 8 570 | 8 425 | 8 434 | 8 521 |
| Poland | 37 847 | 37 821 | 37 580 | 36 543 | 35 163 | 33 717 | 32 049 | 30 367 | 29 132 | 28 465 |
| Republic of Moldova (22) | 4 034 | 4 014 | 3 972 | 3 822 | 3 637 | 3 447 | 3 228 | 3 006 | 2 849 | 2 770 |
| Romania | 19 238 | 18 951 | 18 662 | 18 027 | 17 334 | 16 584 | 15 887 | 15 406 | 15 119 | 14 999 |
| Russian Federation | 145 934 | 146 468 | 146 061 | 144 754 | 144 702 | 145 210 | 145 829 | 148 390 | 153 411 | 159 339 |

| Crop | 2017 متوسط الانتاج طن / هكتار Yield (T/H) | 2017 المساحة (هكتار) (2) Area (Hectare) | 2017 الانتاج (طن) (1) Production (Ton) | 2016 متوسط الانتاج طن / هكتار Yield (T/H) | 2016 المساحة (هكتار) (2) Area (Hectare) | 2016 الانتاج (طن) (1) Production (Ton) | 2015 متوسط الانتاج طن / هكتار Yield (T/H) | 2015 المساحة (هكتار) (2) Area (Hectare) | 2015 الانتاج (طن) (1) Production (Ton) |
|---|---|---|---|---|---|---|---|---|---|
| **1- Cereals** | | **277** | **1377** | | **294** | **1377** | | **308** | **1613** |
| Wheat | 2.3 | 7 | 16 | 2.0 | 3 | 6 | 0.0 | 0 | 0 |
| Barley | 3.0 | 198 | 593 | 3.0 | 224 | 672 | 3.0 | 230 | 689 |
| Maize | 12.4 | 52 | 646 | 12.5 | 46 | 574 | 12.5 | 70 | 876 |
| Other cereals | 6.1 | 20 | 122 | 6.0 | 21 | 125 | 6.1 | 8 | 49 |
| **2 - Vegetables** | | **2159** | **55579** | | **2140** | **53596** | | **2339** | **58077** |
| Tomatoes | 48.1 | 303 | 14562 | 45.5 | 279 | 12694 | 46.0 | 322 | 14796 |
| Sugar-Melons | 12.2 | 77 | 940 | 11.2 | 92 | 1026 | 6.9 | 147 | 1015 |
| Water-Melons | 9.5 | 26 | 248 | 9.4 | 18 | 170 | 9.6 | 18 | 173 |

| Expenditure Components | 2015 Quarters 1 | 2 | 3 | 4 | Year 2015 |
|---|---|---|---|---|---|
| Household Final Consumption Expenditure | 32,198 | 32,493 | 32,775 | 33,764 | 131,230 |
| Government Final Consumption Expenditure | 30,052 | 30,190 | 29,888 | 28,955 | 119,085 |
| Gross capital formation[1] | 52,381 | 54,175 | 53,734 | 57,937 | 218,227 |
| Exports (goods and services)-[F.O.B] | 89,658 | 86,880 | 82,809 | 76,593 | 335,940 |
| Imports (goods and services)-[F.O.B] | 51,993 | 54,882 | 53,252 | 55,621 | 215,748 |
| **Gross Domestic Product** (HFCE+GFCE+GCF+Exp-Imp) | **152,296** | **148,855** | **145,954** | **141,628** | **588,733** |

Fig. 26. Example of Implicit Row Aggregation Terms

"Non-Qataris", "Males" under "Grand Total"), and global (e.g., "Total" under "Grand Total"). At the row level, this table contains only global aggregation (the last row). We also keep track of whether the aggregation columns carry explicit aggregate-meaningful labels (e.g., SUM, TOTAL, AVERAGE, MIN, MAX, etc.) as in the table illustrated in Figure 30.

Certainly, given the compound complexity among the different properties, we could have captured the properties at a more coarse or grained granularity. Nevertheless, we opt for this level of details because it sufficiently captures the underlying complexity of the HCT objects, and would provide good insights on how these properties correlate to QA performance without being dragged into too many features which requires high dimensional analysis. the annotation tool was developed in-house and we plan to release it on GitHub[1] page soon.

### *C. Understanding the distribution of table properties*

The Figure 31 and Figure 32 show the distribution of the top-5 main and detailed real table properties per dataset. The property distribution is very different between datasets. For instance, column and row nesting are present in **QNPC** and **ArXiv** datasets (Figure 31(a) and (d)) but no row nesting appears in **US C.** or **Pak C.** datasets. Column and row aggregation appear in all datasets except in **ArXiv** with only a small proportion of row aggregation. **QNPC** datasets have the most various and complex combinations of main properties, while **ArXiv** have the least various.

[1]https://github.com/shahmeer99/HCT-QA-Benchmark/

| Area/ Age Group | Urban | | Rural | |
|---|---|---|---|---|
| | Women | Men | Women | Men |
| **Pakistan** | | | | |
| Total | 61.81 | 57.05 | 68.05 | 61.84 |
| 15-19 years | 6.37 | 1.32 | 14.59 | 3.26 |
| 20-24 years | 39.75 | 15.24 | 59.19 | 29.29 |
| 25-29 years | 76.32 | 49.86 | 85.47 | 67.33 |
| 30-34 years | 91.70 | 79.45 | 94.15 | 88.35 |
| 35-39 years | 93.74 | 92.13 | 94.83 | 94.41 |
| 40-44 years | 92.93 | 96.07 | 94.10 | 96.60 |
| 45-49 years | 87.74 | 96.36 | 90.72 | 96.47 |
| 50-54 years | 81.53 | 92.69 | 84.37 | 95.32 |
| 55-59 years | 69.55 | 89.96 | 77.04 | 92.67 |
| 60+ | 43.99 | 78.24 | 48.07 | 79.28 |
| **Punjab** | | | | |
| Total | 60.77 | 57.05 | 66.12 | 61.59 |
| 15-19 years | 5.47 | 1.32 | 12.67 | 3.35 |
| 20-24 years | 40.97 | 16.31 | 55.18 | 28.96 |
| 25-29 years | 76.26 | 50.85 | 83.36 | 64.85 |
| 30-34 years | 91.26 | 80.57 | 93.05 | 86.50 |
| 35-39 years | 91.99 | 91.21 | 94.04 | 93.00 |
| 40-44 years | 91.43 | 95.51 | 92.86 | 95.60 |
| 45-49 years | 84.55 | 96.62 | 90.42 | 95.54 |
| 50-54 years | 84.71 | 92.12 | 83.23 | 94.00 |
| 55-59 years | 69.29 | 90.87 | 78.75 | 91.67 |
| 60+ | 45.83 | 76.31 | 50.19 | 77.41 |

| ITEMS | ISIC4 Codes | 2014 | 2015 | 2016 | 2017* |
|---|---|---|---|---|---|
| **C. Oil and Non-Oil** | | | | | |
| 1 - **Oil And gas (Mining)** | | **271,452** | **269,343** | **267,061** | **261,014** |
| - Percent Change | | -0.7 | -0.8 | -0.8 | -2.3 |
| - Percent of Total | | 40.4 | 40.1 | 39.7 | 38.8 |
| 2 - **Non-oil and gas (Non-Mining)** | | **365,116** | **398,934** | **423,600** | **422,928** |
| - Percent Change | | 11.2 | 9.3 | 6.2 | -0.2 |
| - Percent of Total | | 58.9 | 61.4 | 63.3 | 64.2 |
| 3 - **Financial intermediation services indirectly measured (FISIM)** | | **-19,215** | **-20,879** | **-22,637** | **-25,597** |
| - Percent Change | | 16.2 | 8.7 | 8.4 | 13.1 |
| - Percent of Total | | -2.9 | -3.1 | -3.4 | -3.8 |
| **Gross Value Added (GVA) at producer price** | | **617,353** | **647,397** | **668,024** | **658,344** |
| **- Percent Change** | | 5.5 | 4.9 | 3.2 | -1.4 |
| **- Percent of Total** | | 99.6 | 99.7 | 99.8 | 99.9 |
| **Import duties** | | **2,508** | **1,927** | **1,197** | **854** |
| **- Percent Change** | | -28.0 | -23.2 | -37.9 | -28.6 |
| **- Percent of Total** | | 0.4 | 0.3 | 0.2 | 0.1 |
| **Gross domestic product (GDP) at market prices** | | **619,861** | **649,325** | **669,221** | **659,199** |
| **- Percent Change** | | 5.3 | 4.8 | 3.1 | -1.5 |
| **- Percent of Total** | | 100.0 | 100.0 | 100.0 | 100.0 |

Fig. 27. Example of Row Group Label

| Occupation \ Sector | المجموع Total | منزلي Domestic | غير ربحي Non profit | دبلوماسي / دولي / اقليمي Diplomatic/International/Regional | خاص Private |
|---|---|---|---|---|---|
| Legislators, Senior Officials And Managers | **6,257** | 0 | 14 | 82 | 3,763 |
| Professionals | **62,168** | 335 | 412 | 712 | 30,400 |
| Technicians And Associate Professionals | **14,219** | 112 | 98 | 110 | 6,056 |
| Clerks | **32,975** | 0 | 672 | 480 | 17,166 |

| Type of License \ Year | 2016 | 2015 | 2014 |
|---|---|---|---|
| Government | 3522 | 1176 | 2559 |
| Private | 827235 | 697032 | 647923 |
| Private Transport | 347678 | 271719 | 246340 |
| Heavy Equipment | 35135 | 32060 | 25753 |
| Taxis | *10924 | 28997 | 28419 |

Fig. 28. Example of Split Header Cell

| Particulars | 2020 | 2019 | 2018 | 2017 | 2016 |
|---|---|---|---|---|---|
| No. of Telephone Lines | 454,701 | 461,289 | 452,088 | 454,282 | 467,148 |
| No. of G.S.M. Subscribers (Normal) | 1,025,525 | 976,015 | 842,044 | 893,739 | 783,310 |
| No. of G.S.M. Subscribers (Prepaid services) | 2,772,989 | 2,941,556 | 3,103,934 | 2,678,695 | 2,869,269 |
| No. of Internet Subscribers | 296,126 | 284,679 | 267,906 | 248,793 | 253,718 |
| Public Phones | 238 | 238 | 226 | 221 | 212 |

| Dataset | Nodes | Edges | $d_{max}$ | $k_{max}$ | $\|S\|_{max}$ |
|---|---|---|---|---|---|
| Brightkite | 58,228 | 214,078 | 1,134 | 52 | 11,838 |
| Github | 37,700 | 289,003 | 9,458 | 34 | 20,976 |
| Gowalla | 196,591 | 950,327 | 14,730 | 51 | 14,060 |
| NotreDame | 325,729 | 1,090,108 | 10,721 | 155 | 215,052 |
| Stanford | 281,903 | 1,992,636 | 38,625 | 71 | 89,688 |
| Youtube | 1,134,890 | 2,987,624 | 28,754 | 51 | 72,726 |
| DBLP | 1,566,919 | 6,461,300 | 1,522 | 118 | 89,276 |
| Yelp | 1,032,416 | 17,971,548 | 6,367 | 165 | 269,238 |
| Orkut | 3,072,441 | 117,185,083 | 33,313 | 253 | 6,889,284 |

Fig. 29. Example of Standard Relational Table

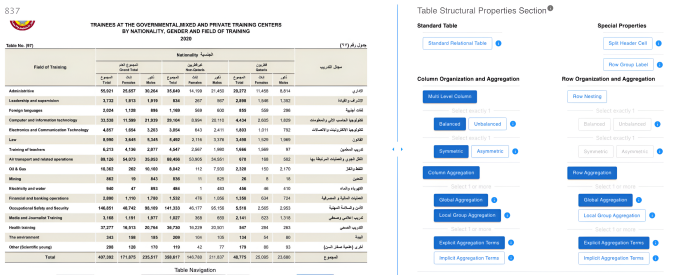



Fig. 30. A user-friendly tool for annotating/viewing HCTs and their properties.

Regarding detailed table properties in Figure 32, **QNPC** and **US C.** show quite complex and various combinations compared to **Pak. C** and **ArXiv**. While balanced column nesting is a typical property for all the tables, it is the major one in **Pak. C** and **ArXiv** while it is more balanced in the other two datasets.

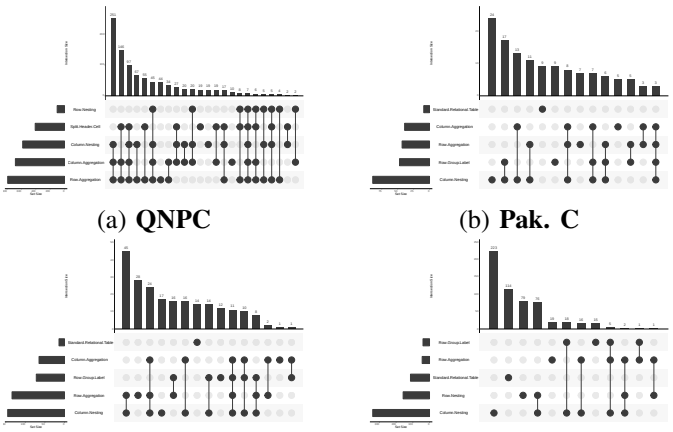
### *D. Synthetic Generator*

In the following subsections, we detail the configuration templates we used for setting the domain vocabulary (Section E) common to both HCTs, relational tables, SQL queries, and natural language questions. Then we detail the templates used for generating the synthetic HCTs (Section F), and those used for generating the SQL queries and the Natural Language questions (Section G).

The codes of the synthetic generator and the instructions for installing and running them are available at https://github.com/qcri/HCTQA-Benchmark/blob/main/synthetic_data_generator/README_SYNTHETIC_GENERATOR.md.

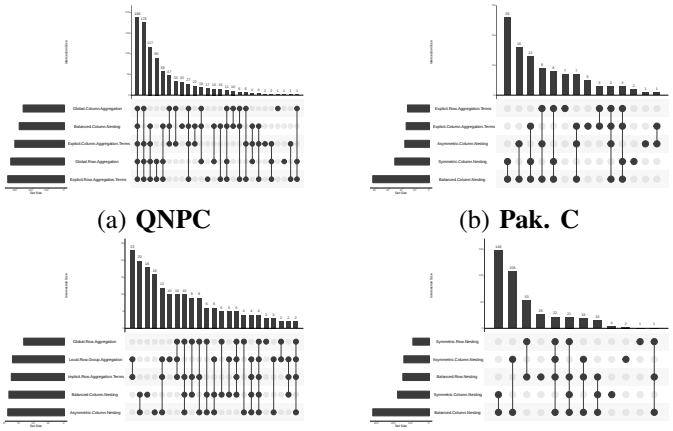
### *E. Domain vocabulary templates*

In this section, all `terms` refer to Figure 33.



(a) **QNPC** (b) **Pak. C** (c) **US C.** (d) **ArXiv**

Fig. 31. Distribution of main table properties for each real dataset.



(a) **QNPC** (b) **Pak. C** (c) **US C.** (d) **ArXiv**

Fig. 32. Distribution of detailed table properties for each real dataset.

Our synthetic data relies on domain vocabulary naming the dataset's attributes and values. A synthetic $T_{HCT}$ (Figure 33b) is obtained by pivoting a relational table $T_{REL}$ (Figure 33a) composed of at least two nominal attributes and a single numerical attribute always named `Value`.

There are two types of nominal attributes:

- **Independent attributes** like `Import-Export`, `Years`, or `Quarters` can be organized as row or column headers at any nesting level in $T_{HCT}$.
- **Hierarchical attributes** like food `Category` (*e.g.* `Meat, Cereals, Dairy, Beverage`) and food `Item` (e.g., `Cream, Soda, Milk`) are dependent on each other. If both attributes appear in the HCT, the food `Item` must be displayed at a lower level than the food `Category`, *i.e.* closer to the inner part of the HCT, and all must appear together either in row headers or column headers.

Domain vocabulary is encoded in a JSON file (Figure 33c) with two main keys: `data` and `values`.

- `data` refers to a list of attributes (e.g., `ImportExport`, `Years`, `Quarters`) or groups of attributes (e.g., `Food`), each described with three keys:
  - `code` is the primary key used in table and question templates to identify the independent attributes (e.g., `ImportExport` or `Years`) and the hierarchical groups of attributes (e.g., `Food`).
  - `names` is the list of attribute names displayed in $T_{REL}$ column headers but typically hidden (implicit) in $T_{HCT}$ and $Q_{NL}$. It is a list with a single name for independent attributes, or with as many names as levels for hierarchical groups of attributes (e.g., `Food`), ordered from top (outer) to bottom (inner) levels (e.g., `Category`, `Item`).
  - `values` is the list of values (e.g., `2015, 2016...` for the attribute `Year`). It can also be the hierarchy of values of a group of attributes. For instance, in `Food` group, `Meat` is a value of the attribute `Category`, and `Beef`, `Chicken`..., are values of the attribute `Item`, whose parents are `Meat` in the `Food` hierarchy. Attribute values are always displayed as row or column headers in $T_{HCT}$.
- `values` refers to the numerical values to be generated in both the core part of $T_{HCT}$ and the `Value` column of $T_{REL}$. This object contains shortcut keys (e.g., `realUnit` or `intSigned`) to generate specific ranges and types of numbers when called in table templates. The value associated with such shortcuts must be a list of two numbers $[m, M]$ representing the minimum $m$ and maximum $M$ possible values. If any of these min and max is a decimal number, the generated value will be a real number with two decimal digits drawn uniformly in this interval. If both min and max are integers, the generated value will be an integer drawn uniformly in this interval. The generator will ensure no replica in the generated values, reals or integers, possibly overflowing the max value if asked to generate more numeric values than the range allows.

**Customization of domain vocabulary** All `terms` in domain vocabulary are customizable except the JSON keys `data`, `code`, `names`, and `values`, following the rules above. The number of decimals of generated real numbers is specified by `NUM_DECIMAL_DIGITS_REAL_FORMAT` in `config.R`. The domain vocabulary is stored in the file `PARAM_semantics.json`.

### *F. Table templates*

The generic table templates described in the file `PARAM_tableTemplate.json` are used to generate a (large) set of $T_{REL}$ and $T_{HCT}$ tables from the domain vocabulary (See Section E).

This template contains the following keys:

- `replica` is an integer setting how many *random* variants of each initial table template from the list `tables` will be generated.
- `shuffle` is a list of column and row shuffling commands that will be applied to all the initial table templates listed in `tables`. It is used to vary the structure

a Food_import-export_in_tons_set266_2_HCT.html

| | Import-Export | Year | Category | Item | Value |
|---|---|---|---|---|---|
| 1 | Import | 2015 | Dairy | Cream | 862 |
| 2 | Export | 2015 | Dairy | Cream | 305 |
| 3 | Import | 2016 | Dairy | Cream | 749 |
| 4 | Export | 2016 | Dairy | Cream | 606 |
| 5 | Import | 2015 | Dairy | Cheese | 242 |
| 6 | Export | 2015 | Dairy | Cheese | 92 |
| 7 | Import | 2016 | Dairy | Cheese | 307 |
| 8 | Export | 2016 | Dairy | Cheese | 789 |
| 9 | Import | 2015 | Beverage | Soda | 414 |
| 10 | Export | 2015 | Beverage | Soda | 828 |
| 11 | Import | 2016 | Beverage | Soda | 301 |
| 12 | Export | 2016 | Beverage | Soda | 417 |
| 13 | Import | 2015 | Beverage | Juice | 783 |
| 14 | Export | 2015 | Beverage | Juice | 489 |
| 15 | Import | 2016 | Beverage | Juice | 180 |
| 16 | Export | 2016 | Beverage | Juice | 91 |

Food_import-export_in_tons_set266_2_DB.html

b

Food import-export in tons

| | | 2015 | | 2016 | |
|---|---|---|---|---|---|
| | | Import | Export | Import | Export |
| Beverage | | 598.50 | 658.50 | 240.50 | 254.00 |
| | Juice | 783.00 | 489.00 | 180.00 | 91.00 |
| | Soda | 414.00 | 828.00 | 301.00 | 417.00 |
| Dairy | | 552.00 | 198.50 | 528.00 | 697.50 |
| | Cheese | 242.00 | 92.00 | 307.00 | 789.00 |
| | Cream | 862.00 | 305.00 | 749.00 | 606.00 |
| Average | | 575.25 | 428.50 | 384.25 | 475.75 |

c PARAM_semantics.json


```
{"data":
[{
  "code": "Food",
  "names": [ "Category", "Item"],
  "values": [{"Meat": ["Beef", "Chicken", "Pork","Camel","Duck","Lamb"]},
             {"Cereals": ["Corn", "Oats", "Rice", "Seeds", "Wheat", "Soybeans"]},
             {"Dairy": ["Milk", "Butter", "Cream", "Cheese", "Yoghurt"]},
             {"Beverage": ["Coffee", "Tea", "Soda","Juice"]}]
},
{
  "code": "ImportExport",
  "names": ["Import-Export"],
  "values": ["Import","Export"]
},
{
  "code": "Years",
  "names": ["Year"],
  "values": ["2015","2016","2017","2018","2019","2020","2021","2022"]
},
{
  "code": "Quarters",
  "names": ["Quarter"],
  "values": ["Q1","Q2","Q3","Q4"]
}
],

"values": {"realUnit": [0.0,1.0],
           "realSignedUnit": [-1.0,1.0],
           "int":[0,100],
           "intSigned":[-100,100]}
}
```


Fig. 33. Domain vocabulary for generating synthetic tables and questions. Examples of $T_{REL}$ (a) and a corresponding $T_{HCT}$ (b) related to the amount of food import-export in some place. (c) JSON format to describe Food Import-Export domain vocabulary used to generate these tables (see Section F) and the related questions (see Section G). Not all vocabulary is used in $T_{REL}$ and $T_{HCT}$.

of the initial table. `none` is the default; `row` mixes only row attributes; `cols` mixes only column attributes; `rowscols` mixes rows and columns, keeping them as rows or columns, respectively; `all` mixes across rows and columns (some columns can become rows and vice-versa, except for hierarchical attributes which stay grouped in rows or columns). The shuffling process is a random permutation of the rows or columns to be mixed.

- `col_row_levels` is a list of authorized depths for nested rows and columns. It must be a list of strings "$dc_dr$" indicating the depth of the column header ($dc$), and of the row header ($dr$), with $dc$ and $dr$ in $\{1, 2, 3\}$.
- `col_row_name_pos` is a list of authorized displays for row and column attribute names. It must be a list of strings "C_R" indicating where the column (C) or row (R) attribute shall be displayed. R and C can be `none` to hide the attribute name, `left` to show it on the left side of the header, or `top` to show it at the top of the header. (*Place-holder for a future version, ignored by the generator*).
- `col_row_agg_pos` is a list of authorized positions for aggregate rows or columns. It must be a list of strings "C_R" indicating on which side of the header the aggregate column (C) or row (R) shall be displayed: R can be set to `top` or `bottom`, C can be set to `left` or `right`, and both can be set to `none` (Default) to hide the corresponding aggregate.
- `row_format` can be set to `new` to use one column for each level of row header, or `indent` to create an empty row for all level $L$ and indent all rows at level $L+1$. The header of the empty row is the attribute value for that row. An aggregate can fill this row if `col_row_agg_pos` is set to `*_top`.
- `tables` is a list of table content specifications (values, headers, and aggregates) with coherent domain vocabulary. Each initial table template in the list has the following keys:
  - `valueName` is the title displayed at the top of the $T_{HCT}$ table (e.g., "Food import-export in tons")
  - `values` is either a shortcut name referring to one of the keys (e.g., `realUnit`) stored in `values` in the domain vocabulary (see Figure 33), or directly an interval of the form $[m, M]$ following the description of the main key `values` in Section E
  - `valueUnit` (*Place-holder for a future version, not yet implemented*) is a character string to be added to give units to values in the core part of the table (e.g., % sign after each value in the core part of $T_{REL}$).
  - `rowCodes` is a list of one or more of the `data code` keys (e.g., `Food` or `Quarters`) listed in the domain

vocabulary. The generator will pick among these codes to generate the rows of $T_{HCT}$, and one part of the nominal attributes of $T_{REL}$.

- `rowSamples` is a list containing as many intervals $[m, M]$ as there are codes listed in `rowCodes`, in the same order. Each interval specifies the minimum $m$ and maximum $M$ number $n$ of values to select among the list $L$ of values of the specified attribute. The generator first picks $n$ at random in $[m, M]$, then selects $n$ consecutive values in $L$ starting at a random position in $L$ but ensuring there are always $n$ values in the result if the list $L$ is long enough, or return $L$ itself otherwise. For instance, let's consider an independent attribute `Years` with a list of values: $L = [2001, 2002, 2003, 2004, 2005, 2006]$, if we set `rowSamples` to $[1, 5]$, and the generator randomly picks $n = 3$, it then picks a random index $i$ between 1 and 4, e.g., $i = 2$, to generate the final subset $[2002, 2003, 2004]$ of years to use in the table. For hierarchical attributes, the same process applies recursively to all levels of the hierarchy, starting from the top level. If the interval is set to $[0]$ instead, all the values of the corresponding independent attribute are generated, or all the values at all hierarchy levels are generated for hierarchical attributes. Note that the `col_row_levels` parameter limits the depth of row and column headers, so the attributes finally displayed in $T_{HCT}$.
- `colCodes` is the same as `rowCodes` but for columns. The generator will pick among these codes to generate the columns of $T_{HCT}$, and the other part of the nominal attributes of $T_{REL}$.
- `colSamples` is the same as `rowSamples` but for columns.
- `agg_name1` is the name of the aggregate to display in the table.
- `agg_fun1` is the aggregate function to use among `sum` (default), `avg` (average), `min`, `max`.

These generic table templates `PARAM_tableTemplate.json` are used with the domain vocabulary `PARAM_semantics.json` to generate individual templates using the script `1_geneAllTablesJSONfromPatterns.R`. The individual templates generated and described below are stored in the file `PARAM_tablesToGenerate.json`.

**Customization of generic table templates** These generic table templates are customizable except for all the JSON keys and following the rules above.

An individual table template is a JSON object instantiating one combination of the parameters `shuffle`, `col_row_levels`, `col_row_name_pos`, `col_row_agg_pos`, and `row_format` common to all initial table templates, and set in the generic table template. It contains the following keys:

- `name` is the identifier of the family of $T_{HCT}$ and $T_{REL}$ tables specified by this individual table template.
- `replica` is identical to `replica` set in the generic table template.
- `agg_fun1` is identical to `agg_fun1` set in the generic table template.
- `agg_name1` is identical to `agg_name1` set in the generic table template.
- `values` is identical to `values` set in the generic table template.
- `valueName` is identical to `valueName` set in the generic table template.
- `shuffle` is one of the shuffling commands listed in `shuffle` of the generic table template.
- `row_format` is one of the row format commands listed in `row_format` of the generic table template.
- `columns` and `rows` are lists of attribute `codes` obtained after filtering the first $dr$ row attributes and $dc$ column attributes and setting the aggregate position using the `col_row_levels` and `agg_pos1` parameters from the parameter combination of this instance. `sample` are identical to `colSamples` and `rowSamples` of the corresponding attribute set in the generic table template.

Finally, `replica` instances of $T_{HCT}$ are generated using the script `2_geneHCTfromJSONtables.R` which takes as input the files `PARAM_semantics.json`, `PARAM_tableTemplate.json`, and `PARAM_tablesToGenerate.json`. It returns HTML files named `[name]_n_DB.html` ($T_{REL}$) and `[name]_n_HCT.html` ($T_{HCT}$) with $n$ the replica number.

This process draws samples from each individual template, randomizing all the remaining parameters: the `shuffle` (stochastic) command is applied to rows and columns attributes, then the values of each attribute are sampled using the `sample` parameter, and the values in the core of the table are sampled from the `values` parameter. The absence or presence of cell borders in the $T_{HCT}$ layout is randomized for each final sample.

Two examples of individual templates with two of the five sampled $T_{REL}$ and corresponding $T_{HCT}$ are shown in Figure 34.

**Customization of individual table templates** There is no possible customization of these individual table templates automatically generated from the generic table templates.

### *G. Natural Language question templates*

We defined 15 SQL templates in Table III and transcribes them into the HCT domain using 15 domain-specific Natural Language templates (Figures 36 and 37 show all the natural language question templates in JSON format related to the seven generated HCT domains).

For instance, the $Q_{NL}1$ template below transcribes the $SQL_1$ template (Table III) for the Food Import-Export domain HCT displayed in Figure 35c:

```
What is the amount of_ $Import_Export
of__ $Item__ $Category in__
$Quarter==of==$Year?
```

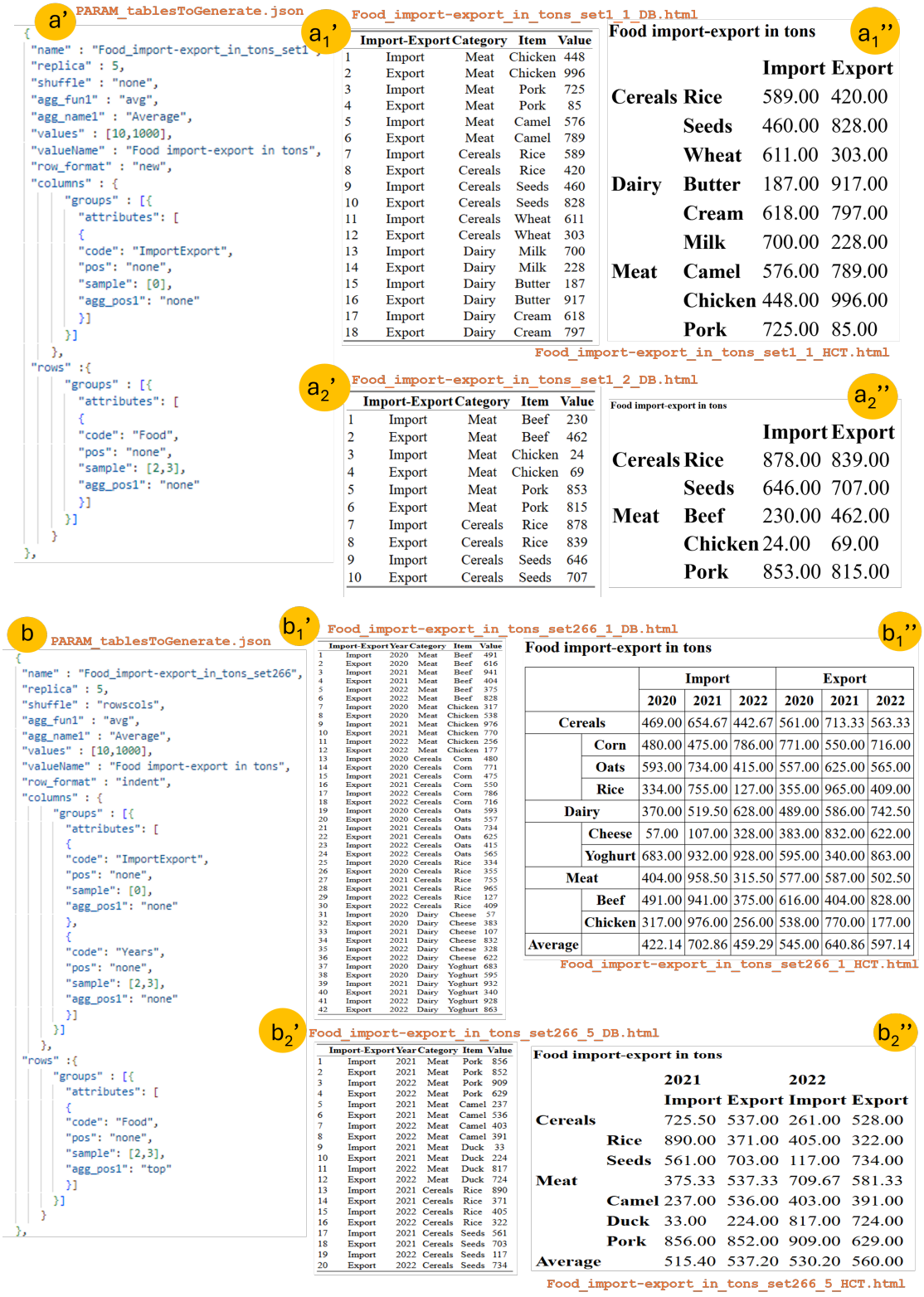



Fig. 34. Two examples of individual table templates (a,b), each with two random samples (1, 2) of $T_{REL}$ (a', b') and $T_{HCT}$ (a", b"). Templates a and b differ by the depth of column headers (`col_row_levels`), row indentation (`row_format`), presence and position of aggregates (`agg_pos1`). The sampled $T_{HCT}$ (1, 2) of a given template (a or b), further differs by the randomized order of the attributes (`shuffle`), number and names of the attribute values (`sample`), core numerical values (`values`), and presence of table cell borders (random).

| | A | B | C | D | E |
|---|---|---|---|---|---|
| 1 | Import-Export | Year | Category | Item | Value |
| 2 | Import_Export | Year | Category | Item | Value |
| 3 | Import | 2017 | Dairy | Milk | 799 |
| 4 | Export | 2017 | Dairy | Milk | 57 |
| 5 | Import | 2018 | Dairy | Milk | 879 |
| 6 | Export | 2018 | Dairy | Milk | 586 |
| 7 | Import | 2017 | Dairy | Butter | 444 |
| 8 | Export | 2017 | Dairy | Butter | 146 |
| 9 | Import | 2018 | Dairy | Butter | 202 |
| 10 | Export | 2018 | Dairy | Butter | 443 |
| 11 | Import | 2017 | Dairy | Cream | 736 |
| 12 | Export | 2017 | Dairy | Cream | 44 |
| 13 | Import | 2018 | Dairy | Cream | 179 |
| 14 | Export | 2018 | Dairy | Cream | 821 |
| 15 | Import | 2017 | Beverage | Coffee | 266 |
| 16 | Export | 2017 | Beverage | Coffee | 980 |
| 17 | Import | 2018 | Beverage | Coffee | 768 |
| 18 | Export | 2018 | Beverage | Coffee | 121 |
| 19 | Import | 2017 | Beverage | Tea | 751 |
| 20 | Export | 2017 | Beverage | Tea | 749 |
| 21 | Import | 2018 | Beverage | Tea | 673 |
| 22 | Export | 2018 | Beverage | Tea | 622 |

| | A | B | C | D | E | F | G | H | I |
|---|---|---|---|---|---|---|---|---|---|
| 1 | | HCT1 | Import | | | Export | | | Average |
| 2 | HCT2 | | 2017 | 2018 | Average | 2017 | 2018 | Average | |
| 3 | Beverage | | 508.5 | 720.5 | 614.5 | 864.5 | 371.5 | 618 | 616.25 |
| 4 | | Coffee | 266 | 768 | 517 | 980 | 121 | 550.5 | 533.75 |
| 5 | | Tea | 751 | 673 | 712 | 749 | 622 | 685.5 | 698.75 |
| 6 | Dairy | | 659.67 | 420 | 539.83 | 82.33 | 616.67 | 349.5 | 444.67 |
| 7 | | Butter | 444 | 202 | 323 | 146 | 443 | 294.5 | 308.75 |
| 8 | | Cream | 736 | 179 | 457.5 | 44 | 821 | 432.5 | 445 |
| 9 | | Milk | 799 | 879 | 839 | 57 | 586 | 321.5 | 580.25 |
| 10 | Average | | 599.2 | 540.2 | 569.7 | 395.2 | 518.6 | 456.9 | 513.3 |

HCT3

Fig. 35. Example of a food import-export domain $T_{REL}$ (left) and $T_{HCT}$(right) to explain $Q_{NL}$ templates. All the values (dashed color frames) of columns A, B, C, and D of the relational table are factorized as row and column headers in the HCTs (HCT1, HCT2). In contrast, the whole value column E (blue frame) is reproduced in the core area of the HCT (HCT3). In this instance, the ground-truth answer $A$ to the question `"What is the amount of Export of Milk in 2017?"` ($Q_{NL}$) related to $T_{HCT}$ is `"57"` (Dashed blue frame). This answer $A$ is obtained by running the SQL query $Q_{SQL}$ paired with $Q_{NL}$ onto $T_{REL}$.

This template is instanciated with detailed row and column attribute values of the HCT, to generate the following natural language question:

```
What is the amount of Export of
Milk in 2017?
```

Notice that not all the table attributes may be displayed in the HCT, and the values vary due to random selection of the HCT instance.

This question will be used as a prompt for the evaluated LLM along with the $T_{HCT}$ instance. The corresponding SQL query instance is:

```
SELECT Value FROM DBdata
WHERE ((Import_Export = 'Export' AND
Year = '2017'))
AND ((Category = 'Dairy' AND Item =
'Milk'));
```

and it will be run on the $T_{REL}$ instance paired with $T_{HCT}$ to get the ground-truth answer $A =$ "57".

Here are the 15 types of natural language questions $Q_{NL}$ generated for the table $T_{HCT}$ in Figure 35 instantiated from the template of Figure 36a (";" and ";" in answers, are column and row separators respectively):

- $\mathbf{Q_{NL}1}$: *one column, one row selection*: "What is the amount of Export of Milk in 2017?" → "57"
- $\mathbf{Q_{NL}2}$: *one column, many rows selection*: "What is the amount of Export of Butter, Coffee, Milk, or Tea in 2017?" → "57; 146 ; 980; 749"
- $\mathbf{Q_{NL}3}$: *many columns, one row selection*: "What is the amount of Export of Tea in 2017 or 2018?" → "749; 622"
- $\mathbf{Q_{NL}4}$: $Q_{NL}3$ + *expression not in the table*: "What are the total and minimum amounts of Export or Import of Milk in 2017 or of Export of Milk in 2018?" → "1442,57"
- $\mathbf{Q_{NL}5}$ (opt): $Q_{NL}4$ + *expression from the table*: (only if an aggregate exists in the *HCT*) "What is the average amount of Export or Import of Coffee?" → "533.75"
- $\mathbf{Q_{NL}6}$: *many rows, many columns selection*: "What is the amount of Export or Import of Butter, Cream, or Tea in 2017 or of Export of Butter, Cream, or Tea in 2018?" → "444; 146; 443; 736; 44; 821; 751; 749; 622"
- $\mathbf{Q_{NL}7}$: $Q_{NL}2$ + *aggregation not in the table*: "What are the total and minimum amounts of Import of Butter, Coffee, Cream, Milk, or Tea in 2017?" → "2996,266"
- $\mathbf{Q_{NL}8}$: $Q_{NL}6$ + *grouping per column* + *reporting*: "What are the minimum and maximum amounts for each Import-Export and Year, of Export or Import of Butter, Coffee, Cream, Milk, or Tea in 2017 or 2018? Please, report the corresponding Import-Export and Year." → "Export,2017,44,980; Export,2018,121,821; Import,2017,266,799; Import,2018,179,879"
- $\mathbf{Q_{NL}9}$: $Q_{NL}2$ + *grouping per row*: "What is the minimum amount for each Category, of Import of Beverage or Dairy in 2018?" → "673;179"
- $\mathbf{Q_{NL}10}$: $Q_{NL}6$ + *grouping per rows* + *reporting*: "What is the minimum amount for each Category, of Import of Beverage or Dairy in 2018? Please, report the corresponding Category." → "Beverage,673; Dairy,179"
- $\mathbf{Q_{NL}11}$: $Q_{NL}6$ + *grouping per rows and columns* + *reporting*: "What is the minimum amount for each Category, Import-Export, and Year, of Import of Beverage or Dairy in 2017 or of Export or Import of Beverage or Dairy in 2018? Please, report the corresponding Category, Import-Export, and Year." → "Beverage,Export,2018,121; Beverage,Import,2017,266; Beverage,Import,2018,673; Dairy,Export,2018,443; Dairy,Import,2017,444; Dairy,Import,2018,179"
- $\mathbf{Q_{NL}12}$: $Q_{NL}2$ + *top-K* + *row filter*: "What are the bottom 5 amounts of Export of Beverage or Dairy in

2018?" → "121; 443; 586; 622; 821"

- $\mathbf{Q_{NL}13}$: $Q_{NL}2$ + *row filter* + *ordering*: "What are the amounts ordered by increasing values of Import of Beverage or Dairy in 2017?" → "266; 444; 736; 751; 799"
- $\mathbf{Q_{NL}14}$: $Q_{NL}2$ + *operation on column*: "What are the Category and Item for which the amount of Import in 2018 is greater than 513.3?" → "Dairy,Milk; Beverage,Coffee; Beverage,Tea"
- $\mathbf{Q_{NL}15}$: $Q_{NL}14$ *with condition* + *reporting*: "What is the amount of Export in 2018 of Category and of Item for which the amount of Import in 2018 is greater than 513.3? Please, report the corresponding Category, Item, and amount of import-export." → "Dairy,Milk,586; Beverage,Coffee,121; Beverage,Tea,622"

Figures 36 and 37 show the natural language question templates related to the seven generated HCT domains, as JSON objects with the following keys:

- `tableName` is identical to the `valueName` key in the generic table template (see Section F).
- `rowColCodes` gather the `rowCodes` and `colCodes` lists of attribute codes used in rows and columns of the generic table template.
- `valueMeaning` is a text characterizing a value in the table continuing the sentence: "A value in this table represents the..." (e.g., "amount of import-export").
- `simplifyNested` lists all the `names` group of hierarchical attributes used in the generic table template.
- `template_report` is a sentence to report a specific list of attributes `$REPORTATTR` selected for the query result in templates $8, 10, 11, 15$.
- `template_N` a list of templates for questions of type N (See the list of 15 $Q_{NL}$ templates above).
  - A template is a natural language sentence with some place-holder variables to be filled by the generator depending on the table instance content and the type of question. All the templates must show all the possible attributes set in the generic table template, but the question generator will randomly pick the ones it will use, depending on the template type; the others will be absent.
  - We use `$attribute_name` to specify variables that will be automatically replaced by the generator with values from that attribute present in the table instance (e.g., `$Category` could be replaced by `Meat`, `Dairy`, `Cereals`, or `Beverage`). These `attribute_name` must appear in the `rowColCodes` list for independent attributes, and in the `simplifyNested` list for groups of hierarchical attributes.
  - All words in capital letters `$RESERVED` and their connected words should not be modified, and must appear as-is in their respective template.
  - In templates $4, 5, 7, 8, 9, 10, 11$, an expression `$EXPR` will be replaced by one of the aggregation names `agg_name1` (`min`, `max...`) in the generic table template. Some question can ask several expressions at once, like the min and the max, calling for a plural version of the sentence. We use / to separate the singular and plural versions of a word that the generator will pick automatically depending on the number of expression terms generated. For instance, the template 7: `What is/are_the_$EXPR_amount/amounts of_$Import_Export of...` can generate *"What is the minimum amount of Export of..."* or *"What are the minimum and maximum amounts of Export of..."*
  - In templates $8, 9, 10, 11$, `$GROUPBY` will be replaced by `for each` followed by a list of attribute names. For instance, the template 9: `What is/are_the_$EXPR_amount/amounts $GROUPBY of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?` can generate the question *"What is the minimum amount for each Category, of Import of Beverage or Dairy in 2018?"*
  - In template 12, `top/bottom_$TOPK` will be replaced by an integer and either top or bottom automatically selected, e.g., *"What are the top 5 amounts of Import of...*
  - In template 13, the group `$ORDERBYDESC_decreasing/increasing_values` must be used as-is and can generate the question *"What are the amounts ordered by increasing values of Import of Beverage or Dairy in 2017?"* or *"What are the amounts ordered by decreasing values of..."*
  - In template 14, the word `$OPPATTR` will be replaced by a list of attribute names, and the word `$OPANDVAL` by either `greater than` or `lower than` and a certain value, leading to a question like: *"What are the Category and Item for which the amount of Import in 2018 is greater than 513.3?"*
  - We use _ to link words like `of`, `in` to their attribute. If an attribute is not present in an instance of a question, all the words connected to it with _ will be absent too (e.g., `of_$Import_Export`).
  - We use __ to link several attributes together. For instance, we use __ in `of__$Item__$Category` so if `$Item` is absent, the `of` word stays connected to `$Category` like in `of Meat`. This pattern can generate `of Beef` and `of Meat`, but will not generate `of Beef Meat` because these attributes are part of the same hierarchical group.
  - We use == to link several attributes separated by a word that only exist if both surrounding attributes are present. For instance: we can say `in Q2 of 2022` or `in 2022` or `in Q2`. Thus, we tell that `of` is only present if both `$Quarter` and `$Year` are present together, by coding `in__$Quarter==of==$Year`.
  - We use `((not_an_attribute))` to ensure the presence of a word which is not an

attribute. For instance, we want to enable the generation of `the number of English male students` but also `the number of students` if the `$Nationality` and `$Gender` attributes are absent from the generated question. As `students` is not an attribute value, we use the template `of_((students))of__$Nationality__$Gender__students` to enable this option. The generator will only produce the first part `of_((students))` if the word `students` does not appear in the rest of the generated sentence (See Figure 36b).

- The order of the attributes used in the template is always preserved.
- Additional rules, a template shall never start with a `$` and a space must be set before the final question mark. Attribute `names` composed of several words separated by - should replace it by _.

All 14 templates are repeated with variations for the 7 domains in the `PARAM_NLquestionTemplates.json` file, guiding the creation of the same templates for new domains. **Note that the question $Q_{NL}15$ is automatically generated from templates 13 and 14.**

The script `3_geneSQLandNLQfromJSONtemplates.R` reads the question template file `PARAM_NLquestionTemplates.json` and the other parameter files, to generate the natural language questions $Q_{NL}$ and corresponding SQL queries $Q_{SQL}$ for each question template as well as their ground truth answers $A$ from the tables generated from `PARAM_tablesToGenerate.json`.

**Customization of question templates** These question templates are customizable except for all the JSON keys and following the rules above.

```
{
    "tableName": "Number of accidents",
    "rowColCodes": ["Accidents","Genders","InjurySeverity","Years","Quarters"],
    "valueMeaning": ["number of accidents"],
    "simplifyNested": [],
    "template_report": ["Please,_report_the_corresponding_$REPORTATTR"],
    "template_1": ["What is the number of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_2": ["What is the number of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_3": ["What is the number of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_4": ["What is/are_the_$EXPR_number/numbers of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_5": ["What is/are_the_$EXPR_number/numbers of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_6": ["What is the number of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_7": ["What is/are_the_$EXPR_number/numbers of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_8": ["What is/are_the_$EXPR_number/numbers of_((accidents)) $GROUPBY of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_9": ["What is/are_the_$EXPR_number/numbers of_((accidents)) $GROUPBY of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_10": ["What is/are_the_$EXPR_number/numbers of_((accidents)) $GROUPBY of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_11": ["What is/are_the_$EXPR_number/numbers of_((accidents)) $GROUPBY of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_12": ["What are the largest/smallest_$TOPK numbers of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_13": ["What are the numbers $ORDERBYDESC_decreasing/increasing_values of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year ?"],
    "template_14": ["What is/are_the_$OPATTR_for_which the number of_((accidents)) of__$Severity__$Event__accidents involving_$Gender_casualties in__$Quarter==of==$Year is_$OPANDVAL ?"]
},
```


(a)


```
{
    "tableName": "Number of constructions",
    "rowColCodes": ["QuarterMonths","USA","Buildings"],
    "valueMeaning": ["number of buildings"],
    "simplifyNested": [["Type of building","Usage"], ["Quarter","Month"]],
    "template_report": ["Please,_report_the_corresponding_$REPORTATTR"],
    "template_1": ["What is the number of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_2": ["What is the number of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_3": ["What is the number of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_4": ["What is/are_the_$EXPR_number/numbers of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_5": ["What is/are_the_$EXPR_number/numbers of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_6": ["What is the number of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_7": ["What is/are_the_$EXPR_number/numbers of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_8": ["What is/are_the_$EXPR_number/numbers of_((buildings)) $GROUPBY of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_9": ["What is/are_the_$EXPR_number/numbers of_((buildings)) $GROUPBY of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_10": ["What is/are_the_$EXPR_number/numbers of_((buildings)) $GROUPBY of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_11": ["What is/are_the_$EXPR_number/numbers of_((buildings)) $GROUPBY of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_12": ["What are the top/bottom_$TOPK numbers of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_13": ["What are the numbers $ORDERBYDESC_descending/ascending_values of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter ?"],
    "template_14": ["What is/are_the_$OPATTR_for_which the number of__$Type_of_building__$Usage__buildings in__$City==,==$State in__$Month==,==$Quarter is_$OPANDVAL ?"]
}
```


(b)


```
{
    "tableName": "Food import-export in tons",
    "rowColCodes": ["Food","ImportExport","Years","Quarters"],
    "valueMeaning": ["amount of import-export"],
    "simplifyNested": [["Category","Item"]],
    "template_report": ["Please,_report_the_corresponding_$REPORTATTR"],
    "template_1": ["What is the amount of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_2": ["What is the amount of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_3": ["What is the amount of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_4": ["What is/are_the_$EXPR_amount/amounts of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_5": ["What is/are_the_$EXPR_amount/amounts of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_6": ["What is the amount of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_7": ["What is/are_the_$EXPR_amount/amounts of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_8": ["What is/are_the_$EXPR_amount/amounts $GROUPBY of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_9": ["What is/are_the_$EXPR_amount/amounts $GROUPBY of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_10": ["What is/are_the_$EXPR_amount/amounts $GROUPBY of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_11": ["What is/are_the_$EXPR_amount/amounts $GROUPBY of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_12": ["What are the top/bottom_$TOPK amounts of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_13": ["What are the amounts $ORDERBYDESC_decreasing/increasing_values of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year ?"],
    "template_14": ["What is/are_the_$OPATTR_for_which the amount of_$Import_Export of__$Item__$Category in__$Quarter==of==$Year is_$OPANDVAL ?"]
},
```


(c)


```
{
    "tableName": "Number of graduations",
    "rowColCodes": ["AcademicYears", "Languages", "UniversityLevels", "Genders", "LocalNationality"],
    "valueMeaning": ["number of graduations"],
    "simplifyNested": [],
    "template_report": ["Please,_report_the_corresponding_$REPORTATTR"],
    "template_1": ["What is the number of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_2": ["What is the number of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_3": ["What is the number of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_4": ["What is/are_the_$EXPR_number/numbers of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_5": ["What is/are_the_$EXPR_number/numbers of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_6": ["What is the number of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_7": ["What is/are_the_$EXPR_number/numbers of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_8": ["What is/are_the_$EXPR_number/numbers of graduations $GROUPBY of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_9": ["What is/are_the_$EXPR_number/numbers of graduations $GROUPBY of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_10": ["What is/are_the_$EXPR_number/numbers of graduations $GROUPBY of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_11": ["What is/are_the_$EXPR_number/numbers of graduations $GROUPBY of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_12": ["What are the top/bottom_$TOPK numbers of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_13": ["What are the numbers of graduations $ORDERBYDESC_decreasing/increasing_values of_$Gender_students with_a_$Degree in_$Language in_$Year ?"],
    "template_14": ["What is/are_the_$OPATTR_for_which the number of graduations of_$Gender_students with_a_$Degree in_$Language in_$Year is_$OPANDVAL ?" ]
},
```


(d)

Fig. 36. Synthetic natural question templates for the Accidents (a), Constructions (b), Food (c), and Graduations (d) domains.

```
{
    "tableName": "Evolution of pollution in percent",
    "rowColCodes": ["Pollutions","USA","Years"],
    "valueMeaning": ["evolution of pollution"],
    "simplifyNested": [["Polluted element","Type of pollution"]],
    "template_report": ["Please,_report_the_corresponding_$REPORTATTR"],
    "template_1": ["What is the evolution of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_2": ["What is the evolution of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_3": ["What is the evolution of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_4": ["What is/are_the_$EXPR_evolution/evolutions of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_5": ["What is/are_the_$EXPR_evolution/evolutions of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_6": ["What is the evolution of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_7": ["What is/are_the_$EXPR_evolution/evolutions of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_8": ["What is/are_the_$EXPR_evolution/evolutions of_((pollution)) $GROUPBY of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_9": ["What is/are_the_$EXPR_evolution/evolutions of_((pollution)) $GROUPBY of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_10": ["What is/are_the_$EXPR_evolution/evolutions of_((pollution)) $GROUPBY of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_11": ["What is/are_the_$EXPR_evolution/evolutions of_((pollution)) $GROUPBY of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_12": ["What are the top/bottom_$TOPK evolutions of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_13": ["What are the evolutions $ORDERBYDESC_decreasing/increasing_values of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year ?"],
    "template_14": ["What is/are_the_$OPATTR_for_which the evolution of_((pollution)) of_$Type_of_pollution_pollution of_$Polluted_element in__$City==,==$State in_$Year is_$OPANDVAL ?"]
},
```


(a)


```
{
    "tableName": "Number of students",
    "rowColCodes": ["SchoolGovPriv","Genders","LocalNationality","AcademicYears","Quarters"],
    "valueMeaning": ["number of students"],
    "simplifyNested": [],
    "template_report": ["Please,_report_the_corresponding_$REPORTATTR"],
    "template_1": ["What is the number of_((students)) of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_2": ["What is the number of_((students)) of__$Nationality__$Gender__students in_$School_type_schools  in__$Quarter==of==$Year ?"],
    "template_3": ["What is the number of_((students)) of__$Nationality__$Gender__students in_$School_type_schools  in__$Quarter==of==$Year ?"],
    "template_4": ["What is/are_the_$EXPR_number/numbers of_((students)) of__$Nationality__$Gender__students in_$School_type_schools  in__$Quarter==of==$Year ?"],
    "template_5": ["What is/are_the_$EXPR_number/numbers of_((students)) of__$Nationality__$Gender__students in_$School_type_schools  in__$Quarter==of==$Year ?"],
    "template_6": ["What is the number of_((students)) of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_7": ["What is/are_the_$EXPR_number/numbers of_((students)) of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_8": ["What is/are_the_$EXPR_number/numbers of_((students)) $GROUPBY of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_9": ["What is/are_the_$EXPR_number/numbers of_((students)) $GROUPBY of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_10": ["What is/are_the_$EXPR_number/numbers of_((students)) $GROUPBY of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_11": ["What is/are_the_$EXPR_number/numbers of_((students)) $GROUPBY of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_12": ["What are the top/bottom_$TOPK numbers of_((students)) of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_13": ["What are the numbers $ORDERBYDESC_decreasing/increasing_values of_((students)) of__$Nationality__$Gender__students  in_$School_type_schools in__$Quarter==of==$Year ?"],
    "template_14": ["What is/are_the_$OPATTR_for_which the number of_((students)) of__$Nationality__$Gender__students in_$School_type_schools in__$Quarter==of==$Year is_$OPANDVAL ?"]
},
```


(b)


```
{
    "tableName": "Weather statistics",
    "rowColCodes": ["Weather","USA","Years"],
    "valueMeaning": ["number of days"],
    "simplifyNested": [],
    "template_report": ["Please,_report_the_corresponding_$REPORTATTR"],
    "template_1": ["How many days with__$Intensity__$Characteristic were there in__$City==,==$State in_$Year ?"],
    "template_2": ["How many days with__$Intensity__$Characteristic were there in__$City==,==$State in_$Year ?"],
    "template_3": ["How many days with__$Intensity__$Characteristic were there in__$City==,==$State in_$Year ?"],
    "template_4": ["What is/are_the_$EXPR_number/numbers_of_days with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_5": ["What is/are_the_$EXPR_number/numbers_of_days with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_6": ["How many days with__$Intensity__$Characteristic were there in__$City==,==$State in_$Year ?"],
    "template_7": ["What is/are_the_$EXPR_number/numbers_of_days with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_8": ["What is/are_the_$EXPR_number/numbers_of_days $GROUPBY with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_9": ["What is/are_the_$EXPR_number/numbers_of_days $GROUPBY with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_10": ["What is/are_the_$EXPR_number/numbers_of_days $GROUPBY with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_11": ["What is/are_the_$EXPR_number/numbers_of_days $GROUPBY with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_12": ["What are the largest/smallest_$TOPK numbers_of_days with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_13": ["What are the numbers of days $ORDERBYDESC_descending/ascending_values with__$Intensity__$Characteristic in__$City==,==$State in_$Year ?"],
    "template_14": ["What is/are_the_$OPATTR_for_which the number of days with__$Intensity__$Characteristic in__$City==,==$State in_$Year is_$OPANDVAL ?"]
},
```


(c)

Fig. 37. Synthetic natural question templates for the Pollution (a), Students (b), and Weather (c) domains.

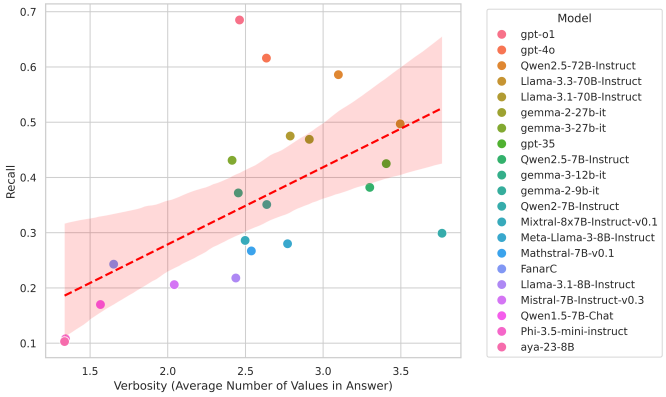



Fig. 38. Recall Score vs. Verbosity for Text-Only Models on Real World HCTs.

### *H. Additional Trend Analysis & Observations*

**(RQ8) How does model verbosity affect the output correctness?** Our initial hypothesis was that if models simply take the kitchen-sink approach and output more values in their answers then they would have a higher probability of matching values in the ground truth and thereby inflate their recall scores. As shown in Figure 38, there is no clear correlation between answer length and recall. Most models, including the best performing ones, seem to be clustered around the middle on the verbosity scale. The average number of values in the *ground-truth* answers for all queries is 4, yet most models tend to provide between 2-3 values in their answers. This suggests that models may in fact be conservative in their answer verbosity as which is antithetical to the overly verbose nature of these generative models.

**(RQ9) How do model perform on different synthetic QA templates?** The synthetic queries we generated have 15 types including various types of selection, projection, expression, and aggregation. The model performance per template type is shown in Figure 39.

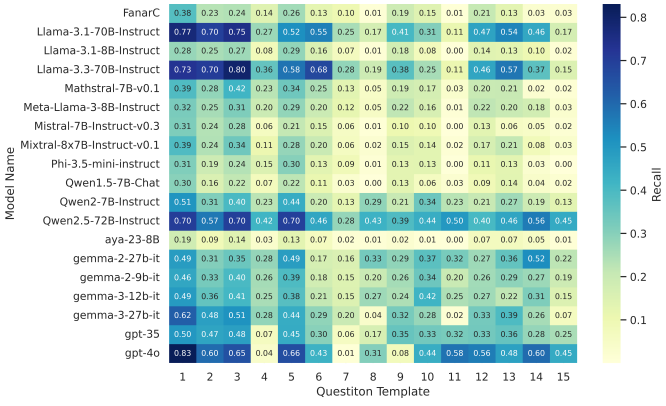



Fig. 39. Recall Score for Text-Only Models on all Synthetic HCTs per Question Template.

As expected, models tend to do best on selection queries be it single-cell or multi-cell selection. This is clear as templates 1, 2, and 3 have the highest average recall across all models. Models tend to do better on queries involving expressions on columns (templates 5, 14) than queries involving aggregations (template 7). In fact, models struggle the most with template 7 which requires doing an actual aggregation (rather than retrieving an existing one from the HCT). Models like LLaMA-3.3-70B significantly outperform even models like GPT-4o and GPT-35 on this template. It is also evident that models clearly have their strengths and weaknesses in terms of templates. GPT-4o gets 58% on template 11, whereas other models like LLaMA and Mistral do not even cross 10%. However, this dominance completely disappears in Templates 4, 7, and 9 where the GPT-4o fails to score more than 8% (compared to other models that get up to 42%). There are also unexpected trends across model generations. Gemma-2-27B scores a 52% recall on template 14 which is only topped by GPT-4o and Qwen2.5 72B, both of which are relatively larger models. However, on the exact same template (14), Gemma-3-27B scores exactly half of its predecessor with a recall score of 26%. This shows that newer generations of models are not necessarily improving their capabilities on this task across the board.

**(RQ10) In what cases do VLMs perform better than LLMs?** Recent advances in the image understanding and reasoning capabilities of VLMs make them a suitable candidate for the task at hand. VLMs also allow to skip the extraction and conversion to CSV process as they can directly consume the images of the tables from their source documents. On average, VLMs score 25.5% recall whereas text-only LLMs score 25% as shown in (Table IV). GPT4o when prompted using the text-modality achieves an average recall of 61.6%, compared to 62.3% when using the image-modality. This illustrates the progress vision models have made in terms of document understanding and reasoning tasks such as this one. We have established that HCTs inherently have visual characteristics that are crucial to understand the content in the table. A lot of those visual clues are lost when the HCT is converted to text format (regardless of which text format its represented in). VLMs are able to overcome this information loss by taking advantage of those visual cues. This is further shown by the top performing small sized VLM model, Pixtral-12B with a recall score of 45.9% which is nearly 8% better than the best performing text-only LLM of that size range (Qwen2.5-7B with recall 38%). Furthermore, Microsoft's Phi-3-Vision-128k (image-modality), which is only a 4.2B parameter model outperforms every text-only LLM in the small size category (many of which are 2x in size). These results clearly highlight the potential of VLMs in visual-reasoning tasks such as HCT-QA. Another interesting observation is that VLMs struggle considerably more than LLMs with larger *HCTs*. GPT4o performs almost identically in both the text and vision modalities except in the US CENSUS dataset which on average has 437 cells per table (nearly 3x more compared to the other datasets. This trend can also be seen in the smaller VLMs as most top performing models drop in performance when dealing with US CENSUS HCTs.

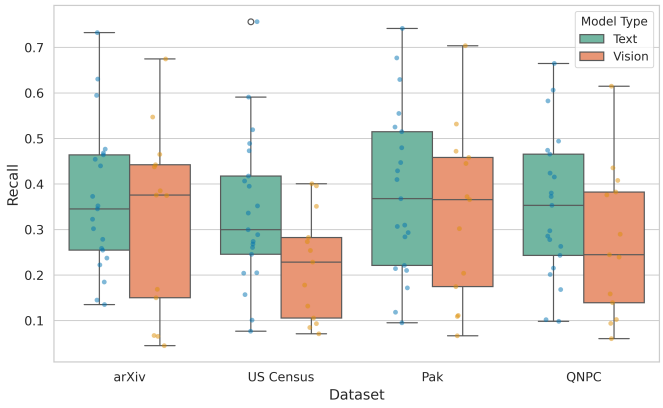



Fig. 40. LLMs vs. VLMs on all Real World HCTs — Recall Score.

### I. Prompting Details & Experiments

For certain models (like Gemma-2) that do not accept a system prompt we add the system prompt as a prefix to the user prompt and send it as one prompt with the ”user” role.

*1) Prompts:* Prompts are used with LLMs to give them context and role, and to format the results to allow automatic evaluation.

**System Prompt for Inference**

You are a table question answering assistant. You are given a table and a question. Your task is to answer the question based on the information in the table. The table structure may be complex and not a standard relational table, so try to understand the structure of the table when answering the question.
If the question cannot be answered using information from the table, return ‘No Answer‘. Do not provide any explanations, equations, code, or text explaining intermediate steps in figuring out the answer. Return only the final answer itself.
If there are multiple values, separate each value with the ‘ | | ‘ token.

**User Prompt for Inference**

Answer the question given the input table. Do not provide any explanations, equations, code, or text explaining intermediate steps in figuring out the answer. Return only the final answer itself.

# INPUTS:
Table: ‘{table}‘
Question: ‘{question}‘

# OUTPUT:
Answer:

*2) 0-shot vs. 1-shot prompting:* The majority of the models we used in our experiments can handle context lengths of 8,000+ tokens. This means that it is possible to do 1-shot prompting with a full table-question-answer triplet as the shot. This approach could help leverage in-context learning to improve the model performance. We experimented on a small set of 1,000 queries using Real World HCTs from all four data sources. Our 1-shot prompt consisted of a fixed example for all the questions in the test set. We avoided testing the models given an example question on the same HCT the test question was on as that reflects a very unrealistic scenario. The results are shown in Table VIII. We can see that adding the 1-shot example does not significantly affect the recall score as for gemma-2-9b-it it only increases by 1.3% and for Llama-3.1-8B-Instruct it actually decreases by less than 1%. Upon manual inspection, we found that the models were able to follow the output format stated in the prompt (values split by ‘ | | ‘ and no explanations) in the 0-shot scenario almost perfectly. As the 1-shot example was on a different table, it provided no ”learning” for the model to answer the test question it was being asked. Due to these results, our remaining experiments use 0-shot prompting.

TABLE VIII
0-SHOT VS 1-SHOT PROMPTING - RECALL SCORE

| Model | 0-Shot Recall | 1-Shot Recall |
|---|---|---|
| gemma-2-9b-it | 0.367 | 0.373 |
| Llama-3.1-8B-Instruct | 0.247 | 0.239 |

### J. Evaluation Metrics

We experimented using *Meta-Llama-3-70B* as judge model to gauge the correctness of a model’s answer. The prompt given to the judge model outlined a clear set of instructions: (1) minor spelling mistakes or typos in textual values can be ignored, (2) rounding numerical values is fine if done reasonably, (3) question must be answered fully in order to be ”correct”. We compared this model’s ”correct” labels to our CC Score (Complete Containment, which we defined as 1 when Recall Score == 1, else 0) on a subset of 500 real-world HCT QA pairs. The comparison is shown in Figure 41. It is evident that the model’s judgment and the CC score closely align with each other. Since running an LLM-as-a-Judge is expensive in terms of computational resources, we opt to use Recall, Precision, and CC Score as our evaluation metrics.

### K. VLMs vs LLMs

We can see from the results in Table IV that all VLMs are sensitive to table size. HCTs arXiv and Pak have relatively lower number of cells and number of words on average and vision models tend to do the best on these tables. It is evident that all vision models struggle with US Census tables (which are the largest tables on average with good margin). Gpt-4o scores 39.6% recall on US Census which is less than majority of medium sized text-only models which are considerably smaller in size.

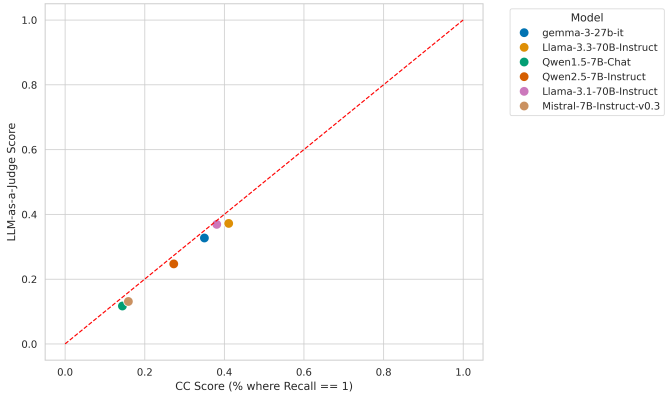



Fig. 41. CC Score vs. LLM-as-a-Judge Score on Real World HCTs.

| Age | Educational Level & Gender: الثانوية التخصصية Secondary Specialized Females | الثانوية التخصصية Secondary Specialized Males | الثانوية Secondary Females | الثانوية Secondary Males | الإعدادية التخصصية Preparatory Specialized Females | الإعدادية التخصصية Preparatory Specialized Males | الإعدادية Preparatory Females | الإعدادية Preparatory Males | الابتدائية Primary Females | الابتدائية Primary Males | ماقبل الابتدائية[1] Pre-primary (0) Females | ماقبل الابتدائية[1] Pre-primary (0) Males |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| <3 | - | - | - | - | - | - | - | - | 0 | 0 | 1,118 | 1,274 |
| 3 | - | - | - | - | - | - | - | - | 0 | 0 | 6,044 | 6,394 |
| 4 | - | - | - | - | - | - | - | - | 44 | 53 | 11,946 | 12,701 |
| 5 | - | - | - | - | - | - | - | - | 4,900 | 4,964 | 7,711 | 8,245 |
| 6 | - | - | - | - | - | - | - | - | 13,592 | 14,020 | 108 | 136 |
| 7 | - | - | - | - | - | - | - | - | 13,759 | 13,938 | 2 | 4 |
| 8 | - | - | - | - | - | - | - | - | 13,188 | 13,624 | - | - |
| 9 | - | - | - | - | - | - | - | - | 12,710 | 13,264 | - | - |
| 10 | - | - | - | - | 0 | 0 | 54 | 36 | 12,026 | 12,595 | - | - |
| 11 | - | - | - | - | 0 | 0 | 3,460 | 3,525 | 7,997 | 8,352 | - | - |
| 12 | - | - | - | - | 0 | 1 | 9,522 | 9,644 | 880 | 1,321 | - | - |
| 13 | - | - | - | - | 5 | 30 | 9,676 | 9,639 | 124 | 311 | - | - |
| 14 | 15 | 129 | 2,822 | 2,606 | 2 | 41 | 6,448 | 6,892 | 28 | 121 | - | - |
| 15 | 39 | 362 | 7,621 | 7,155 | 0 | 1 | 1,019 | 1,557 | 12 | 65 | - | - |
| 16 | 38 | 249 | 7,725 | 7,633 | 0 | 0 | 294 | 523 | 0 | 0 | - | - |
| 17 | 21 | 156 | 5,427 | 5,648 | 0 | 0 | 91 | 188 | 0 | 0 | - | - |
| 18 | 0 | 55 | 1,304 | 1,842 | 0 | 0 | 55 | 75 | 0 | 0 | - | - |
| 19 | 1 | 15 | 424 | 585 | 0 | 0 | 0 | 0 | 0 | 0 | - | - |
| 20 | 0 | 4 | 160 | 162 | 0 | 0 | 0 | 0 | 0 | 0 | - | - |
| 21 | 0 | 1 | 41 | 43 | 0 | 0 | 0 | 0 | 0 | 0 | - | - |
| 22 | 0 | 0 | 14 | 12 | 0 | 0 | 0 | 0 | 0 | 0 | - | - |
| 22< | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | - | - |
| Total | 114 | 971 | 25,538 | 25,686 | 7 | 73 | 30,619 | 32,079 | 79,260 | 82,628 | 26,929 | 28,754 |

**Question**

What is the sum of the total number of Non-Qatari females across all education levels?

**GPT4o-Text Answer**
30,619

**GPT4o-vision Answer**
114

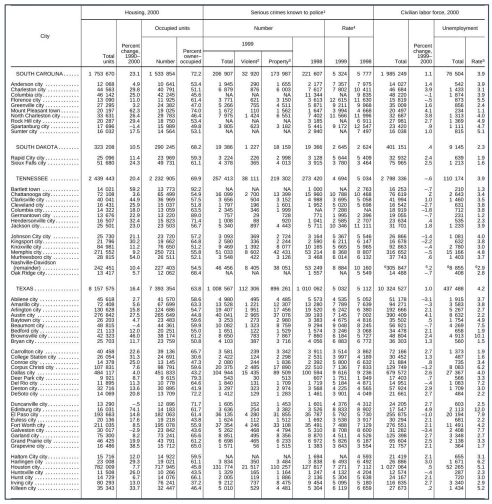
Fig. 42. **gpt-4o-vision** is correct but **gpt-4o** text-only is wrong.

This is further clear by the fact the gpt-4o-vision which outperforms its text-only version (gpt-4o) in all 3 datasets (QNPC, Pak, arXiv) but only scores a recall of 39.6% for US Census which is 20% less than that of gpt-4o (text-only) at 59%. Some examples of cases where gpt-4o gets answers a question correctly in one modality but gets it wrong in the other are provided in Figure 42 and Figure 43.

## *L. Finetuning Details*

We fine-tuned 2 models (Meta-Llama-3-8B-Instruct and FanarC-9B) using LLaMA Factory[2] using the following hyperparameters:

[2]https://github.com/hiyouga/LLaMA-Factory



| City | Housing, 2000: Total units | Housing, 2000: Percent change, 1990–2000 | Occupied units: Number | Occupied units: Percent owner-occupied | Serious crimes known to police: Number, 1999: Total | Number, 1999: Violent[2] | Number, 1999: Property[3] | Rate[4]: 1998 | Rate[4]: 1999 | Rate[4]: 1998 | Civilian labor force, 2000: Total | Civilian labor force, 2000: Percent change, 1990–2000 | Unemployment: Total | Unemployment: Rate[5] |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SOUTH CAROLINA | 1 753 670 | 23.1 | 1 533 854 | 72.2 | 206 907 | 32 920 | 173 987 | 221 607 | 5 324 | 5 777 | 1 985 249 | 1.1 | 76 504 | 3.9 |
| Anderson city | 12 068 | 4.9 | 10 641 | 53.4 | 1 945 | 290 | 1 655 | 2 177 | 7 307 | 7 975 | 14 027 | 1.4 | 542 | 3.9 |
| Charleston city | 44 540 | 29.0 | 40 791 | 51.1 | 6 679 | 879 | 5 800 | 7 617 | 7 902 | 10 411 | 46 664 | 3.9 | 1 630 | 3.5 |
| Columbia city | 46 142 | 25.0 | 41 245 | 45.6 | NA | NA | NA | 11 344 | NA | 13 835 | 48 220 | –1 | 1 874 | 3.9 |
| Florence city | 17 290 | 10.2 | 15 240 | 47.9 | 3 266 | 705 | 4 511 | 6 873 | 9 211 | 9 848 | 20 008 | 1.6 | 652 | 3.4 |
| Greenville city | 27 199 | 2.2 | 24 362 | 47.0 | 5 266 | 705 | 4 511 | 6 873 | 9 211 | 9 848 | 35 008 | 1.6 | 652 | 2.4 |
| Mount Pleasant town | 20 197 | 62.3 | 18 025 | 74.0 | 1 672 | 110 | 1 562 | 1 947 | 3 994 | 4 668 | 30 497 | 4.1 | 734 | 1.1 |
| North Charleston city | 50 287 | 21.3 | 44 750 | 43.2 | 9 805 | 1 305 | 8 500 | 11 385 | 13 985 | 15 966 | 70 111 | 3.1 | 1 999 | 2.5 |
| Rock Hill city | 20 287 | 25.4 | 18 793 | 53.4 | NA | NA | NA | 3 185 | NA | 6 911 | 27 961 | 2.7 | 1 365 | 4.9 |
| Spartanburg city | 17 686 | –1.4 | 15 969 | 49.9 | 3 805 | 623 | 3 182 | 5 441 | 9 172 | 10 547 | 23 458 | 0 | 1 111 | 4.7 |
| Sumter city | 16 032 | 17.5 | 14 564 | 53.1 | NA | NA | NA | 2 940 | NA | 7 497 | 16 038 | 1.0 | 615 | 3.1 |
| SOUTH DAKOTA | 323 208 | 10.5 | 290 245 | 68.2 | 19 366 | 1 227 | 18 159 | 19 366 | 2 645 | 2 624 | 401 151 | .4 | 9 145 | 2.3 |
| Rapid City city | 25 096 | 11.4 | 23 969 | 59.3 | 3 274 | 226 | 2 996 | 3 128 | 5 644 | 5 408 | 32 932 | 2.4 | 628 | 1.9 |
| Sioux Falls city | 51 680 | 24.5 | 48 731 | 61.1 | 4 370 | 365 | 4 015 | 3 933 | 3 590 | 3 464 | 75 965 | 2.5 | 1 213 | 1.6 |
| TENNESSEE | 2 439 443 | 20.4 | 2 232 905 | 69.9 | 257 413 | 38 111 | 219 302 | 273 420 | 4 694 | 5 034 | 2 798 326 | –.6 | 110 174 | 3.9 |
| Bartlett town | 14 021 | 59.2 | 13 772 | 92.2 | NA | NA | NA | 1 000 | NA | 2 783 | 16 253 | –.7 | 210 | 1.3 |
| Chattanooga city | 72 108 | 1.6 | 65 499 | 54.9 | 16 099 | 2 700 | 13 399 | 15 960 | 10 788 | 10 468 | 76 619 | .6 | 2 643 | 3.4 |
| Clarksville city | 37 358 | 44.3 | 34 469 | 54.2 | 4 766 | 1 010 | 3 756 | 4 766 | 4 010 | 4 010 | 76 619 | 2.1 | 5 267 | 6.9 |
| Cleveland city | 16 431 | 25.9 | 15 037 | 51.8 | 1 797 | 196 | 1 601 | 1 952 | 5 020 | 5 698 | 16 542 | –2.7 | 651 | 3.9 |
| Columbia city | 14 322 | 18.0 | 13 059 | 63.5 | 2 345 | 345 | 1 999 | NA | 7 288 | NA | 18 919 | –1.8 | 712 | 3.8 |
| Germantown city | 13 676 | 22.9 | 12 220 | 89.0 | 757 | 29 | 728 | 771 | 1 966 | 2 286 | 19 066 | –.7 | 231 | 1.2 |
| Hendersonville city | 16 507 | 32.4 | 15 923 | 73.4 | 1 006 | 89 | 920 | 1 041 | 3 965 | 2 707 | 33 934 | .4 | 535 | 2.3 |
| Jackson city | 25 561 | 2.3 | 23 955 | 56.7 | 3 342 | 897 | 4 440 | 5 731 | 10 349 | 11 111 | 31 781 | 1.4 | 1 205 | 3.9 |
| Johnson City city | 25 730 | 11.1 | 23 720 | 57.2 | 3 060 | 369 | 2 724 | 3 184 | 5 307 | 5 146 | 36 866 | –1.4 | 1 061 | 4.0 |
| Kingsport city | 27 787 | 10.3 | 25 731 | 57.3 | 3 060 | 369 | 2 724 | 3 184 | 5 307 | 5 146 | 36 866 | –1.4 | 1 061 | 4.0 |
| Knoxville city | 84 981 | 11.2 | 76 950 | 51.2 | 9 469 | 1 392 | 8 077 | 10 185 | 5 665 | 5 995 | 92 993 | –.4 | 2 782 | 3.0 |
| Memphis city | 271 552 | 8.2 | 253 721 | 55.8 | 51 033 | 8 602 | 42 431 | 53 214 | 8 369 | 8 882 | 316 652 | –.5 | 15 166 | 4.8 |
| Murfreesboro city | 28 833 | 54.0 | 26 511 | 52.1 | 3 549 | 422 | 3 126 | 3 468 | 6 014 | 6 339 | 37 743 | .6 | 1 403 | 3.7 |
| Nashville-Davidson (balance) | 242 451 | 10.4 | 227 463 | 54.5 | 46 456 | 8 405 | 38 051 | 53 249 | 8 884 | 10 160 | 7305 847 | *.0 | *8 866 | *2.9 |
| Oak Ridge city | 13 427 | 5.7 | 12 092 | 68.4 | NA | NA | NA | 1 937 | NA | 5 549 | 14 488 | –.7 | 408 | 2.9 |
| TEXAS | 8 157 575 | 18.4 | 7 393 354 | 63.8 | 1 008 567 | 112 309 | 896 261 | 1 010 062 | 5 092 | 5 112 | 10 324 527 | 1.0 | 437 486 | 4.2 |
| Abilene city | 45 618 | 2.7 | 41 570 | 58.6 | 4 989 | 495 | 4 495 | 5 573 | 4 535 | 5 052 | 61 178 | –3.1 | 1 915 | 3.7 |
| Amarillo city | 72 488 | 5.6 | 67 699 | 63.5 | 11 532 | 1 331 | 10 201 | 13 268 | 7 559 | 7 638 | 94 273 | –.1 | 3 665 | 3.9 |
| Arlington city | 130 428 | 29.8 | 124 666 | 54.7 | 19 407 | 1 951 | 17 456 | 19 120 | 6 240 | 6 580 | 192 666 | 2.1 | 5 267 | 2.7 |
| Austin city | 276 842 | 27.5 | 255 640 | 44.8 | 40 041 | 3 965 | 37 076 | 39 193 | 7 145 | 7 002 | 390 456 | 4.1 | 8 632 | 2.2 |
| Baytown city | 26 203 | 4.7 | 23 463 | 59.6 | 3 253 | 373 | 3 880 | 3 383 | 4 675 | 4 816 | 35 502 | 0 | 1 754 | 4.9 |
| Beaumont city | 51 115 | .2 | 45 251 | 55.0 | 6 953 | 1 022 | 5 929 | 6 514 | 5 048 | 5 048 | 54 478 | –2.1 | 4 066 | 7.5 |
| Bedford city | 21 113 | 12.0 | 20 254 | 59.0 | 1 651 | 122 | 1 529 | 1 514 | 5 048 | 5 048 | 34 478 | 2.1 | 668 | 1.9 |
| Brownsville city | 42 323 | 48.0 | 38 174 | 61.2 | 9 650 | 783 | 7 867 | 7 860 | 6 184 | 5 727 | 48 804 | 2.4 | 4 915 | 10.1 |
| Bryan city | 25 703 | 13.7 | 23 799 | 50.8 | 4 103 | 397 | 3 716 | 4 056 | 6 863 | 6 773 | 36 303 | 1.3 | 560 | 1.5 |
| Carrollton city | 40 458 | 22.6 | 38 136 | 66.7 | 3 981 | 299 | 3 342 | 3 911 | 3 514 | 3 862 | 72 166 | 2.7 | 1 373 | 1.9 |
| College Station city | 26 054 | 31.3 | 24 691 | 30.6 | 2 422 | 124 | 2 298 | 2 531 | 3 997 | 4 180 | 30 452 | 1.3 | 497 | 1.6 |
| Conroe city | 14 378 | 75.0 | 13 145 | 47.3 | 2 000 | 244 | 1 836 | 2 382 | 5 800 | 6 632 | 21 664 | .8 | 735 | 3.4 |
| Corpus Christi city | 93 102 | 4.1 | 84 140 | 57.0 | 13 480 | 1 945 | 11 535 | 13 480 | 6 020 | 6 020 | 114 284 | –2.1 | 8 569 | 7.5 |
| Dallas city | 484 117 | 4.0 | 451 933 | 43.2 | 104 944 | 15 435 | 89 509 | 100 594 | 9 616 | 9 236 | 679 572 | 2.6 | 27 967 | 4.0 |
| Deer Park city | 9 831 | 8.7 | 8 615 | 79.3 | 545 | 30 | 515 | 507 | 1 751 | 1 849 | 17 403 | 7 | 566 | 3.2 |
| Del Rio city | 11 895 | 10.2 | 10 775 | 64.0 | 1 640 | 111 | 1 529 | 1 729 | 5 194 | 4 873 | 14 951 | –.6 | 1 053 | 7.0 |
| Denton city | 32 716 | 13.6 | 30 995 | 41.9 | 3 267 | 323 | 2 974 | 3 568 | 4 075 | 4 565 | 57 924 | 2.9 | 1 709 | 3.0 |
| DeSoto city | 14 989 | 33.9 | 13 709 | 72.2 | 1 412 | 129 | 1 283 | 1 446 | 3 961 | 4 048 | 21 681 | .7 | 484 | 2.2 |
| Duncanville city | 13 290 | –.5 | 12 896 | 71.7 | 1 605 | 152 | 1 453 | 1 601 | 4 376 | 4 312 | 24 205 | 2.7 | 603 | 2.5 |
| Edinburg city | 20 731 | 34.1 | 18 190 | 61.4 | 3 000 | 347 | 2 653 | 3 000 | 5 792 | 5 792 | 36 787 | 1.0 | 1 284 | 3.5 |
| El Paso city | 165 643 | 14.8 | 151 063 | 61.4 | 36 135 | 4 290 | 31 856 | 36 787 | 5 792 | 5 792 | 255 875 | –1.0 | 20 194 | 7.9 |
| Euless city | 20 136 | 17.6 | 19 216 | 43.8 | 1 624 | 112 | 1 512 | 1 892 | 3 938 | 3 912 | 30 394 | 2.1 | 661 | 2.2 |
| Fort Worth city | 211 025 | 8.0 | 185 078 | 55.9 | 37 264 | 4 246 | 33 108 | 35 481 | 7 488 | 7 488 | 276 782 | 2.1 | 11 459 | 4.2 |
| Galveston city | 25 197 | –5.0 | 21 970 | 53.8 | 4 268 | 1 040 | 3 228 | 4 268 | 6 220 | 6 220 | 29 282 | 2.3 | 1 452 | 5.0 |
| Garland city | 70 930 | 8.2 | 67 341 | 65.6 | 8 951 | 495 | 8 456 | 8 970 | 4 511 | 4 526 | 139 398 | 2.7 | 3 948 | 2.7 |
| Grand Prairie city | 49 425 | 19.9 | 47 791 | 61.2 | 6 698 | 465 | 6 233 | 6 972 | 5 639 | 6 187 | 85 604 | 2.5 | 2 138 | 3.5 |
| Grapevine city | 16 488 | 38.5 | 15 712 | 65.0 | 1 571 | 56 | 1 515 | 1 383 | 3 843 | 3 554 | 21 757 | 2.1 | 364 | 1.7 |
| Haltom City city | 15 719 | 12.0 | 14 927 | 69.5 | NA | NA | NA | 1 884 | NA | 4 980 | 21 419 | 2.1 | 655 | 3.1 |
| Harlingen city | 23 008 | 29.3 | 19 021 | 61.1 | 3 834 | 390 | 3 444 | 3 838 | 6 493 | 6 482 | 36 886 | 3.0 | 1 671 | 8.2 |
| Houston city | 782 099 | 7.7 | 717 945 | 45.8 | 131 774 | 21 517 | 110 257 | 127 817 | 7 273 | 7 318 | 1 927 564 | .5 | 92 265 | 5.1 |
| Huntsville city | 11 526 | 24.2 | 10 476 | 60.1 | 2 006 | 169 | 1 837 | 2 194 | 6 534 | 6 534 | 34 167 | 2.1 | 720 | 3.0 |
| Hurst city | 14 709 | 6.3 | 14 076 | 68.1 | 2 006 | 169 | 1 837 | 2 194 | 6 534 | 6 838 | 34 167 | 2.1 | 720 | 3.0 |
| Irving city | 60 293 | 13.0 | 56 241 | 57.2 | 9 212 | 737 | 8 475 | 9 494 | 5 099 | 5 180 | 116 636 | 2.7 | 3 540 | 2.9 |
| Killeen city | 35 343 | 33.7 | 32 447 | 46.4 | 5 010 | 529 | 4 481 | 5 354 | 8 119 | 6 659 | 27 673 | .2 | 1 434 | 5.2 |

**Question**

For the Spartanburg city, what is the Violent value for 1999 under "Serious crimes known to police"?

**GPT4o-Text Answer**
623

**GPT4o-vision Answer**
3,303

Fig. 43. **gpt-4o-vision** is wrong but **gpt-4o** text-only is correct.

**Finetuning Configuration Summary**

**Finetuning Type:** Full-parameter finetuning
**Max Sequence Length:** 4096 tokens
**Precision:** bfloat16 (bf16)
**Training Epochs:** 1
**Batch Size:** 2 per device
**Gradient Accumulation:** 2 steps
**Learning Rate:** 5e-7
**Learning Rate Schedule:** Cosine with min LR = 5e-8

The dataset itself, with the exact train + validation + test splits used in our finetuning experiments, can be found on HuggingFace[3].

[3]https://huggingface.co/datasets/qcri-ai/HCTQA